Software Design Document

for

OpenBench

Design of Release 1 against the Software Requirements Specification, version 0.3

Version 1.0

2026-10-05

Prepared by

Jai Nana Brian Lujan

Fotih Turopov Yerassyl Shnazbayev

CSDS 393 Software Engineering

Case Western Reserve University

Revision History

Name	Date	Reason for changes	Reviewed by	Version
J. Nana	2026-10-05	First complete draft: architecture, module specifications, runtime scenarios, data, API, crosscutting concerns, evolution, traceability.	Team inspection, Appendix C	1.0

How to read this document. Section 1 states the baseline and the principles; Section 2 gives the architecture in four figures; Section 3 allocates every responsibility to a module; Section 4 specifies each module with one template; Section 5 shows the transactions, the state machines, and seven scenarios as sequence diagrams; Sections 6 to 8 give the schema, the API, and the crosscutting concerns; Section 9 says how the design evolves. Appendix A traces every requirement of the SRS to a module and a section. Capitalized words such as Request, Hold, and Loan have the meaning given in the glossary, Appendix B; code identifiers are set in `monospace`; module names in **bold sans-serif**.

Contents

1 Introduction **4**
1.1 Purpose and Scope . . . 4
1.2 Requirements Baseline and Changes . . . 4
1.3 Readers and Roadmap . . . 4
1.4 Design Principles . . . 5
1.5 Notation . . . 5
1.6 Conventions . . . 6

2 Architecture **7**
2.1 Context . . . 7
2.2 Architectural Style and the Dependency Rule . . . 7
2.3 Module Map . . . 8
2.4 Deployment . . . 11
2.5 How the Quality Attributes Are Met . . . 11
2.6 Decisions and Alternatives . . . 12

3 Problem Decomposition **15**
3.1 Responsibilities . . . 15
3.2 Subproblems and How Each Was Broken Down . . . 17

4 Module Specifications **19**
4.1 Members (`members`) . . . 19
4.2 Inventory (`inventory`) . . . 21
4.3 Settings (`settings`) . . . 23
4.4 Notifications (`notifications`) . . . 26
4.5 Allocation (`allocation`) . . . 27
4.6 Ledger (`ledger`) . . . 32
4.7 Requests (`requests`) . . . 35
4.8 Dashboard (`dashboard`) . . . 40
4.9 Jobs (`jobs`) . . . 42
4.10 Web client (`web`) . . . 44

5 Runtime View **47**
5.1 Transactions, locks, and the correctness guarantee . . . 47
5.2 State machines . . . 48
5.3 Scenarios . . . 49

6 Data Design **62**
6.1 Schema by owner . . . 62
6.2 Constraints that carry business rules . . . 65

6.3 Migrations and the test database . . . 67

7 REST API **68**
7.1 Conventions . . . 68
7.2 Authentication and Sessions . . . 69
7.3 Endpoints . . . 69

8 Crosscutting Concerns **72**
8.1 Authorization and Data Scoping . . . 72
8.2 Audit Trail . . . 72
8.3 Notifications . . . 73
8.4 Error Taxonomy and Validation . . . 73
8.5 Configuration and Time . . . 74
8.6 Testing Strategy . . . 74
8.7 Logging . . . 75

9 Maintenance and Evolution **76**
9.1 Extension Points . . . 76
9.2 Assumptions That Would Break the Design . . . 77
9.3 Risks . . . 78

10 References **79**

A Traceability **80**

B Glossary **82**

C Inspection Report **84**

List of Figures

1 System context. One system, five actors that are roles held by Members, and the Clock, which stands for the scheduled jobs inside the server. No external system is connected in Release 1. . . . 7
2 Modules and their dependencies (UML component diagram). A solid arrow is a call to the target's application services. The ball on **Allocation** and the socket on **Inventory** show the port `ReductionGuard`, declared by Inventory and implemented by Allocation. The dashed arrows labeled read-only are **Dashboard**'s SQL access through views, the one exception to calling services. . . . 9
3 Layers (UML package diagram). Dependencies point only downward; the inset shows the four layers repeated inside every module, from controller to repository. . . . 10
4 Deployment (UML deployment diagram). One server runs one container holding the Spring Boot jar; embedded Tomcat serves `/api` and the single-page application, and the scheduled jobs run in the same process. PostgreSQL 16 holds all data. The nginx node is optional and only terminates TLS. . . . 14
5 Entities and application services of the three foundation modules, **Members**, **Inventory**, and **Settings**. Each package is one module; a module's entities are package-private and reached only through the services shown. The `ReductionGuard` socket on **Inventory** is the one port of the design (realized by **Allocation**, Figure 6). . . . 25
6 Classes of **Allocation**: the `Allocator` interface with its five operations, the three capability interfaces, the registry, the three allocators and the capabilities each realizes, the shared lock and query services, and the six entities the module owns. Dashed arrows are dependencies; hollow-triangle dashed arrows are realizations. . . . 27
7 Classes of **Ledger**: `LedgerService`, the pure `Pricing` functions, the `Account` with its three numbers, the append-only `Transaction` with its four kinds, and the value types `Quote`, `PriceBand`, and `RefundSchedule`. . . . 32
8 Classes of **Requests**: the Draft-editing and Suggestion services, the orchestrating `RequestLifecycleService`, the entities and state enums, and the module's dependencies on `Allocator`, `AllocatorRegistry`, `AllocationLock`, `LedgerService`, and `Notifier`. . . . 35

9 Routes of the web client grouped by the role that uses them, and the API groups each route calls (dashed arrows). The client holds no business rules: every Quote, Shortlist, and state change comes from the API. . . . 46
10 Request state machine, unchanged from the SRS. Withdraw is drawn once, from Draft; it is also allowed from Pending and Waiting. Table 11 gives the guard and side effects of each arrow. . . . 50
11 Request Line state machine. Exclusive and Metered lines move through Held, Allocated, Waiting, and Bumped; Matched lines through AwaitingApproval, Approved, and Loaned. Dropped (Optional lines) and Canceled are reachable from any state and drawn once. . . . 51
12 S1, submit a Request (UC-4). The `alt` box shows the three outcomes of evaluation (allocated at once, Pending with Holds and approvals, Waiting); the refusal for an insufficient Available Balance comes before any lock. . . . 52
13 S2, the last Required approval allocates the Request (UC-6). Holds commit and Charges, Loans, and Earnings post in one transaction; the `alt` box shows a Candidate who is no longer eligible. . . . 53
14 S3, refusal and replacement (UC-6, UC-5). The refusal of a Required line releases the Holds, which are placed again when a replacement is picked, without restarting the Objection Window; a refused Optional line keeps them; the `alt` box shows the Request Refused when no Candidate is left. . . . 55
15 S4, the Objection Window lapses (UC-7). For a Pending Request the job releases every Hold and the Request moves to Waiting; for a Pending or Waiting one its Outstanding approvals escalate to the Capacity Planner, whose decision continues as S2 or S3. . . . 56
16 S5, a Priority line Bumps a standard line (UC-8). Shows the flush before the insert, the full Refund for an Allocated Bumped line or the release of held credits for a Held one, the Waitlist head for a Bumped Required line, and, in the `alt` box, the bump cutoff forbidding the Bump. . . . 58
17 S6, Release and waitlist fill (UC-9, UC-10). The Refund follows the Refund Schedule, and the freed capacity is offered to the Waitlist in the same transaction; the `alt` box places Holds for a waiting Request that still needs approvals. . . . 59
18 S7, Period rollover (UC-11). Granted credits expire, the new Grants post, and Managers are notified; the start-up catch-up runs the same steps after a missed boundary. . . . 60
19 Database schema, foundation modules: above, the tables owned by **Members** and **Notifications**; below, those owned by **Inventory** and **Settings**, and the three read-only **Dashboard** views. Lines are foreign keys with crow's-foot cardinalities; only the owning module writes a table. . . . 62
20 Database schema, core modules: above, the tables owned by **Requests** and **Ledger**; below, those owned by **Allocation**. Foundation tables they reference appear as stubs; a key that crosses a module boundary is a read-only reference. The notes quote the two exclusion constraints and the append-only rule. . . . 63

1 Introduction

1.1 Purpose and Scope

This document specifies the design of Release 1 of OpenBench, the internal marketplace in which a Manager posts one Request for people, Exclusive resources, and Metered capacity, and the system allocates each Request Line under the policy of its Allocation Class and settles it in credits. It is the bridge from the requirements to the code: the team implements from it, the tests are derived from its interfaces and scenarios, and reviewers check it against the SRS.

The design covers every Release 1 requirement of the SRS: the 41 functional requirements, the design and implementation constraints, the interface requirements, and the performance, security, and quality requirements. It divides the system into nine back-end modules and a web client (Section 2), gives every responsibility to exactly one owning module (Section 3), and specifies each module down to Java signatures, owned tables, invariants, and failure behavior (Section 4). The six Stretch requirements are designed only as extension points, and the Future items are named only so that the design leaves room for them (Section 9). The code itself, a separate test plan, and the project schedule are outside this document.

1.2 Requirements Baseline and Changes

The baseline is the OpenBench Software Requirements Specification, version 0.3 of 2026-10-05. Identifiers are those of its short edition: 41 Release 1 and 6 Stretch functional requirements in dot notation (for example **Lifecycle.Hold.Place**), twelve use cases UC-1 to UC-12, and the CO, UI, SI, CI, PE, SE, QA, UD, AS, and DE identifiers. The full edition uses the same functional identifiers. Version 0.3 differs from version 0.2, the edition that was graded, by the three requirements changes below. Each is recorded in the revision history of the SRS.

Change	What changed	Why	Where it lands
RC-1 One Loan at a time	A Member with a Loan overlapping any day of a Matched line's window is not a Candidate for that line. The Shortlist is ordered by fewest person-days lent in the current Period, ties by name. A Loan that ends early frees the Person from the day after its new last day (**Profile.Availability**, **Matched.Shortlist.Build**).	Grader feedback. Version 0.2 kept such Members on the Shortlist and only ranked them by overlapping Loans, which left open whether a Person could be lent twice at once. A Person is lent whole, so an overlap now removes eligibility. Because no Candidate then has an overlapping Loan, the order uses person-days lent instead, which still spreads Loans across Members.	Shortlist eligibility in `MatchedAllocator` (Section 4.5), re-checked at approval and at Loan formation; exclusion constraint on `loan` (Section 6).
RC-2 Inspection participation	The inspection record of the SRS states that all four team members read and inspected it.	Grader feedback asked that the whole-team inspection be explicit.	No design effect. This document's inspection record (Appendix C) follows the same rule.
RC-3 Test database	Automated tests run on PostgreSQL 16 in a container started by the test suite (Testcontainers), with the same migrations as production. The in-memory H2 database is dropped (**CO-2**, **SI-1**).	Design decision. The design enforces correctness with PostgreSQL features that H2 does not have: a partial exclusion constraint over time ranges, row locks with PostgreSQL's READ COMMITTED behavior, and a native hourly-sum query. Tests on H2 would test a different system.	ADR-0006; schema and migrations (Section 6); testing strategy (Section 8).

1.3 Readers and Roadmap

Three kinds of reader use this document. An *implementer* builds one module and needs its interface, its data, and the rules it must not break. A *tester* writes the automated tests that **QA-7** requires and needs the invariants, the failure behavior, and the scenarios. A *reviewer* (a teammate at inspection, or the grader) checks the design against the SRS and needs the rationale and the traceability. In the table, • marks a section that reader reads closely and ○ one they consult when needed.

Section	What it answers	Impl.	Test	Review
1 Introduction	What is designed, against which requirements, and how to read the figures	○	○	●
2 Architecture	What the parts are, how they may depend on each other, where they run, how each quality attribute is met, and why	●	○	●
3 Problem Decomposition	Which module owns each responsibility and requirement, and how the hard parts were broken down	●	●	●
4 Module Specifications	What each module provides, requires, owns, guarantees, and hides	●	●	○
5 Runtime behavior	What one transaction locks, which states a Request and a line pass through, and how the seven scenarios run	●	●	○
6 Data	Which tables exist, which module writes each, and which constraints carry rules	●	●	○
7 API	Which endpoints exist, who may call them, and which errors they return	●	●	○
8 Crosscutting concerns	How authorization, audit, notifications, errors, time, testing, and logging work everywhere	●	●	○
9 Maintenance and evolution	Where Stretch and Future items land, which assumptions would break the design, which risks remain	○	○	●
Appendix A Traceability	Where each SRS identifier is satisfied	○	●	●
Appendix B Glossary	What each domain and design term means	○	○	○
Appendix C Inspection	How the team reviewed this document			●

1.4 Design Principles

- **Simplicity first.** The team's standing rule is the simplest architecture and program that meets the SRS: one process, one database, direct method calls, no message broker, no event sourcing, no microservices. This document adds a mechanism only where a requirement needs it, and Section 2 names the requirement each time (the one port, the one read-only module, the two scheduled jobs).
- **Modularity and information hiding.** Each of the nine modules hides its entities and tables behind its application services; every module specification ends with an *Information hiding* field that states what the other modules must not know.
- **Low coupling and high cohesion.** Dependencies point down a fixed layer order with no cycles, and each module owns one cluster of the vocabulary: **Allocation** decides all capacity and **Ledger** all credit arithmetic, and neither calls the other.
- **Separation of concerns.** What state a Request is in (**Requests**), whether capacity exists (**Allocation**), and what it costs (**Ledger**) are answered by different modules; inside each module HTTP, rules, and persistence sit in separate layers, and pricing is a pure function.
- **Correctness enforced in the database.** Invariants that must survive concurrent users (no double-allocated instance, no oversold hour, no overlapping Loan, an append-only Ledger) are database constraints, row locks, or grants, not only application checks (ADR-0006).
- **Uniformity of patterns.** Every module has the same four inner layers, every user or job action is one transactional service method, every module is specified with the same template, every error is a problem response with a stable code, and every timed action goes through the owning module's application service, like a user action, with the system recorded as the actor.

1.5 Notation

The figures are UML 2 diagrams drawn with PlantUML from sources kept with this document: a context diagram (Section 2), a component diagram and a package diagram of the modules, a deployment diagram, class diagrams per module (Section 4), state machine diagrams and one sequence diagram per scenario (Section 5), and an entity-relationship diagram of the schema (Section 6). Color carries no meaning, so every figure reads in grayscale. Four elements go beyond plain boxes and arrows:

- **Interface lollipop and socket.** In the component diagram the port `ReductionGuard` is drawn as a provided interface (a ball, or lollipop) on **Allocation** and a required interface (a socket) on **Inventory**. Inventory declares the interface in its own package and calls it; Allocation implements it. The call at run time goes from Inventory to Allocation, but the compile-time dependency goes from Allocation to Inventory, so the layer order holds.
- **Dashed arrows labeled read-only.** A solid arrow is a call to another module's application service. The dashed

arrows labeled *read-only* from **Dashboard** stand for SQL reads through views over other modules' tables, the one documented exception to the rule that a module reaches another module's data only through its services.

- **`alt` and `opt` frames.** In sequence diagrams an `alt` frame holds alternative branches, each with its guard in brackets; failure branches are always drawn as an `alt` branch, never left out. An `opt` frame holds a step that runs only when its guard holds.
- **Numbered messages.** Every message in a sequence diagram is numbered in order, so the text can cite "message 7". Message names are the operation names of Section 4 exactly, and dashed arrows are returns.

1.6 Conventions

- **Vocabulary.** Capitalized words such as Request, Hold, Loan, Shortlist, Waitlist, and Refund have the meaning given in the glossary (Appendix B), taken from the project vocabulary file. Credits are for showback only, so no money words are used.
- **Requirement identifiers** are set in bold. Functional requirements keep their dot notation, for example **Matched.Shortlist.Build**. Other identifiers print with the numbers the SRS gives them, for example **QA-1**; the table below gives the key each number stands for in the sources (**QA-1** is `QA-correct`).
- **Design identifiers.** Responsibilities are **R1** to **R34** (Section 3), requirements changes **RC-1** to **RC-3** (above), architecture decisions D-A to D-L and their records ADR-0001 to ADR-0007 (Section 2).
- **Code.** Identifiers are set in `monospace`, module names in **bold sans-serif**. Each module is one Java package `edu.cwru.openbench.<module>`; signatures are Java 17; every application-service method runs in one transaction unless marked *(read-only)*. Tables and columns are `snake_case`.
- **Time.** Instants are `timestamptz`; windows are half-open, [*start*, *end*); Loan days and availability ranges are inclusive dates. All windows, Price Bands, and Periods are in the company's one time zone.

Class	Keys with the number the SRS prints
CO	stack 1, db 2, api 3, tx 4, jobs 5, auth 6, plugin 7
UI, SI, CI	UI: draft 1, shortlist 2, inventory 3; SI: db 1, api 2; CI: https 1, inapp 2
PE, SE	PE: scale 1, page 2, shortlist 3, jobs 4; SE: roles 1, password 2, data 3, visibility 4
QA	correct 1, atomic 2, audit 3, available 4, usable 5, extensible 6, retention 7
UD, AS, DE	UD: guide 1, admin 2, help 3; AS: planner 1, inventory 2, honest 3, calendar 4; DE: platform 1, tooling 2
LI	company 1, money 2, integrations 3, consent 4

2 Architecture

2.1 Context

Figure 1 places OpenBench among the people who use it. The five actors are roles, not separate populations: every actor is a Member, and the Administrator may give one Member several roles, for example a Manager who is also the Capacity Planner. The Clock is not a person. It stands for the work the server does on its own when time passes (Objection Window lapse, Waitlist expiry, Period rollover), which runs inside the same process (**CO-5**). Release 1 has no external integrations (**LI-3**): no calendar, HR system, identity provider, cluster scheduler, or mail server. The browser is the only client, and every flow in the figure is a REST/JSON call over HTTPS (**SI-2**, **CI-1**).

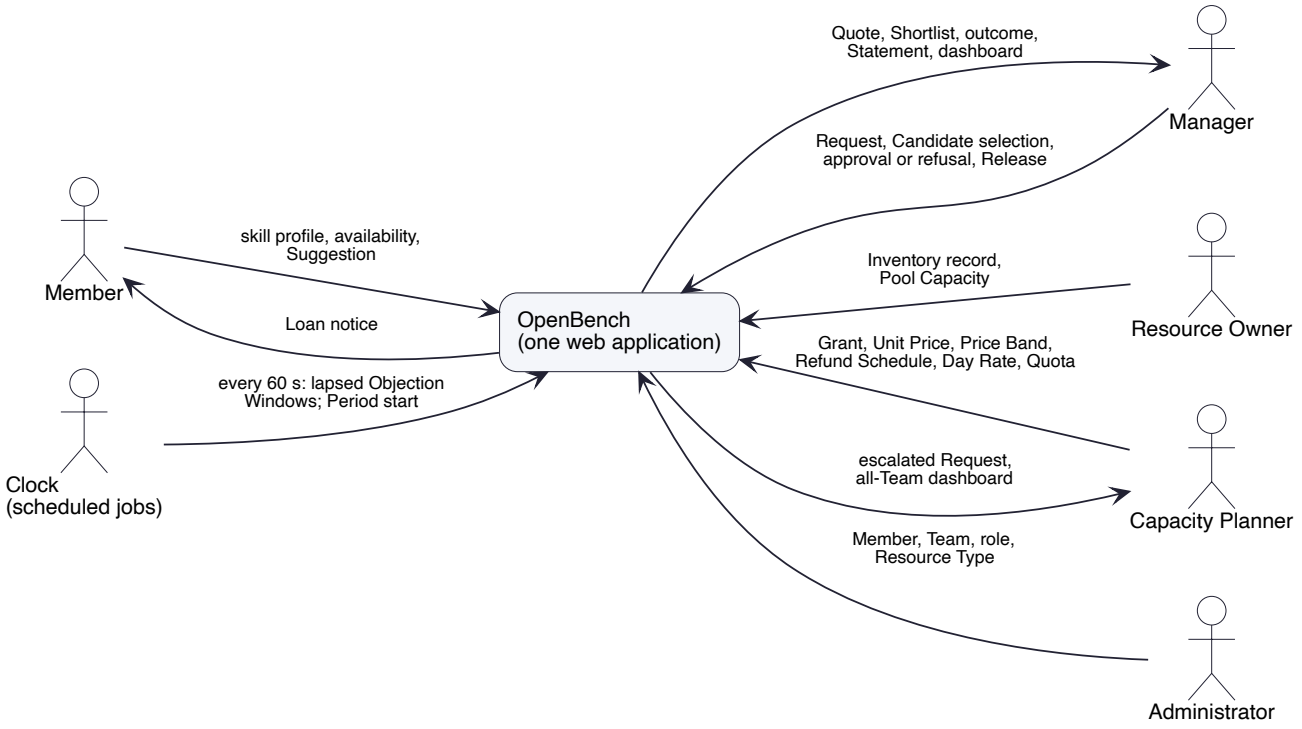


Figure 1: System context. One system, five actors that are roles held by Members, and the Clock, which stands for the scheduled jobs inside the server. No external system is connected in Release 1.

Each actor reaches the system through the module that owns what it touches. A Member maintains a profile in **Members** and makes Suggestions in **Requests**; a Manager drafts, submits, selects Candidates, approves or refuses Loans, and Releases lines through **Requests**, and reads the Statement and dashboard from **Ledger** and **Dashboard**; a Resource Owner maintains **Inventory** and reads holder timelines from **Allocation**; the Capacity Planner sets policy in **Settings**, posts Grants through **Ledger**, and decides escalations through **Requests**; the Administrator manages **Members** and the Resource Types of **Inventory**; and the Clock drives **Jobs**.

2.2 Architectural Style and the Dependency Rule

D-A Modular monolith (ADR-0005). OpenBench is one Spring Boot 3 application (Java 17, built with Maven), one PostgreSQL 16 database, and one React single-page application served by the same process. The back end has nine modules, one Java package each under `edu.cwru.openbench.<module>`: `members`, `inventory`, `settings`, `notifications`, `allocation`, `ledger`, `requests`, `dashboard`, and `jobs`. The web client is the tenth component. Modules collaborate by calling each other's application services (Spring beans) directly, inside the caller's transaction. There are no events between modules and no message broker.

The style follows from the two requirements that cannot bend: Holds commit and Charges and Earnings post in one transaction (**CO-4**, **QA-2**), and concurrent submissions never double-allocate or oversell (**QA-1**). With direct calls in one process, the calling service's transaction spans **Requests**, **Allocation**, and **Ledger**, so atomicity needs no compensation logic and no outbox; about 50 Managers on one server need no second process.

D-B The dependency rule keeps the modules apart inside one deployable:

(1) A module's only public surface is its application service classes and the DTOs they take and return. Entities and repositories are package-private.
(2) A module writes only its own tables (the sole-writer rule). Foreign keys may point into another module's tables as read-only references.
(3) Dependencies point downward in the layer order of D-C, with no cycles. Two modules in the same layer may depend on each other in one direction only, as the module map in Section 2.3 lists.
(4) The one upward hook is the port `ReductionGuard`: **Inventory** declares it and **Allocation** implements it, so Inventory can refuse a capacity reduction below future usage without depending on Allocation.
(5) **Dashboard** is read-only. It may query any table through SQL views, and it writes nothing.

One ArchUnit test class checks rules (1) and (3) and fails the build on a violation. Code review covers rule (2) for native SQL and rule (5), which an import check cannot see.

D-C Layers, top to bottom: the web client; then **Requests**, **Dashboard**, and **Jobs**; then **Allocation** and **Ledger**; then **Inventory**, **Members**, **Settings**, and **Notifications**. Inside every module the same four layers repeat: controller (REST), application service, domain (entities and pure rules), and repository (Spring Data JPA). A controller calls only its own module's services; a service may call its own domain and repositories and other modules' services. Figure 2 shows every module and dependency; Figure 3 shows the layers.

Three packages are shared infrastructure, not modules. `security` holds the Spring Security configuration, which loads users from **Members**; `common` holds the error types and their mapping to problem responses, the value types `Credits`, `DateRange`, and `TimeWindow`, and the typed ids; `config` holds the remaining wiring. Any module may use `common`, which uses no module.

2.3 Module Map

The modules are listed bottom-up, in layer order. Each paragraph gives the purpose, the tables owned, and the dependencies; Section 4 specifies each in full.

Members (`members`, Section 4.1). Who the people are: Members, Teams with exactly one Manager each, roles, the skill list, each Member's skills and availability ranges, and the login identity that Spring Security authenticates against. Owns `team`, `member`, `member_role`, `skill`, `member_skill`, and `availability_range`. Depends on nothing. It knows nothing about Loans: the one-Loan-at-a-time rule of **RC-1** is enforced by **Allocation**.

Inventory (`inventory`, Section 4.2). What exists: Resource Types as data (name, Allocation Class, attribute definitions) and Resources (instances, units, owner). Pool Capacity is computed as the sum of the units of a Metered type's unretired Resources, never stored. Owns `resource_type`, `resource_type_attribute`, `resource`, and `resource_attribute_value`. Depends on **Members** for owner identity. It declares the port `ReductionGuard` and calls it before any retirement or reduction of units. Its `resource_type` rows also serve as the serialization point for allocation: **Allocation** locks them but never writes them.

Settings (`settings`, Section 4.3). Every Capacity Planner setting as data, with a change log: the Period, each Team's Grant per Period, each type's Unit Price, Price Bands, Refund Schedule, and bump cutoff, the Day Rate, Quotas, the priority premium, and the Objection Window. Owns `period`, `grant_plan`, `type_pricing`, `price_band`, `refund_step`, `quota`, `planner_setting`, and `setting_change`. Depends on **Inventory** (type ids) and **Members** (Team ids). Other modules read one typed snapshot, `PlannerSettings`.

Notifications (`notifications`, Section 4.4). In-app notifications: write one for a Member, list the unread ones, mark one read. A `Delivery` interface has one implementation now, `InAppDelivery`, and takes `EmailDelivery` in the Stretch release. Owns `notification`. Depends on **Members**. A notification is written in the transaction of the change it reports, so it exists exactly when that change committed.

Allocation (`allocation`, Section 4.5). Whether capacity exists, and who has it. It holds the `Allocator` interface, the `AllocatorRegistry`, and the three allocators `ExclusiveAllocator`, `MeteredAllocator`, and `MatchedAllocator`; Holds (a state on occurrence and Metered rows), the Waitlist, Shortlists, Candidates, approvals, and Loans; the type lock `AllocationLock`; and the read-only `AllocationQueries`. It implements `ReductionGuard`. Owns `exclusive_occurrence`, `metered_allocation`, `waitlist_entry`, `candidate`, `approval`, and `loan`. Depends on **Inventory**, **Members**, **Settings**, and **Notifications**. It knows nothing about credits or Request states.

Ledger (`ledger`, Section 4.6). Everything about credits: Accounts (granted, earned, held), append-only Transactions of the four kinds with the granted and earned split, Statements, affordability, Period rollover, and `Pricing`, a pure class that computes Quotes and Refunds. Owns `account`, `transaction`, `held_credit`, and `period_close`. Depends on **Settings**, **Members**, and **Notifications** (rollover and mid-Period Grant notices). It is the only writer of credit figures and knows nothing about capacity.

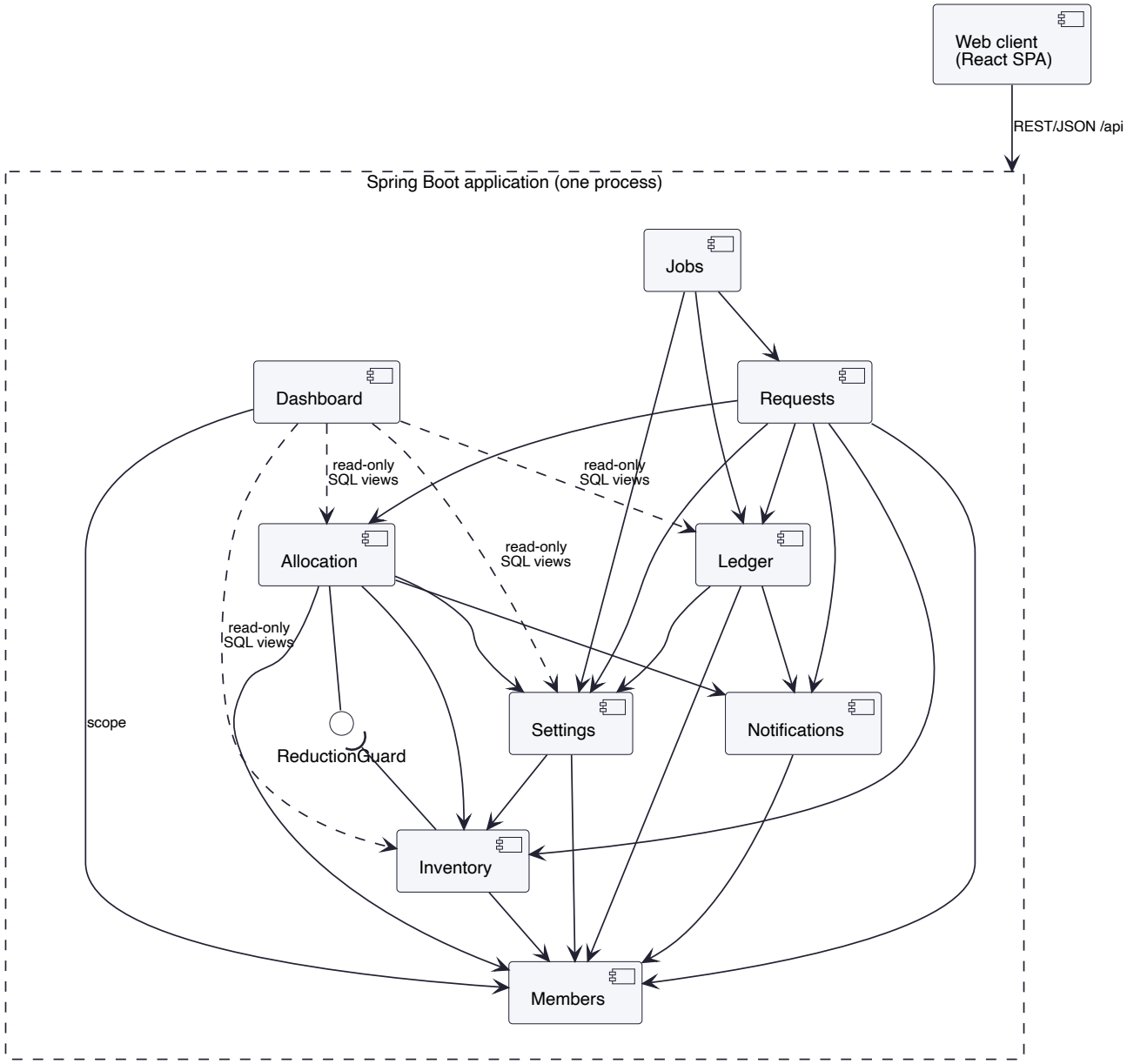


Figure 2: Modules and their dependencies (UML component diagram). A solid arrow is a call to the target's application services. The ball on **Allocation** and the socket on **Inventory** show the port `ReductionGuard`, declared by Inventory and implemented by Allocation. The dashed arrows labeled read-only are **Dashboard**'s SQL access through views, the one exception to calling services.

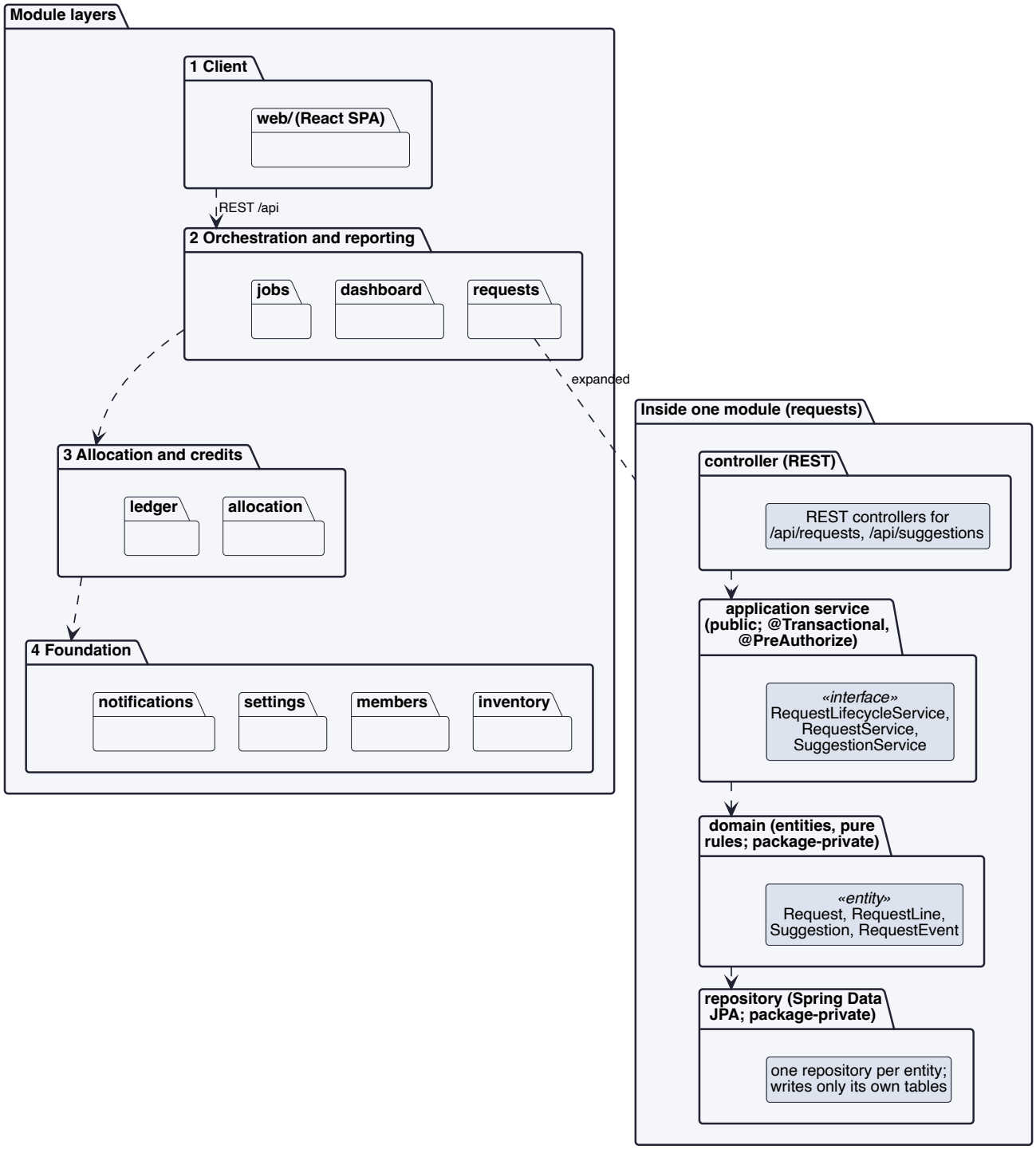


Figure 3: Layers (UML package diagram). Dependencies point only downward; the inset shows the four layers repeated inside every module, from controller to repository.

Requests (`requests`, Section 4.7). The Request from Draft to its final state: Requests, Request Lines, and Suggestions; Draft editing; submission with the affordability check and locked Quotes; the state machine, driven by the orchestrating `RequestLifecycleService`; the state history; escalations; Release; and re-evaluation of Waiting Requests. Owns `request`, `request_line`, `suggestion`, and `request_event`. Depends on **Allocation**, **Ledger**, **Inventory**, **Members**, **Settings**, and **Notifications**. It is the only module that changes a Request's state and the only one that calls both **Allocation** and **Ledger**, so a transition's capacity effects and credit effects always commit together.

Dashboard (`dashboard`, Section 4.8). Showback figures: utilization and idle capacity per Resource Type and Period, waiting-line counts, credits per Team and Period, and person-days lent and borrowed, each scoped to the viewer's roles. Owns no tables; reads the SQL views `v_type_hours`, `v_team_credits`, and `v_waiting` over other modules' tables under rule (5). Depends on **Members** for the viewer's scope.

Jobs (`jobs`, Section 4.9). Timed work: `DueWorkJob`, every 60 seconds, and `PeriodRolloverJob`, daily and at start-up. It also declares the `java.time.Clock` bean, set to the company time zone, that every module injects to read the time; the type is the JDK's, so injecting it creates no dependency on **Jobs**. Owns nothing. Depends on **Requests**, **Ledger**, and **Settings**. Each job acts only through the application services of **Requests** and **Ledger**, like a user action, with the system recorded as the actor.

Web client (`web/`, Section 4.10). The React single-page application: one route per screen of the SRS (Requests with the Draft editor, Suggestions, approvals, Shortlists, Inventory, dashboard, Statement, settings and escalations, administration, profile), shown according to the viewer's roles. Depends only on the REST API (**CO-3**). It never computes a Quote but fetches it, so the Quote shown is the Quote locked.

2.4 Deployment

Figure 4 shows the one deployment of Release 1. A single on-premises server runs one Docker container holding the Spring Boot executable jar. Its embedded Tomcat serves the REST API under `/api` and the compiled single-page application from the same origin, so the browser needs no cross-origin access, and the scheduled jobs run as threads of the same process (**CO-5**). Maven builds the front end with `frontend-maven-plugin` into the jar's static resources, so one artifact holds the whole system (**CO-1**). The build and documentation toolchain is pinned in the repository's Nix flake (**DE-2**).

The process talks to PostgreSQL 16 over JDBC. The database may run in its own container on the same host or as an existing installation; it needs the `btree_gist` extension, which the first migration creates. At start-up Flyway applies the SQL migrations and Hibernate validates the schema against the entities (**SI-1**). An optional nginx reverse proxy terminates TLS and redirects plain HTTP to HTTPS (**CI-1**); without it, Spring Boot's own TLS configuration does the same.

There is exactly one instance: the SRS sets no high-availability requirement (**LI-1**, **QA-4**), and the two jobs assume no second copy of themselves (Section 9 gives the job lock a second instance would need). The company's one time zone is set once in configuration and used by the `Clock` bean for windows, Price Bands, and Period boundaries. Database address, credentials, time zone, and TLS settings come from the container's environment. Tests start PostgreSQL 16 with the same migrations through Testcontainers (**RC-3**).

2.5 How the Quality Attributes Are Met

Each keyed requirement that shapes the architecture is met by a named mechanism, detailed in the section given. The remaining identifiers (**PE-1** and the UI, UD, AS, and DE identifiers) are traced in Appendix A.

Requirement	Mechanism	Detailed in
QA-1	Exclusive: a partial exclusion constraint on `exclusive_occurrence` over (Resource, time range) for Held and Allocated rows. Metered: lock each touched `resource_type` row in ascending id order, then check every hour against Pool Capacity and Quota, at READ COMMITTED. Loans: an exclusion constraint on `loan`.	5, 6, ADR-0006
QA-2	One `@Transactional` method of `RequestLifecycleService` commits the Holds and posts the Charges and Earnings, by direct calls.	4.7, 5
QA-3	A `request_event` row for every state change, in the same transaction; `setting_change`; `transaction` is append-only, with no UPDATE or DELETE grant.	8, 6
QA-6	A Resource Type is a row; a new Allocation Class is one new `Allocator` bean found by `AllocatorRegistry`.	4.5, ADR-0007

continues on next page

Quality attributes (continued)

Requirement	Mechanism	Detailed in
QA-5	One Draft screen with server Quotes, sums, Available Balance, and shortfall; menus per role; help on every field; a test with three outside users.	4.10, 8
QA-4	One instance, restarted by the container runtime; both jobs catch up at start-up; current desktop browsers only.	4.9, 4.10
QA-7	No operation deletes a Request, history, Suggestion, or Transaction; tests run on PostgreSQL 16 in Testcontainers (**RC-3**).	6, 8
PE-2	One process, so a page is one round trip and a few indexed queries, scoped by Team in SQL; list endpoints are paginated.	6, 7
PE-3	One SQL query over skills, availability ranges, and Loans, ordered by person-days lent.	4.5
PE-4	`DueWorkJob` every 60 s against a five-minute bound; `PeriodRolloverJob` at 00:05 and at start-up against a fifteen-minute bound; one transaction per Request.	4.9, 5
SE-1	Roles as granted authorities, checked by `@PreAuthorize` on application services: one path for the client and the API.	8, 7
SE-2	bcrypt hashes; HTTPS; a session cookie that expires after 12 hours idle; every login attempt logged.	7, 8
SE-3	`member` holds only name, email, username, password hash, Team, and an active flag.	4.1, 6
SE-4	The caller's scope (own record, Team, owned Resources, or the whole company) is a SQL condition of every query.	8
CO-4	As **QA-2**.	5
CO-5	Spring `@Scheduled` methods in **Jobs**, inside the server process.	4.9
CO-7	`Allocator` plus the optional `Waitlistable`, `Bumpable`, and `ApprovalDriven`, found through `AllocatorRegistry`.	4.5, ADR-0007
CO-3	The client calls only `/api`; authorization lives in the services; OpenAPI at `/api/docs`.	7, 4.10

The guarantee behind **QA-1**, stated once here and relied on in Section 5:

> *Every allocation decision runs in one PostgreSQL READ COMMITTED transaction that first locks the row of each Resource Type it touches, in ascending id order, so checks and writes for the same type never interleave; an exclusion constraint additionally makes it impossible for two Held or Allocated occurrences on the same Exclusive instance to overlap.*

2.6 Decisions and Alternatives

Each decision lists the alternatives weighed and the reason for the choice. Decisions that are hard to reverse have a record in `docs/adr/`; the first four records date from the SRS and fix rules that the design implements.

Decision	Alternatives considered	Why this one	Record
D-A Modular monolith with direct calls	Microservices; events between modules; hexagonal ports everywhere; Spring Modulith	One transaction spans the modules, so **CO-4** needs no compensation; events would split it or hide the call order; about 50 Managers on one server need no distribution	ADR-0005
D-B The five-part dependency rule	Shared repositories; two-way calls; Dashboard calling every module's services	Private entities let a module change its tables; one writer per table keeps each invariant in one place; views spare Dashboard nine query interfaces	ADR-0005
D-C Layers of modules, and four layers inside each	Application-wide controller, service, and repository packages	Application-wide tiers scatter one concept across the code; per-module layers keep it together in a checkable order	ADR-0005
D-D Correctness in the database: exclusion constraint, type-row lock, hourly sum at READ COMMITTED; Flyway; Testcontainers	Checks in Java only; SERIALIZABLE with retry; advisory locks; hour buckets; a single-writer queue; H2	The constraint holds on every code path; the type lock is the per-type serialization **QA-1** names; REPEATABLE READ oversells and SERIALIZABLE fails unrelated types; buckets duplicate the allocations; H2 runs none of it	ADR-0006, ADR-0001

continues on next page

Decisions (continued)

Decision	Alternatives considered	Why this one	Record
D-E One guarantee sentence, quoted above	Concurrency rules stated per scenario	One checkable statement that a two-thread integration test asserts	ADR-0006
D-F One `Allocator` interface plus optional capabilities	One wide interface; three services chosen by class; a rules engine	Every class needs the same five steps; capabilities make a new class one bean (**CO-7**); choosing by class would edit the orchestrator for every new class	ADR-0007, ADR-0003
D-G One transaction per action; lock type rows ascending, then the Team's `account` row	A lock per instance; one global lock; notifications sent after commit	The type is the unit **QA-1** names; a fixed order prevents deadlock; the account lock keeps held credits honest; an in-app notification is a row written in the same transaction	ADR-0006, ADR-0002
D-H Explicit audit rows written by the changing module	Hibernate Envers; an event store; triggers	The module that makes the change records actor and reason; Envers records row versions without intent; an event store is the opposite of simple	(none)
D-I Polling jobs and an injected `Clock`	A timer per due item; cron; Quartz; scheduling in the database	Survives a restart with no job store and meets the five-minute bound with margin; **CO-5** rules out cron	ADR-0001
D-J Session login, roles checked on services, scoping in SQL, problem responses	JWT tokens; checks in controllers; filtering after the query; GraphQL	A same-origin client makes a session cookie the simplest secure choice, expired on the server; one authorization path (**SE-1**); other Teams' rows are never loaded	(none)
D-K React, Vite, TypeScript, TanStack Query, generated API types, built into the jar	Quotes computed in the browser; a global store; a component library; a separate client deployment	The Quote shown is the Quote locked (**Draft.Submit.Lock**); generated types keep client and API in step; one artifact to deploy	(none)
D-L `Pricing` as a pure function; the Quote locked on the line	Pricing inside each allocator; the Charge computed at commit	One formula for drafting and submission, tested without a database; the Charge equals the Quote whatever the settings become	ADR-0002, ADR-0003, ADR-0004

Architecture decision records. The records are kept in `docs/adr/` of the project repository.

- **ADR-0001 Holds bounded by the Objection Window.** Lines are Required or Optional; a Pending Request places uncharged Holds that block like allocations and never outlive the Objection Window.
- **ADR-0002 A two-part Account.** Granted credits expire at the end of the Period, earned credits carry over; a Charge draws granted credits first and records the split.
- **ADR-0003 Priority bought with a premium, not granted by rank.** A Priority line costs a premium, may Bump a standard Exclusive line outside the bump cutoff, and is never Bumped; the Bumped line gets a full Refund.
- **ADR-0004 Matched lines cost nothing.** Borrowing a Person is free; lending earns a flat Day Rate per person-day, posted when the Loan forms.
- **ADR-0005 One process, nine modules, direct calls.** A modular monolith with the dependency rule and layers of this section, enforced by an ArchUnit test.
- **ADR-0006 Allocation correctness enforced in PostgreSQL; tests run on PostgreSQL.** The exclusion constraint, the type-row lock at READ COMMITTED, Flyway migrations, and Testcontainers.
- **ADR-0007 One Allocator interface plus optional capability interfaces.** Five common operations; `Waitlistable`, `Bumpable`, and `ApprovalDriven` found through the registry.

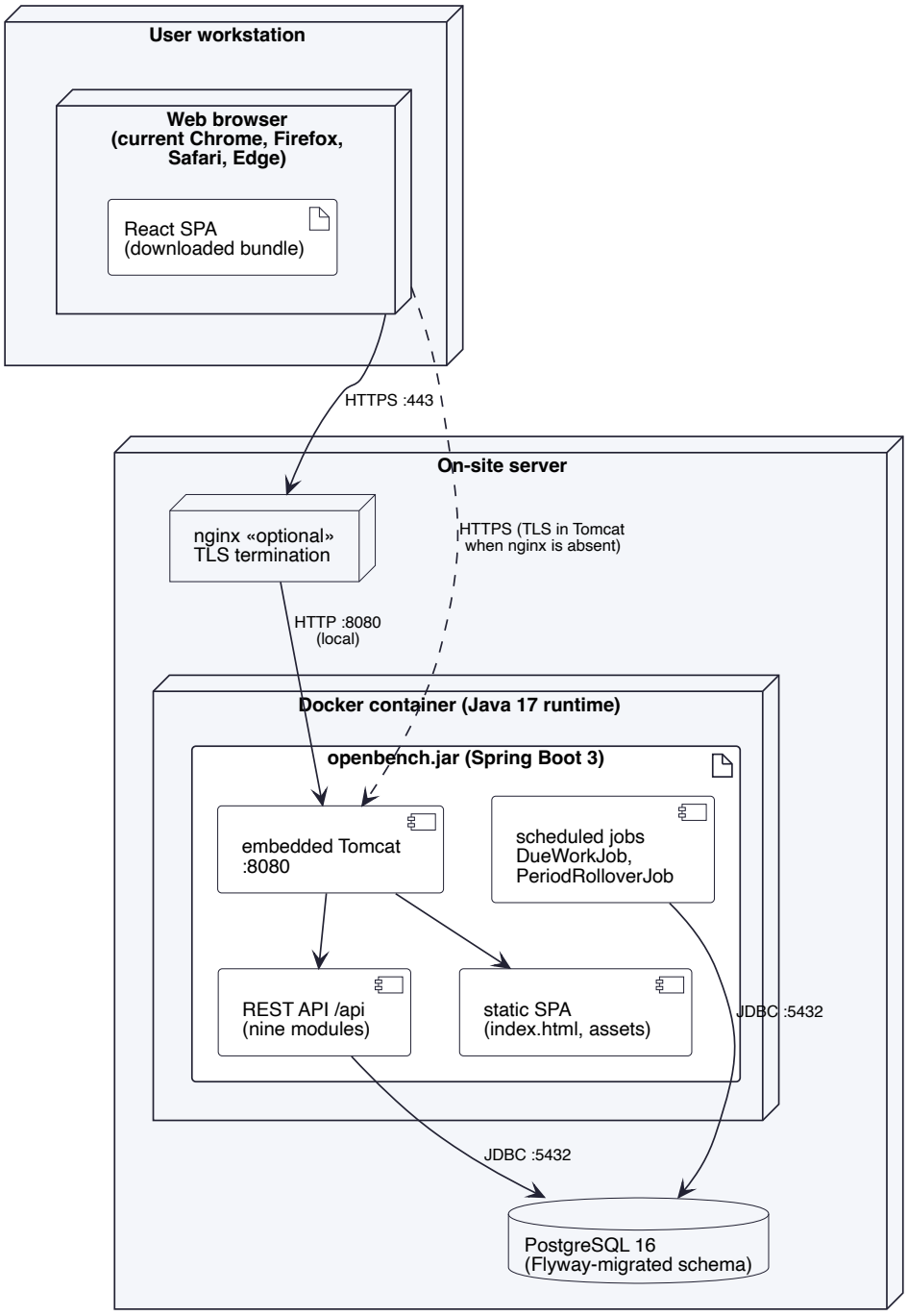


Figure 4: Deployment (UML deployment diagram). One server runs one container holding the Spring Boot jar; embedded Tomcat serves `/api` and the single-page application, and the scheduled jobs run in the same process. PostgreSQL 16 holds all data. The nginx node is optional and only terminates TLS.

3 Problem Decomposition

The problem was broken down in two passes. The first pass turned the twelve features of the SRS, plus the concerns that cut across them, into responsibilities small enough for one module to own. The second pass took the four parts that resist a clean split (three allocation policies, a Request whose parts resolve at different speeds, credits, and timed work) and decided how each is divided. Section 3.1 gives the result of the first pass, Section 3.2 the reasoning of the second.

3.1 Responsibilities

Every responsibility has exactly one owning module, the one that holds its data and enforces its rules; collaborators are the modules it calls or is called by. Where two modules share a responsibility, the first named owns the decision and the second carries it out. Every one of the 41 Release 1 functional requirements appears in at least one row, and Appendix A gives the reverse mapping. The module specifications of Section 4 cite these identifiers.

Id	Responsibility	SRS identifiers	Owner	Collaborators
Inventory and profiles				
R1	Define, change, and retire Resource Types as data, with one Allocation Class and typed attributes; protect the built-in type Person	**Inventory.Type.Define**, **QA-6**	**Inventory**	**Members**
R2	Keep Resources, their units, and Pool Capacity; refuse a retirement or reduction below future usage, naming the earliest hour	**Inventory.Resource.Add**	**Inventory**	**Allocation** (`ReductionGuard`)
R3	Show who holds a Resource in each part of a window; filter Resources by type, attribute, and free for a window	**Inventory.View.Holder**, **UI-3**	**Allocation**	**Inventory**, Web client
R4	Keep Members, Teams with exactly one Manager, roles, the skill list, each Member's skills and availability; answer who is available with which skills	**Admin.Member.Create**, **Profile.Skills**, **Profile.Availability**, **SE-3**	**Members**	`security`
Suggestions, drafting, and submission				
R5	Create Suggestions; accept one into a new or existing line, or decline it with a reason	**Suggest.Create**, **Suggest.Accept.Existing**	**Requests**	**Members**, **Notifications**
R6	Edit Drafts: add, edit, split, and remove lines; mark Required or Optional and Priority; refuse past windows and retired types	**Draft.Create**, **Draft.Line.Add**, **Draft.Line.Required**, **UI-1**	**Requests**	**Inventory**, **Settings**
R7	Price a line (Quote) and a Release (Refund fraction) with the pure class `Pricing`, from the line, the settings, and the lead time	**Ledger.Price.Formula**, **Draft.Quote.Line**, **Metered.Line.Form**, **Release.Refund**	**Ledger**	**Settings**, **Requests**
R8	Check affordability and submit: refuse a shortfall in Required lines, or any priced Required line while the Balance is negative; lock every Quote; record the submission time	**Draft.Quote.Line**, **Draft.Submit.Lock**, **Release.Loan.Prorate**	**Requests**	**Ledger**
Allocation per class				
R9	Decide Matched eligibility (skills, availability, other Team, active, no overlapping Loan) and build the Shortlist least-lent first; form Loans, one per Person at a time	**Matched.Shortlist.Build**, **Profile.Availability**, **Matched.Loan.Form**, **PE-3**, **UI-2**	**Allocation**	**Members**, **Requests**

continues on next page

Responsibilities (continued)

Id	Responsibility	SRS identifiers	Owner	Collaborators
R10	Route one approval per selected Candidate to the lending Manager; record each decision; on refusal release Holds and ask for a replacement, or refuse or drop the line	**Matched.Approve.Route**, **Matched.Refuse.Replace**, **UI-2**	**Allocation**, **Requests**	**Members**, **Ledger**, **Notifications**
R11	Detect Exclusive conflicts against Held and Allocated occurrences and pick a free instance; never double-allocate	**Exclusive.Conflict**, **QA-1**	**Allocation**	**Inventory**
R12	Check a Metered line hour by hour against Pool Capacity and the Team's Quota; never oversell	**Metered.Line.Form**, **Metered.Pool.Check**, **QA-1**	**Allocation**	**Inventory**, **Settings**
R13	Keep the Waitlist: join, order (Priority, then submission time, Bumped at the head), positions, and expiry at window start	**Exclusive.Waitlist.Join**, **Metered.Pool.Check**	**Allocation**	**Requests**
R14	Plan and apply Bumps for Priority lines outside the bump cutoff; refund the Bumped line in full and put it at the head of the Waitlist	**Exclusive.Priority.Bump**, **Exclusive.Bump.Effect**	**Allocation**, **Requests**	**Ledger**, **Settings**, **Notifications**
Request lifecycle				
R15	Run the Request state machine; evaluate every line on submission and choose Allocated, Pending, or Waiting	**Lifecycle.States**, **Lifecycle.Evaluate**	**Requests**	**Allocation**
R16	Place Holds with held credits when a Request enters Pending; release them when it leaves Pending; place none after the window lapses	**Lifecycle.Hold.Place**	**Requests**	**Allocation**, **Ledger**
R17	Allocate a Request in one transaction: commit Holds, post Charges and Earnings, form Loans, drop Optional lines with a reason	**Lifecycle.Allocate**, **Matched.Loan.Form**, **QA-2**, **CO-4**	**Requests**	**Allocation**, **Ledger**, **Notifications**
R18	Start the Objection Window on first entry to Pending; on lapse release Holds, move to Waiting, and escalate to the Capacity Planner	**Lifecycle.Window.Start**, **Matched.Escalate**, **Planner.Settings**	**Requests**	**Allocation**, **Ledger**, **Jobs**
R19	Re-evaluate Waiting Requests when capacity frees or an approval arrives; allocate one only when every Required line fits at once	**Lifecycle.Waiting.Reevaluate**, **Exclusive.Waitlist.Join**	**Requests**	**Allocation**, **Ledger**
Release and Ledger				
R20	Release a line in whole or in part and end a Loan early; post the Refund or the Earning correction; offer the freed capacity	**Release.Line**, **Release.Refund**, **Release.Loan.Prorate**	**Requests**	**Allocation**, **Ledger**
R21	Keep Accounts (granted, earned, held) and append-only Transactions with the spend-order split; post Earnings; produce Statements	**Ledger.Account**, **Ledger.Transaction.Kinds**, **Ledger.Earning.Post**, **QA-3**	**Ledger**	**Settings**, **Members**
R22	Post planned and mid-Period Grants; roll the Period over, expiring granted credits and keeping earned ones	**Ledger.Grant.Period**, **PE-4**	**Ledger**	**Jobs**, **Settings**
Reporting and settings				
R23	Show utilization, idle capacity, waiting lines, credits, and person-days, scoped to the viewer's roles	**Dashboard.Utilization**, **Dashboard.Credits**, **UI-3**	**Dashboard**	**Members**

continues on next page

Responsibilities (continued)

Id	Responsibility	SRS identifiers	Owner	Collaborators
R24	Keep every Capacity Planner setting as data, applied to future Quotes and events, with a change log	**Planner.Settings**, **Admin.Member.Create**	**Settings**	**Inventory**, **Members**
Crosscutting				
R25	Authenticate Members by username and password; enforce roles on every action, from the client or the API alike	**Admin.Member.Create**, **SE-1**, **SE-2**, **CO-6**, **CO-3**	`security`, **Members**	every module
R26	Scope every read to the caller's own record, Team, owned Resources, or role	**SE-4**, **Profile.Availability**, **Ledger.Account**, **Dashboard.Credits**	every module	`security`
R27	Record actor, change, and time for every Request and line state change and every setting change	**Lifecycle.States**, **Exclusive.Bump.Effect**, **QA-3**, **QA-7**	**Requests**, **Settings**	**Ledger**
R28	Run timed work: lapse Objection Windows, expire waiting lines, roll Periods over, from one injected `Clock`	**Lifecycle.Window.Start**, **Ledger.Grant.Period**, **PE-4**, **CO-5**	**Jobs**	**Requests**, **Ledger**, **Settings**
R29	Notify Members in-app of approval requests, outcomes, Bumps, Loans, escalations, and declined Suggestions	**CI-2**, **Lifecycle.Allocate**, **Matched.Loan.Form**, **Suggest.Accept.Existing**	**Notifications**	**Requests**, **Allocation**
R30	Present the role-based screens over the REST API, with Quotes fetched from the server	**UI-1**, **UI-2**, **UI-3**, **QA-5**, **QA-4**	Web client	every controller
Stretch, designed as extension points (Section 9)				
R31	Service tiers with a premium on Metered lines, and Bumps of lower-tier units by `Bumpable` on `MeteredAllocator`	**Metered.Tier.Define**, **Metered.Tier.Bump**	**Allocation**	**Settings**, **Ledger**
R32	Record units actually used per Metered allocation and compare requested with used	**Metered.Usage.Record**, **Metered.Usage.Compare**, **Dashboard.Usage**	**Allocation**, **Dashboard**	**Inventory**
R33	Add a demand surcharge multiplier inside `Pricing.quote` that rises as the requested window fills	**Ledger.Surcharge**	**Ledger**	**Settings**; **Requests** passes the fill from **Allocation**
R34	Deliver notifications by email as well as in-app, through a second `Delivery`, `EmailDelivery`	**CI-2** (Stretch part)	**Notifications**	**Members**

3.2 Subproblems and How Each Was Broken Down

3.2.1 Three allocation policies behind one interface

The three Allocation Classes answer the same question, "can this line have its capacity, and on what terms?", in different ways. An Exclusive line checks occurrences for overlap and may Bump; a Metered line sums units hour by hour; a Matched line waits for a lending Manager's approval. The lifecycle should not care which. The breakdown separated what the lifecycle needs from every class from what only some classes do. Every class must evaluate a line, hold it, commit it, release it, and cancel it, so those five operations form `Allocator`. The rest became three optional capabilities that **Requests** asks the `AllocatorRegistry` for and skips when a class lacks them (ADR-0007):

Allocator	`Allocator` (five operations)	`Waitlistable`	`Bumpable`	`ApprovalDriven`
`ExclusiveAllocator`	all; `hold` writes Held occurrences	yes	yes	no
`MeteredAllocator`	all; `hold` writes Held units	yes	no (Stretch: yes)	no
`MatchedAllocator`	all; `hold` is a no-op, `commit` forms Loans	no	no	yes

The parts most likely to be wrong are pure functions inside the allocators, testable without a database: conflict

detection over windows and weekly recurrences, the hourly sum against Pool Capacity and Quota, and the Shortlist filter and order. *Rejected:* one wide interface with every operation, which forces Matched to fake a Waitlist and every new class to implement methods that mean nothing for it; and three unrelated services that **Requests** picks by class, which would make every new class edit the orchestrator, against **CO-7** and **QA-6**.

3.2.2 A Request whose parts resolve at different speeds

Exclusive and Metered lines resolve at once against the database; a Matched line waits days for a person. Allocating the room before the approvals arrive charges for a room that may go unused; waiting for the approvals loses the room. The breakdown has three parts. First, one orchestrator: `RequestLifecycleService` in **Requests** owns the state machine and is the only code that moves a Request between states. Second, Holds bridge the gap (ADR-0001): when a Request enters Pending, every satisfiable Exclusive and Metered line is Held, blocking capacity without a Charge, and its Quote counts as held credits, all bounded by the Objection Window. Third, one transaction per transition: submission, an approval decision, a lapse, and a Release (with the waitlist fill it triggers) are each one `@Transactional` method that calls the allocators and **Ledger** directly, so a Request is never seen half allocated. Waiting is the single recovery state for refusals, lapses, and Bumps, which keeps the state machine to seven states. *Rejected:* a saga of modules reacting to each other's events, with a compensating step for each failure, which exposes half-allocated Requests and needs compensation logic for every transition; and best-effort allocation that charges whatever fits now, which charges for GPU hours nobody can use when the people are refused.

3.2.3 Credits as pure pricing plus append-only posting

Credits raise two different questions: what a line costs, and what happened to an Account. They were split. *Pricing* is a pure class, `Pricing`, in **Ledger**: `quote` sums quantity times Unit Price times Price Band multiplier over every hour, times the priority premium for a Priority line, and `refund` applies the Refund Schedule to a lead time. It reads no database and is used both while drafting and at submission, and the Quote it returns is locked on the Request Line, so the Charge always equals the Quote shown. *Posting* is `LedgerService`: each operation updates the Team's `account` row (granted, earned, held) under that row's lock, in the caller's transaction, and every Grant, Charge, Earning, or Refund also appends a Transaction row. Affordability is a check of the Available Balance on the locked row; the spend order (granted first, then earned) is recorded on each Charge so a Refund can return credits in the same proportions (ADR-0002). Matched lines never reach pricing at all (ADR-0004). *Rejected:* computing the Charge at commit time from the settings then in force, which lets a Charge differ from the Quote the Manager accepted; and deriving every balance by summing Transactions, which still needs a lock to be safe and grows slower with history.

3.2.4 Timed work as polling in one process

Four things happen because time passes: an Objection Window lapses, a Hold expires with it, a waiting line's window start passes, and a Period ends. Each was reduced to a row whose time has come: a Pending or Waiting Request whose `window_ends_at` is past with an approval not yet escalated, a waiting line whose start is past, and a Period whose end is past. `DueWorkJob` polls for the first two every 60 seconds and `PeriodRolloverJob` handles the third at 00:05 on the boundary day and again at start-up. Each due item is handled in its own transaction by an application-service method of the owning module (`onWindowLapse`, `expireWaitingLines`, `rollover`) that reuses the steps user actions use, such as releasing Holds and moving a Request to Waiting, with the system recorded as the actor. Because each query selects only items still in the due state, a second run finds nothing to do, and a restart loses nothing. A `java.time.Clock` bean is injected wherever time is read, so tests move time instead of waiting. *Rejected:* scheduling one timer per due item, which loses timers on restart unless a job store such as Quartz is added; operating-system cron, which **CO-5** rules out; and scheduling inside the database, which would move lifecycle logic out of the code that owns it.

4 Module Specifications

Each module is specified with the same fields: purpose, responsibilities, provided interface, required interfaces, data owned, invariants, failure behavior, information hiding, and, where a rule is not obvious, a worked example. Signatures are Java; every application-service method runs in one transaction unless marked read-only. The modules are presented bottom-up, in the order of the layers of Section 2.

4.1 Members (`members`)

Purpose. Members records who the people are: Members, Teams, the assignable roles, the skill list, and each Member's availability. It is the identity that `security` authenticates against, and it answers the two questions matching asks of a profile: is this Member available for every day of a window, and which active Members outside a Team hold a set of skills and are available. It calls no other module.

Responsibilities.

- **Profile.Skills**: one Team per Member, set by an Administrator; skills chosen from the Administrator's list, so matching is exact.
- **Profile.Availability**: availability as date ranges and the test "every day of the window lies inside one range". The one-Loan-at-a-time rule is applied by **Allocation**, which owns Loans.
- **Admin.Member.Create**: Members, Teams, moves, the skill list, and roles; one Manager per Team; deactivated Members kept out of login and off every Shortlist. The setting-change log in the same requirement belongs to **Settings**.
- **SE-3**: no column for personal data beyond name, email, Team, skills, and availability. With `security`, Members also carries **SE-2** (bcrypt hashes) and the profile and roster scope of **SE-4**.

Provided interface.

```
interface MemberService {
  Member createMember(NewMember cmd);                        // ADMIN; team required; initial password hashed
  void deactivate(MemberId id);                              // ADMIN; cannot log in, no Shortlist
  void moveToTeam(MemberId id, TeamId team);                 // ADMIN
  void grantRole(MemberId id, Role role);                    // ADMIN; MANAGER: the Team's previous Manager loses it
     (exactly one per Team)
  void revokeRole(MemberId id, Role role);                   // ADMIN; refused if it would leave a Team with no
     Manager
  Team createTeam(String name); void renameTeam(TeamId, String); void retireTeam(TeamId);  // ADMIN; retire
     refused while Members remain
  void updateSkills(MemberId id, Set<SkillId> skills);       // self
  void updateAvailability(MemberId id, List<DateRange> ranges);  // self or own Manager
  MemberProfile profile(MemberId id);                        // (read-only) self, own Manager, PLANNER, ADMIN
  MemberSummary summary(MemberId id);                        // (read-only, unscoped) name, Team, active, roles; for
     other modules' rules
  List<MemberSummary> roster(TeamId team);                   // (read-only) that Team's Manager, PLANNER, ADMIN
  boolean isAvailable(MemberId id, DateRange window);        // (read-only) every day inside one range
  Set<MemberId> availableWithSkills(Set<SkillId> skills, DateRange window, TeamId excludeTeam); // (read-only)
     active Members only
  List<Skill> skills(); Skill addSkill(String name); void retireSkill(SkillId id);  // ADMIN for writes
  List<MemberId> withRole(Role role);                        // (read-only) active Members holding the role
  MemberId managerOf(TeamId team);                           // (read-only) the Team's current Manager
  List<Team> teams();                                        // (read-only)
}
```

Every active Member holds the authority `MEMBER`; `member_role` stores only `MANAGER`, `RESOURCE_OWNER`, `PLANNER`, and `ADMIN`. Members also provides the `UserDetailsService` bean that `security` installs in Spring Security; it reports a deactivated Member as disabled and is the only code that reads `password_hash`.

Operation	Preconditions (refused with)	Notes
`createMember`	username unused (`USERNAME_TAKEN`); Team not retired (`TEAM_RETIRED`)	password hashed with bcrypt; plain text never stored or logged
`deactivate`, `moveToTeam`	a Manager leaves only as the Team's last active Member (`TEAM_NEEDS_MANAGER`); an active Administrator remains (`LAST_ADMINISTRATOR`); target Team not retired (`TEAM_RETIRED`)	history, formed Loans, and submitted Requests are unchanged
`grantRole`	Member active (`MEMBER_INACTIVE`)	`MANAGER`: the previous Manager of the Member's own Team loses it in the same transaction
`revokeRole`	role is not `MANAGER` (`TEAM_NEEDS_MANAGER`); not the last `ADMIN` (`LAST_ADMINISTRATOR`)	a Manager is replaced by granting the role to the successor
`createTeam`, `renameTeam`, `retireTeam`	name unused (`TEAM_NAME_TAKEN`); retire only without active Members (`TEAM_NOT_EMPTY`)	a retired Team's Account and Statements stay readable
`updateSkills`, `addSkill`, `retireSkill`	skill unretired unless already held (`SKILL_RETIRED`); name unused, ignoring case (`SKILL_NAME_TAKEN`)	a retired skill stays where it is used but is no longer offered
`updateAvailability`	every range has from $\leq$ to (`RANGE_INVALID`)	replaces the list; overlapping and touching ranges are merged on save

Required interfaces. None. Members hashes passwords with the bcrypt `PasswordEncoder` bean of the `security` configuration.

Data owned. `team(id, name, manager_member_id, retired_at)`, `member(id, team_id, name, email, username, password_hash, active)`, `member_role(member_id, team_id, role)`, `skill(id, name, retired_at)`, `member_skill(member_id, skill_id)`, `availability_range(id, member_id, from_date, to_date)`. Username, Team name, and skill name are unique. A composite foreign key from `team(manager_member_id, id)` to `member(id, team_id)` keeps every Team's Manager inside that Team.

Invariants.

- Each Team has exactly one Manager at any time once the role is first granted in it: the Manager is one column, so there cannot be two, and the role moves only to a successor. `member_role` has at most one `MANAGER` row per Team (partial unique index), mirrored in `team.manager_member_id`. A Team that has never had a Manager can own no Request, since only its Manager can create one.
- A deactivated Member cannot log in and appears on no Shortlist: the `UserDetailsService` reports them disabled, `availableWithSkills` returns active Members only, and `isAvailable` returns false for them.
- A Member's ranges never overlap or touch, so "available for a window" is the single test "one range contains it"; a Loan never changes them. At least one active Administrator always exists.

Failure behavior. A refusal is a `RuleViolation` with a code above; the transaction rolls back whole, and the code reaches the client in the `problem+json` body (Section 8). Unknown ids raise `NotFoundException`, and so does an object outside the caller's scope (404); acting on a visible object without the right to do so raises `ForbiddenException`. A deactivated Member's login fails like a wrong password, and every attempt is logged.

Information hiding. No DTO carries the password hash, so it never leaves the module. How roles are stored (`member_role` plus `team.manager_member_id`) and how ranges are merged are private; other modules ask `isAvailable` and `availableWithSkills` and never interpret ranges.

Worked example. A Member records 2–6 and 9–13 November. For a Matched line from 5 to 10 November `isAvailable` is false, because 7 and 8 November lie in neither range. Recording 2–6 and 7–13 November instead saves one range, 2–13 November, and the same window is available.

Figures. Figure 5.

4.2 Inventory (`inventory`)

Purpose. Inventory holds Resource Types as data and the Resources that exist, with their type, attribute values, owner, and, for Metered Resources, units. It derives Pool Capacity and refuses a reduction below future usage, which it learns through the port `ReductionGuard` so that it never depends on the layer above. Who holds a Resource is not stored here; Holds and allocations belong to **Allocation**.

Responsibilities.

- **Inventory.Type.Define**: Resource Types as rows (unique name, one Allocation Class, typed attributes) defined, changed, and retired with no code or deployment, which is also the first half of **QA-6**; the built-in Person type cannot be retired.
- **Inventory.Resource.Add**: adding, editing, and retiring Exclusive and Metered Resources; Pool Capacity as the sum of unretired units; a retirement or reduction below future usage refused, naming the earliest hour.
- **Inventory.View.Holder**: listing and filtering Resources by type and attribute value. The Holds timeline and the "free for a window" filter come from `AllocationQueries.holders` through Allocation's controller; the Web client calls both APIs.

Provided interface.

```
interface ResourceTypeService {
  ResourceType define(ResourceTypeDefinition def);          // ADMIN; unique name, one Allocation Class, attributes
  ResourceType change(ResourceTypeId id, ResourceTypeDefinition def);  // ADMIN; class immutable once a Resource
     exists
  void retire(ResourceTypeId id);                           // ADMIN; refused for built-in Person; retired types
     accept no new lines
}
interface ResourceService {
  Resource add(NewResource cmd);                            // RESOURCE_OWNER; attributes per type; units for
     Metered
  Resource edit(ResourceId id, ResourceEdit cmd);           // owner of that Resource
  void setUnits(ResourceId id, int units);                  // owner; calls ReductionGuard; refused naming the
     earliest failing hour
  void retire(ResourceId id);                               // owner; same guard with units 0
}
interface InventoryQueries {                                // all read-only
  ResourceType type(ResourceTypeId id); List<ResourceType> types(boolean includeRetired);
  List<Resource> instancesOf(ResourceTypeId id);           // unretired
  int poolCapacity(ResourceTypeId id);                      // sum of units of unretired Resources
  List<Resource> search(ResourceFilter f);                  // by type, attribute value; free-for-window answered
     by Allocation
}
interface ReductionGuard {                                   // port declared here, implemented in allocation
  Optional<Instant> earliestHourExceeding(ResourceTypeId type, Optional<ResourceId> instance, int newUnits,
     Instant from);
}
```

Operation	Preconditions (refused with)	Notes
`define`, `change`	name unused (`TYPE_NAME_TAKEN`); attributes well formed (`ATTRIBUTE_INVALID`); class unchanged once a Resource exists (`CLASS_FIXED`) and never for Person (`PERSON_TYPE_FIXED`); no attribute or allowed value in use removed (`ATTRIBUTE_IN_USE`)	no delete operation exists, so a type a line refers to is never deleted
`retire` (type)	not Person (`PERSON_TYPE_FIXED`)	Resources and submitted lines untouched; **Requests** refuses new lines (`TYPE_RETIRED`)
`add`	type Exclusive or Metered (`MATCHED_HAS_NO_RESOURCES`) and not retired (`TYPE_RETIRED`); values match the attribute kinds (`ATTRIBUTE_INVALID`); Metered units ≥ 1 (`UNITS_INVALID`)	the caller becomes the owner; an Exclusive Resource has units 1
`edit`	a new owner is active and holds `RESOURCE_OWNER` (`NOT_A_RESOURCE_OWNER`)	by the owner, or by an Administrator reassigning the owner
`setUnits`	Metered (`NOT_METERED`); units ≥ 1 (`UNITS_INVALID`); the guard finds no failing hour (`UNITS_BELOW_USAGE`)	an increase skips the guard
`retire` (Resource)	the guard, with units 0, finds no failing hour (`UNITS_BELOW_USAGE`; `RESOURCE_IN_USE` for Exclusive)	a retired Resource is never offered again

`setUnits` and `retire` first lock the type's `resource_type` row (`SELECT ... FOR UPDATE`), the same lock

`AllocationLock.lockTypes` takes, which Inventory may take directly because it owns the row. They then call `earliestHourExceeding(type, Optional.of(resource), newUnits, clock.instant())`, and only then write. **Allocation** answers with the hourly sum of the Metered check against the Pool Capacity that would result, or, for an Exclusive instance, the start of its first future Held or Allocated occurrence. Because the lock is held from check to write, a concurrent allocation of the type has either committed and is counted, or waits and then sees the new capacity.

Required interfaces. **Members**: `MemberService.summary`, the unscoped read, to check a new owner. **Allocation**, through the port only: `ReductionGuard.earliestHourExceeding`; Inventory compiles against the interface it declares, and Spring injects Allocation's implementation.

Data owned. `resource_type(id, name, allocation_class, retired_at)`, with Person seeded by the first migration; `resource_type_attribute(id, type_id, name, kind, allowed_values)`, kind being text, number, date, or list; `resource(id, type_id, name, owner_member_id, units, retired_at)`, with `units >= 1`; `resource_attribute_value(resource_id, attribute_id, value)`, the value stored as text and validated by kind on write.

Invariants.

- A Resource Type's Allocation Class never changes once a Resource of it exists. The built-in Person type (class Matched) cannot be retired, and its class never changes.
- No Resource Type or Resource is ever deleted, only retired. A Matched type has no Resource rows (its Resources are Members); an Exclusive Resource has units 1.
- Pool Capacity equals the sum of the units of the unretired Resources of the type. It is one `SUM` on every call and never stored, so it cannot drift.
- A reduction or retirement is refused, naming the earliest hour at which allocated plus held units would exceed the new Pool Capacity, or at which the Exclusive instance is in use. A Hold counts exactly as an allocation.
- `instancesOf` never returns a retired Resource, so no allocator can pick one.
- Resource Owners keep the Inventory current (**AS-2**).

Failure behavior. A refusal is a `RuleViolation` with a code above, and nothing is written; the two guard refusals carry an `earliestHour` property so the screen names the blocking hour. A lock conflict surfaces as `ConcurrencyRetryException` after one retry.

Information hiding. Attribute storage is private: callers see typed values on `Resource` and filter through `ResourceFilter`, so moving the text rows to a `jsonb` column would change no caller. Inventory does not know how usage is computed, and **Allocation** knows Pool Capacity only through `poolCapacity`.

Worked example. A Metered type *GPU* has one Resource, *node-a*, with 8 units; Pool Capacity is 8. On 15 October 2026 from 14:00 to 15:00, 3 units are allocated and 2 are on Hold for a Pending Request, 5 in all; earlier that day at most 4 are in use. On 12 October the owner calls `setUnits(node-a, 4)`. Inventory locks the *GPU* row and asks the guard, which finds that usage first exceeds 4 at 15 October 14:00; the call is refused with `UNITS_BELOW_USAGE` naming that hour. A reduction to 5 succeeds, and so does one to 4 once the Hold has been released.

Figures. Figure 5; the `ReductionGuard` port in Figure 2.

4.3 Settings (settings)

Purpose. Settings holds every Capacity Planner setting as data and logs every change. Other modules read one typed snapshot, `PlannerSettings`, and never see how settings are stored. Settings stores and validates values; the arithmetic that uses them is in **Ledger** (`Pricing`), **Allocation**, and **Requests**.

Responsibilities.

- **Planner.Settings**: the Period, Grants per Team and Period, each Exclusive and Metered type's Unit Price, Price Bands, and Refund Schedule, the Day Rate, Quotas, the priority premium, bump cutoffs, and the Objection Window, all set without deployment; a change affects only Quotes not yet locked and events after it. The escalation list of the same requirement belongs to **Requests**.
- **Admin.Member.Create**, last sentence: every setting change recorded with actor, old and new value, and timestamp.
- The rules the SRS states on settings: Price Bands do not overlap (**Ledger.Price.Formula**); no refund step reaches 100% (**Release.Refund**).

Provided interface.

```
interface SettingsService {
  PlannerSettings current();                                  // (read-only) typed snapshot, see below
  void update(SettingChange change);                          // PLANNER; writes setting_change(actor, key, old, new,
     at)
  List<Period> periods();                                     // (read-only) past and planned Periods
}
record PlannerSettings(Period currentPeriod, int objectionWindowWorkingDays, Credits dayRate,
                       BigDecimal priorityPremium, Map<ResourceTypeId, TypePricing> pricing,
                       Map<TeamTypeKey, Integer> quotas, Map<TeamId, Credits> grants) {}
record TypePricing(Credits unitPrice, List<PriceBand> bands, RefundSchedule refundSchedule, Duration bumpCutoff) {}
```

`SettingChange` is a sealed interface with one record per kind of setting, so `update` is one entry point with typed arguments. The value types are declared here because `TypePricing` holds them and Settings sits below **Ledger**.

```
sealed interface SettingChange {      // every record below implements SettingChange
  record PeriodLength(int months) {}              record DayRate(Credits rate) {}
  record PriorityPremium(BigDecimal multiple) {}  record ObjectionWindow(int workingDays) {}
  record GrantPlan(TeamId team, PeriodId period, Credits amount) {}
  record UnitPrice(ResourceTypeId type, Credits price) {}
  record PriceBands(ResourceTypeId type, List<PriceBand> bands) {}
  record Refunds(ResourceTypeId type, RefundSchedule schedule) {}
  record BumpCutoff(ResourceTypeId type, Duration cutoff) {}
  record Quota(TeamId team, ResourceTypeId type, Optional<Integer> maxUnits) {}   // empty: no Quota
}
record Period(PeriodId id, LocalDate startsOn, LocalDate endsOn, Optional<PeriodId> previous) {}
record PriceBand(Set<DayOfWeek> weekdays, LocalTime start, LocalTime end, BigDecimal multiplier) {}  // end
     exclusive
record RefundSchedule(List<RefundStep> steps) {}      record RefundStep(Duration minLead, BigDecimal fraction) {}
```

Change	Validation (refused with)	Takes effect for
`PeriodLength`	1 to 12 calendar months (`VALUE_OUT_OF_RANGE`)	Periods not yet started
`GrantPlan`	Period not started (`PERIOD_STARTED`); amount ≥ 0	that Period's start; a mid-Period Grant is `LedgerService.postGrant`
`UnitPrice`, `PriceBands`	type Exclusive or Metered (`CLASS_MISMATCH`); each band has weekdays, start before end, multiplier > 0 (`BAND_INVALID`); no hour in two bands (`BANDS_OVERLAP`)	Quotes computed after the commit; a locked Quote never changes
`Refunds`	type Exclusive or Metered; every fraction in $[0, 1)$ (`REFUND_STEP_TOO_HIGH`)	Releases after the change
`BumpCutoff`, `Quota`	Exclusive, respectively Metered, type (`CLASS_MISMATCH`); cutoff ≥ 0, Quota ≥ 1 or empty (`VALUE_OUT_OF_RANGE`)	Bumps planned and lines evaluated after the change; existing allocations stand
`DayRate`, `PriorityPremium`, `ObjectionWindow`	rate ≥ 0, premium ≥ 1, window ≥ 1 working day (`VALUE_OUT_OF_RANGE`)	Loans formed, Quotes computed, and Requests entering Pending after the change

Periods are rows. When the length is set (first by the installation migration, with the default of one calendar month), Settings writes the `period` rows ten years ahead, each starting the day after the previous one ends; `current()`

returns the row containing today by the injected `Clock`. A Team's Grant for a Period is that Period's `grant_plan` row or else the Team's most recent earlier one, so a Grant set once recurs until changed.

The Objection Window is counted in working days. The setting is `objection_window_working_days` (the SRS example default is 2), and `window_ends_at` is the instant the Request first enters Pending, usually its submission, plus that many Monday-to-Friday days. With the default, the window of a Request submitted on a Friday at 10:00 ends on the next Tuesday at 10:00.

Required interfaces. **Inventory**: `InventoryQueries.type`, to check that a type exists and has the Allocation Class the setting requires. **Members**: no call; `team_id` and `actor_id` are read-only foreign keys to `team` and `member`.

Data owned. `period(id, starts_on, ends_on)` with an exclusion constraint against overlap; `grant_plan(team_id, period_id, amount)`; `type_pricing(type_id, unit_price, bump_cutoff_hours)`; `price_band(id, type_id, weekdays, start_time, end_time, multiplier)`; `refund_step(id, type_id, min_lead_hours, fraction)` with `check (fraction >= 0 and fraction < 1)`; `quota(team_id, type_id, max_units)`; `planner_setting(key, value)` for `day_rate`, `priority_premium`, `objection_window_working_days`, and `period_months`; `setting_change(id, key, old_value, new_value, actor_id, at)`, insert only.

Invariants.

- Every setting change is logged: `update` writes exactly one `setting_change` row (actor, key, old value, new value, timestamp) in the same transaction as the change, so neither exists without the other.
- A changed price affects only Quotes not yet locked: locked Quotes live on the Request Line, owned by **Requests**, and Settings never reads or writes them.
- The Price Bands of a type never overlap; an hour no band covers has multiplier 1.0. No refund step reaches 100%.
- Periods are contiguous and never overlap, so exactly one contains any date; a started Period never changes its dates.

Failure behavior. A refused change is a `RuleViolation` with a code above; the change and its log row roll back together. `update` locks the changed row before reading the old value, so concurrent edits of one setting are serialized and each logged old value is the one replaced.

Information hiding. The settings table layout is hidden behind `PlannerSettings`: no caller knows that some settings are key–value rows and others typed tables, or that Periods are written ahead. Only the **Dashboard** views read a settings table (`period`).

Worked example. The SRS example defaults for a room type are a Unit Price of 10 credits per room-hour, weekdays 09:00–17:00 at $\times 1.5$, and other hours at $\times 0.5$. Unbanded hours count at $\times 1.0$, so "other hours" must be stored as bands: Monday to Friday 00:00–09:00 and 17:00–24:00, and Saturday and Sunday 00:00–24:00, each at 0.5, beside the 09:00–17:00 band at 1.5. A one-room line on Thursday 15 October 2026 from 16:00 to 19:00 then quotes $15 + 5 + 5 = 25$ credits, or 50 as a Priority line at the default premium of 2. If the Unit Price rises to 12 on 10 October, a Request submitted on 9 October keeps its locked 25, while a Draft viewed after the change shows $18 + 6 + 6 = 30$.

Figures. Figure 5.

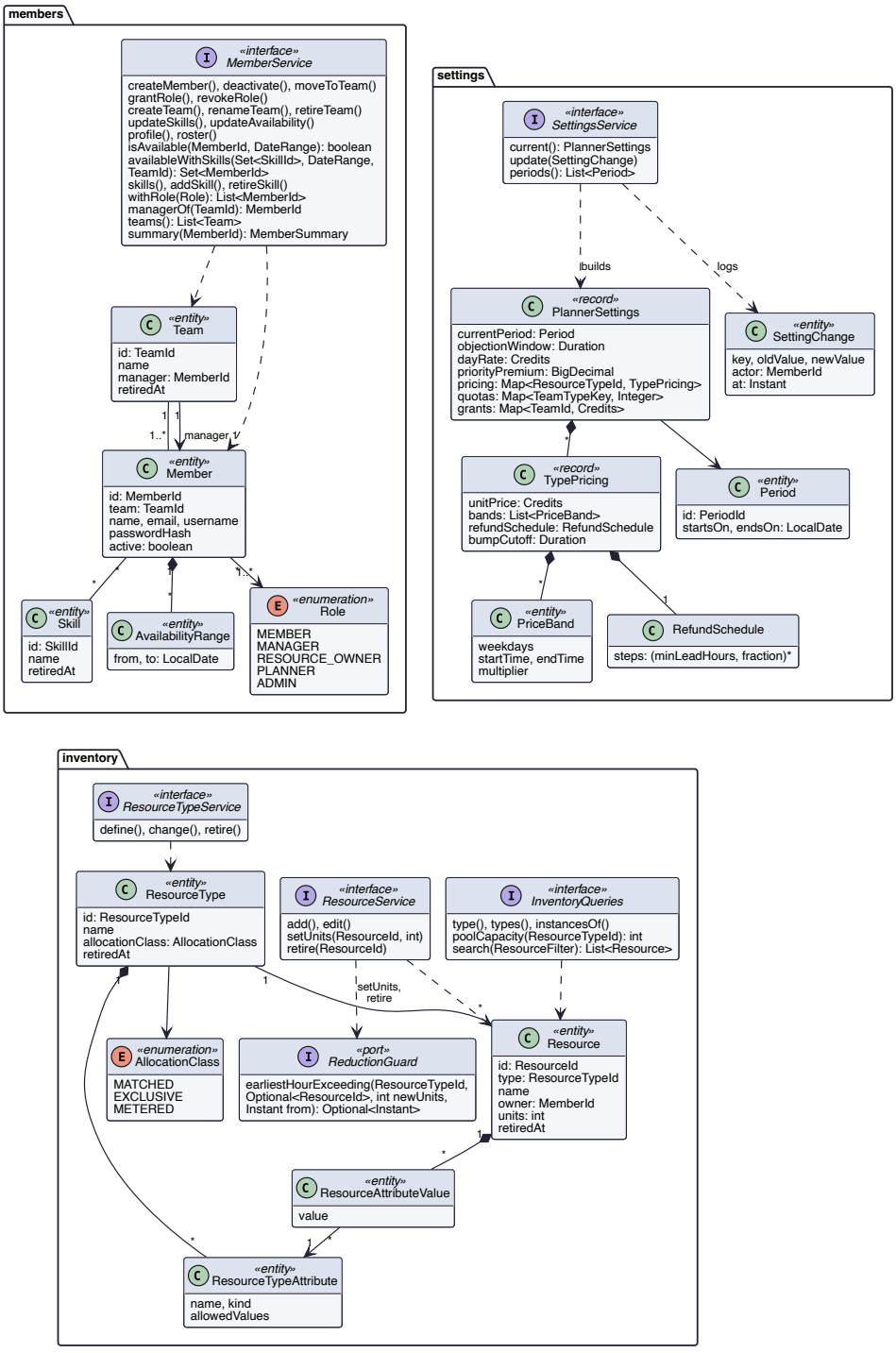


Figure 5: Entities and application services of the three foundation modules, **Members**, **Inventory**, and **Settings**. Each package is one module; a module's entities are package-private and reached only through the services shown. The `ReductionGuard` socket on **Inventory** is the one port of the design (realized by **Allocation**, Figure 6).

4.4 Notifications (notifications)

Purpose. Notifications tells a Member, inside the application, that something concerning them has happened. Callers decide when a notice is due and what it says, and call `Notifier.notify` inside their own transaction; Notifications decides how the notice is stored and delivered.

Responsibilities.

- **CI-2**: every notification in Release 1 is in-app; email (Stretch) is a second `Delivery`.
- The notification sentences of **Lifecycle.Allocate** (allocation, state changes, Bumps, Waitlist fills, escalations, approval requests), **Matched.Loan.Form** (the Candidate when selected, the Person when the Loan forms), **Exclusive.Bump.Effect** (the Bumped line's Manager, with the reason), and **Suggest.Accept.Existing** (the Member when declined, with the reason); and the notification list every screen shows.

Provided interface.

```
interface Notifier { void notify(MemberId to, NotificationKind kind, String text, String link); }
interface NotificationQueries { List<Notification> unread(MemberId me); void markRead(NotificationId id); }
interface Delivery { void deliver(Notification n); }         // InAppDelivery (row only); EmailDelivery is Stretch
```

`notify` inserts the row, then hands it to every `Delivery` bean; Release 1 has only `InAppDelivery`, which does nothing more because the row is the delivery. The text is complete when written, and `link` is a Web client route such as `/requests/42`. `unread` answers only for the signed-in Member; `markRead` sets `read_at` once.

`NotificationKind`	Raised during	Recipient
`APPROVAL_REQUESTED`	routing approvals; any approval on an escalated Request	the lending Manager; the Capacity Planners once escalated
`CANDIDATE_SELECTED`, `LOAN_FORMED`	`selectCandidates`; `allocate`	the Person
`REQUEST_UPDATE`	every Request state change, a dropped Optional line, a refusal asking for a replacement	the requesting Manager
`BUMPED`, `WAITLIST_FILLED`	after `applyBump`, by `RequestLifecycleService`; `onCapacityFreed`	the Manager of the Bumped line or filled Request
`ESCALATED`	`onWindowLapse`	the Capacity Planners and the requesting Manager
`SUGGESTION`	creating, accepting, or declining a Suggestion	the Manager (new); the Member (decided)

Required interfaces. **Members**, for recipient identity only: `member_id` references `member(id)` and no operation is called. A caller that addresses a role, such as an escalation to the Capacity Planners, resolves its holders before calling `notify`.

Data owned. `notification(id, member_id, kind, text, link, created_at, read_at)`, with a partial index on `member_id` where `read_at` is null.

Invariants.

- Notifications are rows written in the caller's transaction: a notice exists exactly when the action that raised it committed, and nothing is sent after commit in Release 1.
- A notice is visible only to its recipient; `read_at` is set at most once, and no notice is deleted.

Failure behavior. No `RuleViolation` codes. If the insert fails, the caller's action rolls back with it; a database that cannot insert the row could not have committed the action either. `markRead` on another Member's notice raises `NotFoundException`, which does not reveal that the notice exists.

Information hiding. The delivery channel is hidden behind `Delivery`: callers never know whether a notice becomes a row, an email, or both. A Stretch `EmailDelivery` would send from an after-commit callback, so a rolled-back action sends no email, and no caller would change (Section 9).

Figures. `Notifier` calls inside the allocating transaction appear in Figure 13 and Figure 16.

4.5 Allocation (`allocation`)

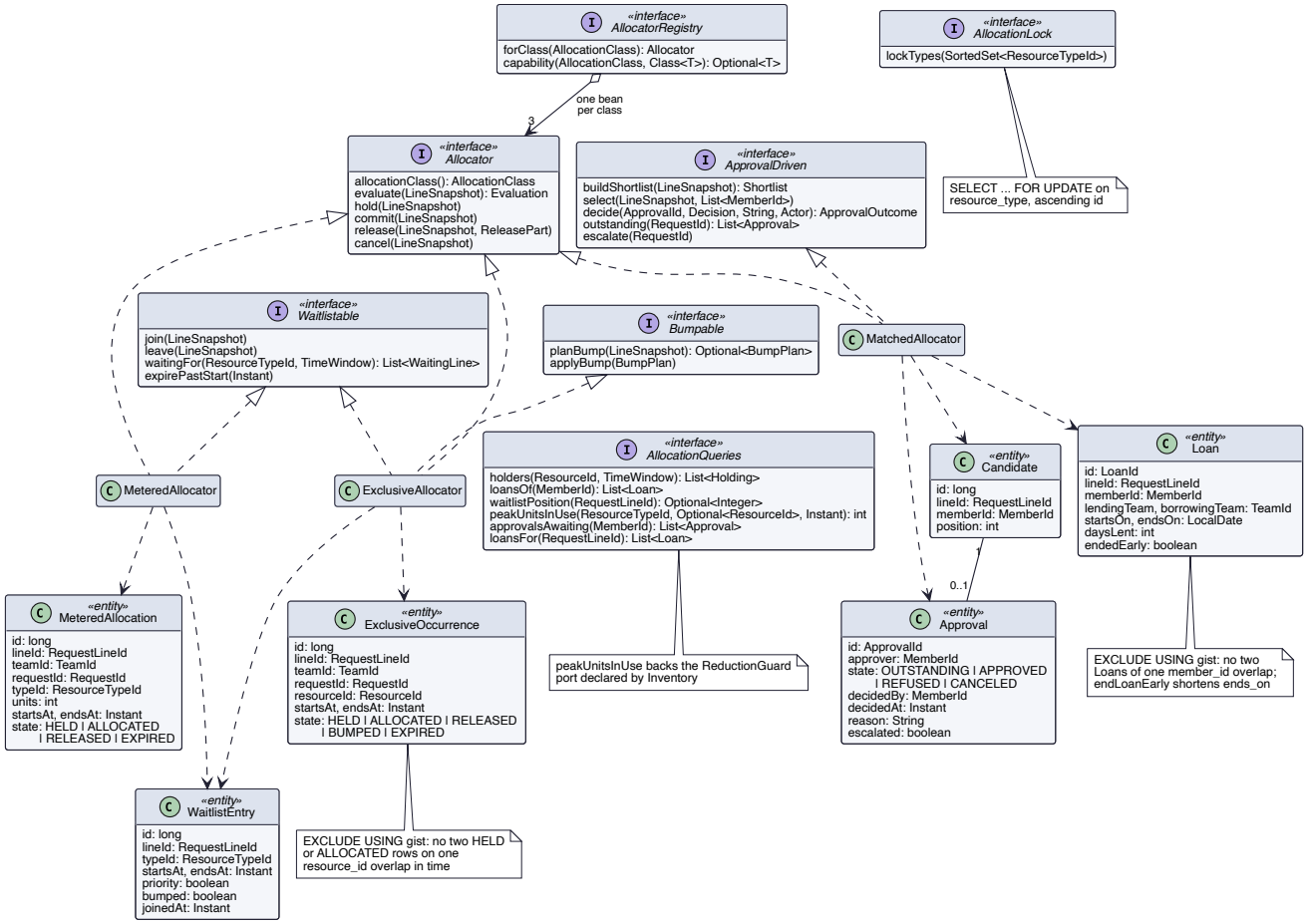


Figure 6: Classes of **Allocation**: the `Allocator` interface with its five operations, the three capability interfaces, the registry, the three allocators and the capabilities each realizes, the shared lock and query services, and the six entities the module owns. Dashed arrows are dependencies; hollow-triangle dashed arrows are realizations.

Purpose. Allocation decides, one Request Line at a time, whether the capacity it asks for can be had now, and records the answer as rows; a Hold is simply the state `HELD` of such a row. Everything that differs between Allocation Classes lives here, behind one interface that **Requests** drives without branching on the class.

Responsibilities.

- One interface for every Allocation Class (**CO-7**, **QA-6**); every allocation decision serialized per Resource Type in the database (**QA-1**).
- Exclusive lines: conflicts, instance choice, waitlist, and Bump mechanics (**Exclusive.Conflict**, **Exclusive.Waitlist.Join**, **Exclusive.Priority.Bump**, **Exclusive.Bump.Effect**). Metered lines: the hourly fit (**Metered.Line.Form**, **Metered.Pool.Check**). Holds as a row state that blocks like an allocation (**Lifecycle.Hold.Place**).
- Matched lines: Shortlists, approvals, and Loans (**Matched.Shortlist.Build**, **PE-3**, **Matched.Approve.Route**, **Matched.Loan.Form**).
- The holder view of a Resource and Inventory's reduction guard (**Inventory.View.Holder**).

Provided interface.

```
interface Allocator {
  AllocationClass allocationClass();
  Evaluation evaluate(LineSnapshot line);                    // SATISFIABLE | NEEDS_APPROVAL | UNSATISFIABLE(reason)
  void hold(LineSnapshot line);                              // Exclusive, Metered: rows in state HELD; Matched:
     no-op
  void commit(LineSnapshot line);                            // HELD -> ALLOCATED; Matched: form Loans
```

```
  void release(LineSnapshot line, ReleasePart part);         // whole or part; Matched: end Loan early
  void cancel(LineSnapshot line);                            // drop Holds, approvals, waitlist entries
}
interface Waitlistable { void join(LineSnapshot line); void leave(LineSnapshot line);
  List<WaitingLine> waitingFor(ResourceTypeId type, TimeWindow freed); void expirePastStart(Instant now); }
interface Bumpable { Optional<BumpPlan> planBump(LineSnapshot priorityLine); void applyBump(BumpPlan plan); }  //
    Exclusive only
interface ApprovalDriven {
  Shortlist buildShortlist(LineSnapshot line);               // eligible = skills, available, no overlapping Loan,
    other Team, active; order by person-days lent this Period, then name
  void select(LineSnapshot line, List<MemberId> candidates); // exactly the line's quantity; creates one approval
    per Candidate
  ApprovalOutcome decide(ApprovalId id, Decision d, String reason, Actor actor);  // lending Manager or PLANNER
    (escalated)
  List<Approval> outstanding(RequestId request); void escalate(RequestId request);
}
interface AllocatorRegistry { Allocator forClass(AllocationClass c); <T> Optional<T> capability(AllocationClass c,
    Class<T> cap); }
interface AllocationLock { void lockTypes(SortedSet<ResourceTypeId> types); }   // SELECT ... FOR UPDATE on
    resource_type, ascending id
interface AllocationQueries {                                // read-only
  List<Holding> holders(ResourceId resource, TimeWindow window);   // free / HELD / ALLOCATED segments with Team
    and Request
  List<Loan> loansOf(MemberId member); Optional<Integer> waitlistPosition(RequestLineId line);
  List<Loan> loansFor(RequestLineId line);                   // the Loans a Matched line's commit formed (for
    postEarning)
  int peakUnitsInUse(ResourceTypeId type, Optional<ResourceId> instance, Instant from);  // backs ReductionGuard
  List<Approval> approvalsAwaiting(MemberId approver);       // outstanding approvals addressed to this Manager;
    every escalated one for a PLANNER
}
```

`LineSnapshot` is an immutable copy of a line built by **Requests** (ids, type and class, quantity or units, window and recurrence, named Resource or Person, skills, Required and Priority marks, submission time, a `bumped` flag); `ReleasePart` is the whole line, some occurrences, an earlier end, fewer units, or one Loan with a new last day.

Why five operations. **Requests** runs one lifecycle for every line: decide (`evaluate`), block capacity while approvals are outstanding (`hold`), make it final (`commit`), give part back (`release`), abandon it before allocation (`cancel`). Every class needs exactly these, so they form `Allocator`; anything only some classes do sits on a capability, so no class carries empty methods. `ExclusiveAllocator` implements `Waitlistable` and `Bumpable`, `MeteredAllocator` `Waitlistable`, and `MatchedAllocator` `ApprovalDriven`. The one exception is the no-op `hold` of a Matched line, cheaper than a capability check at the one call site that holds every satisfiable line.

Registry and extension. `AllocatorRegistry` receives `List<Allocator>`, which Spring fills with every `Allocator` bean, and indexes it by `allocationClass()`; start-up fails if a class has two allocators or none. `capability(c, cap)` returns the allocator cast to `cap` if it implements it, so **Requests** never names a class: the Priority mark is offered for a `Bumpable` class, waiting for a `Waitlistable` one, and a Quote whenever the type has pricing settings. A fourth Allocation Class is one new `@Component` implementing `Allocator` and its capabilities, plus one `AllocationClass` constant; **Requests**, **Ledger**, and the existing allocators do not change (**CO-7**, **QA-6**).

Table 4: Allocation operations. Errors are `RuleViolation` codes unless named otherwise.

Operation	Preconditions	Errors
`evaluate`	The line's type is locked	none; returns `NO_FREE_INSTANCE`, `POOL_FULL`, `QUOTA_EXCEEDED`, `SHORTLIST_TOO_SHORT`, or `NOT_A_CANDIDATE` as the reason
`hold`, `commit`	Type locked; the line fit in this transaction or is Held; Matched `commit`: quantity Approved and the line re-evaluated in this transaction	`NO_FREE_INSTANCE`, `POOL_FULL`, `QUOTA_EXCEEDED`; Matched `commit`: `ALLOCATION_CONFLICT` (guard)
`release`, `cancel`	`release`: the part lies inside the live allocation; `cancel` is idempotent	`NOT_ALLOCATED`
`join`, `leave`, `waitingFor`, `expirePastStart`	`join`: a Required line that does not fit; one entry per line	none
`planBump`, `applyBump`	Type locked; Priority line with no free instance; plan made in this transaction	`INSIDE_BUMP_CUTOFF` (stale plan)
`buildShortlist`, `select`	`select`: new plus Outstanding or Approved Candidates equal the quantity; all on the Shortlist	`LINE_NOT_ATOMIC`, `NOT_A_CANDIDATE`, `OWN_TEAM`

Operation	Preconditions	Errors
`decide`, `escalate`	Undecided; actor the approver before the window ends, a Capacity Planner after; Candidate still eligible	`ALREADY_DECIDED`, `WINDOW_LAPSED`, `OWN_TEAM`, `NOT_A_CANDIDATE`
`lockTypes`	Inside a transaction, before any account lock, except in a waitlist fill; re-locking is a no-op	`ConcurrencyRetryException`

Required interfaces. **Inventory**: `InventoryQueries.type`, `instancesOf`, `poolCapacity`. **Members**: `MemberService.availableWithSkills`, `isAvailable`, `summary` (name, Team, active, roles; the unscoped read for this module's rules), `managerOf` (the approver for a Candidate's Team), `withRole(PLANNER)` (escalation notices to every Capacity Planner). **Settings**: `SettingsService.current` (Period, bump cutoffs, Quotas, Day Rate). **Notifications**: `Notifier.notify` (approval requests, Candidate selected, escalation).

Data owned. `exclusive_occurrence(id, line_id, team_id, request_id, resource_id, starts_at, ends_at, state)`, one row per occurrence and instance, and `metered_allocation(id, line_id, team_id, request_id, type_id, units, starts_at, ends_at, state)`, one per line, in states `HELD`, `ALLOCATED`, `RELEASED`, `BUMPED` (Exclusive), and `EXPIRED` (Hold dropped). `waitlist_entry(id, line_id, type_id, starts_at, ends_at, priority, bumped, joined_at)`; `candidate(id, line_id, member_id, position)`; `approval(id, candidate_id, approver_member_id, state, decided_by, decided_at, reason, escalated)`, in state `OUTSTANDING`, `APPROVED`, `REFUSED`, or `CANCELED`, where `escalated` is a flag, not a state, that `escalate` sets on an Outstanding approval and that stays set after the decision; `loan(id, line_id, member_id, lending_team_id, borrowing_team_id, starts_on, ends_on, days_lent, ended_early)`. Added: `team_id` and `request_id` on both `exclusive_occurrence` and `metered_allocation` (so the Quota sum and holder view read only this module's tables) and `ended_early` on `loan`. The two guarantees (`btree_gist`):

```
ALTER TABLE exclusive_occurrence ADD EXCLUDE USING gist
  (resource_id WITH =, tstzrange(starts_at, ends_at, '[)') WITH &&) WHERE (state IN ('HELD','ALLOCATED'));
ALTER TABLE loan ADD EXCLUDE USING gist (member_id WITH =, daterange(starts_on, ends_on, '[]') WITH &&);
```

Invariants.

- No two live (`HELD` or `ALLOCATED`) occurrences on one Exclusive instance overlap; in no hour do a Metered type's live units exceed its Pool Capacity, or a Team's its Quota (**QA-1**).
- Every write that consumes capacity, and every capacity reduction, comes after the lock on that Resource Type in the same transaction.
- A line's live rows are all or nothing: every occurrence on the same instances, or the full units for the full window.
- A Priority line is never Bumped, and no line is Bumped inside the bump cutoff.
- A Person is in at most one Loan on any day. A Candidate is active and outside the requesting Team, and until escalation only the Manager of the Candidate's Team decides.

Failure behavior. A misfit is a result of `evaluate`, never an exception; the codes of Table 4 report bad input or a precondition the caller should have established. Allocator beans use `Propagation.MANDATORY`: they never commit alone, and any exception rolls back the whole transition. A 23P01 from either exclusion constraint means a path skipped the lock or a check; the transition rolls back, so nothing is written, and the error is logged as a bug and answered 409 `ALLOCATION_CONFLICT`. The guard in `MatchedAllocator.commit` throws the same code. A lock wait past `lock_timeout` (5 s) or a deadlock is retried once, then reported as `ConcurrencyRetryException` (503).

Information hiding. Row layout, Holds as a state, the constraints, the hourly query, the Bump search, and the Shortlist ranking are hidden behind evaluations and DTOs; hour buckets could replace the computed sum without touching **Requests**.

Shared parts

`AllocationLock` runs one `SELECT ... FOR UPDATE` on `resource_type` per id, ascending (`@Lock(PESSIMISTIC_WRITE)`), at the default READ COMMITTED isolation; a transition calls it once, first, for the types of its own lines, and keeps the locks to commit, so allocation on a type is serial; only a waitlist fill may lock further types later, those of a waiting Request, possibly after an account lock, a case the retry covers (Section 5). `AllocationQueries` reads live rows: `holders` cuts a window into free, held, and allocated segments; `peakUnitsInUse` is the largest hourly sum of live units, Holds counted, from an instant on. The `ReductionGuard` implementation locks the type, computes the capacity after the change, and, if `peakUnitsInUse` exceeds it, names the earliest failing hour with the hourly query; the lock lasts until Inventory commits, so nothing slips in between.

Pure parts, unit-tested without Spring or a database: `Windows` (half-open overlap; weekly recurrence expanded in the firm's time zone, so 14:00 stays 14:00 across a daylight-saving change; Monday-to-Friday day counts), `HourlyLoad` (the earliest hour in which per-hour sums plus requested units exceed a limit), and `ShortlistRanking` (eligibility filter and order).

ExclusiveAllocator

Free instances. The window and weekly recurrence expand into occurrences; an instance is free when no live row on it overlaps any of them. Without a named Resource the allocator takes the free instances with the fewest `ALLOCATED` hours in the Period, ties by id, and every occurrence lands on the same instance. `hold` inserts one `HELD` row per occurrence and instance, `commit` turns them `ALLOCATED`, and `release` marks given-back occurrences (or a shortened tail) `RELEASED`, so history stays. Instances are chosen under the type lock, so the exclusion constraint never fires in normal operation; it stops a double allocation by a forgotten path.

Bump plan. For a Priority line with no free instance, `planBump` collects per instance the lines whose live rows overlap its occurrences. An instance can be cleared when all are standard and none has an occurrence starting before now plus the bump cutoff; the plan takes the instances needing the fewest Bumps, ties toward the latest-submitted lines. A line is Bumped whole, being atomic. With no plan the Priority line waits ahead of standard lines. `applyBump` marks the Bumped lines' live rows `BUMPED` with a bulk JPQL update (or an update plus `EntityManager.flush()`) and only then inserts the Priority occurrences as `HELD`: Hibernate runs inserts before updates, so without the flush the insert would meet the NOT DEFERRABLE constraint while the old rows are still live. **Requests** then joins each Bumped line to its waitlist with `bumped` set (scenario S5).

Waitlist. One `waitlist_entry` per waiting Required line. `waitingFor` returns the type's entries overlapping the freed window ordered by `bumped`, then Priority, then `joined_at`, then id; `joined_at` holds the Request's submission time, not the moment of joining. `expirePastStart` deletes entries whose start has passed; the Request stays Waiting.

Worked example. A lab-room type has two rooms and a 24-hour cutoff. On Monday at 10:00 a Priority line asks for Thursday 14:00 to 17:00. Room 1 is blocked by two standard lines, L1 (13:00 to 15:00, submitted September 28) and L2 (15:00 to 18:00, October 1); Room 2 by one, L3 (09:00 to 16:00, September 30). Room 2 needs fewer Bumps, so the plan Bumps L3. Had L3 had another occurrence on Tuesday at 09:00, inside the cutoff, the plan would Bump L1 and L2 on Room 1; had L1 also been a Priority line, there would be no plan and the line would wait.

MeteredAllocator

A line asks for units held concurrently over its whole window, one `metered_allocation` row. Under the type lock, `evaluate` runs the query below and `HourlyLoad` finds the earliest hour where `used` plus the line's units exceed `poolCapacity(type)` (`POOL_FULL`) or, when the Team has a Quota on the type, where `team_used` plus the units exceed it (`QUOTA_EXCEEDED`). The line fits with full units in every hour or not at all. Holds, `release` (fewer units or an earlier end), and the waitlist follow the Exclusive rules; there is no Bump.

```
SELECT h.hour, coalesce(sum(m.units), 0) AS used,
       coalesce(sum(m.units) FILTER (WHERE m.team_id = :team_id), 0) AS team_used
FROM generate_series(:start, :end - interval '1 hour', interval '1 hour') AS h(hour)
LEFT JOIN metered_allocation m ON m.type_id = :type AND m.state IN ('HELD', 'ALLOCATED')
  AND tstzrange(m.starts_at, m.ends_at, '[)') && tstzrange(h.hour, h.hour + interval '1 hour', '[)')
GROUP BY h.hour ORDER BY h.hour;
```

Isolation. This is correct only at READ COMMITTED, where each statement sees what was committed before it began, so the query, run after the lock, sees all the previous lock holder wrote. REPEATABLE READ takes its snapshot at the first statement, the lock itself, so a waiting transaction would sum stale rows and pass; a local two-session test oversold a pool of 8 to 10 that way. The path never raises the isolation level.

Worked example. A pool of 8 units holds A (3 units, 10:00 to 15:00) and B (4 units, 13:00 to 17:00); a line asks for 2 units from 14:00 to 17:00. The hourly sums are 7 at 14:00 and 4 at 15:00 and 16:00, so with the line they are 9, 6, and 6: `POOL_FULL` at 14:00, one unit short, and a Required line waits until A gives back that hour. Had the Team owned B under a Quota of 5, 14:00 would also fail the Quota ($4 + 2 > 5$); the pool is checked first.

MatchedAllocator

Shortlist. A Candidate holds every listed skill, is available on every day of the window, has no Loan overlapping any day of it, belongs to another Team, and is active. `buildShortlist` takes `availableWithSkills(skills, window, requestingTeam)` from **Members**, drops Members with an overlapping Loan, and orders the rest by person-days

lent in the current Period (weekdays of formed Loans inside it), then name: two indexed queries and a sort of at most 500 Members (**PE-3**), redone on every call so profile changes show at once. A named Person is checked only for availability and an overlapping Loan (`OWN_TEAM` for the requesting Team's own Member), and naming counts as the Manager's selection.

Approvals. `select` writes a `candidate` row (with its Shortlist position) and an `approval` for the Manager of the Candidate's Team, and notifies that Manager (requesting Team, window, Earning) and the Candidate. `decide` accepts that Manager until the window ends and a Capacity Planner after it; a late lending Manager gets `WINDOW_LAPSED`. An approval re-checks eligibility, because a Loan from another Request may have formed since selection, and fails with `NOT_A_CANDIDATE`.

Loans. A Person can be approved on two Requests with overlapping windows before either Loan forms. **Requests** therefore re-evaluates the line under the type lock at the start of `allocate`, before any commit, where `evaluate` reports an Approved Candidate who is no longer available every day, or now has an overlapping Loan, as `UNSATISFIABLE(NOT_A_CANDIDATE)`: a Required line is then unsatisfied and the Request goes to Waiting with a replacement asked; an Optional line is Dropped. `commit` repeats the check immediately before forming each Loan only as a guard: a failure there means a path skipped the re-evaluation, so it throws `RuleViolation("ALLOCATION_CONFLICT")` and the transition rolls back. The `loan` constraint stops even a path that forgets both. A Loan covers the window's dates with `days_lent` its Monday-to-Friday days. `endLoanEarly` calls `release`, which sets `ends_on` to the new last day and `ended_early` and recomputes `days_lent`, freeing the Person from the next day; the shortened Loan stays in the constraint for the days it lent.

Worked example. A line asks for two Members with one skill from Monday, October 12 to Friday, November 20. Of six skilled Members outside the Team, Ford (Loan November 2 to 6) and Gupta (available only to November 13) are left out; Table 5 orders the rest. Abe's Loan ended before the window, so it only counts toward his October days. The Manager picks two; the system picks none.

Table 5: A Shortlist of four, by person-days lent in October, then name.

Position	Member	Formed Loans	October person-days
1	Diaz	none	0
2	Baptiste	Oct 5 to 7	3
3	Chen	Oct 1 to 5 (Thu, Fri, Mon)	3
4	Abe	Sep 28 to Oct 9 (October days only)	7

Figures. Figure 6 shows the interfaces, the allocators, the registry, the lock, the queries, and the six entities; Figure 16 shows a Bump.

4.6 Ledger (`ledger`)

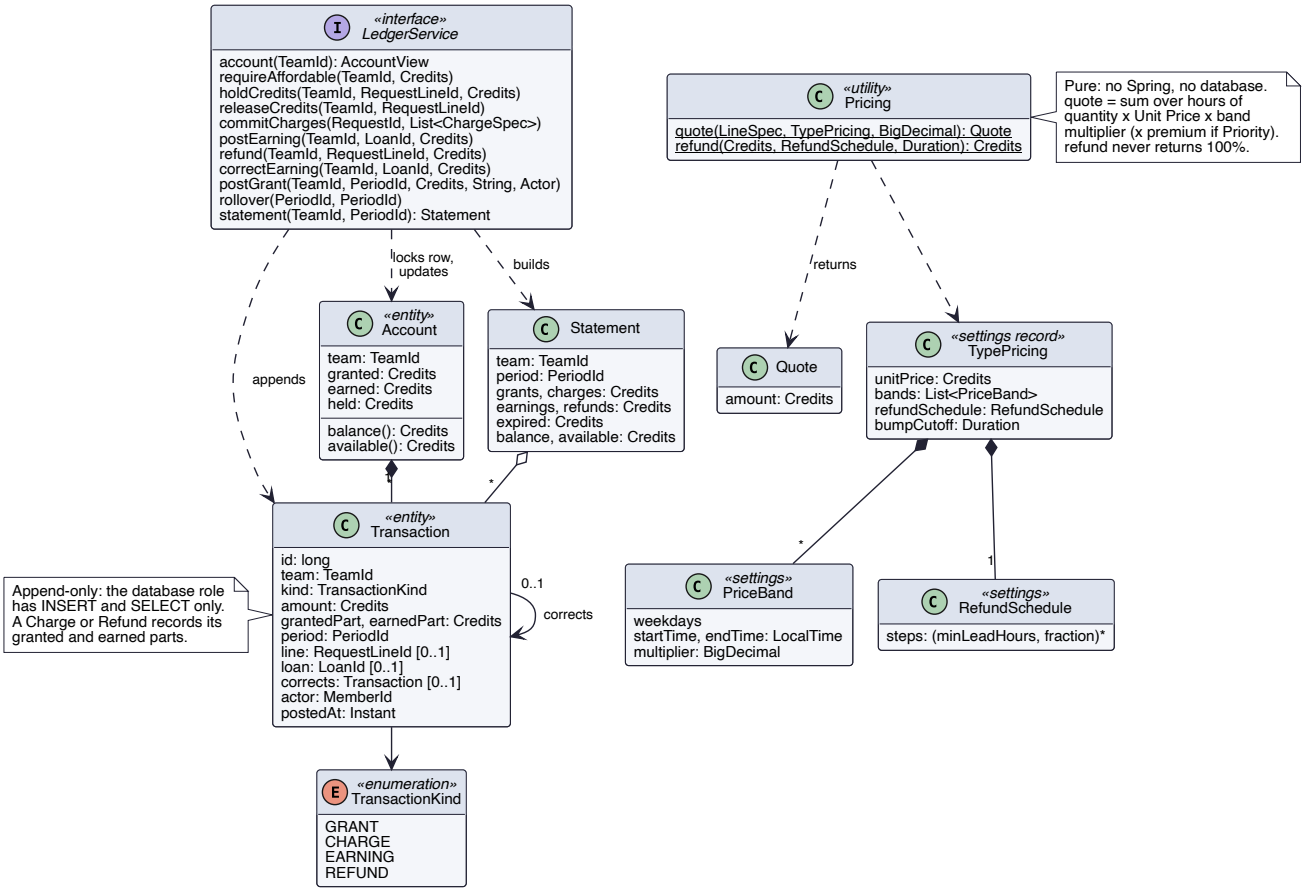


Figure 7: Classes of **Ledger**: `LedgerService`, the pure `Pricing` functions, the `Account` with its three numbers, the append-only `Transaction` with its four kinds, and the value types `Quote`, `PriceBand`, and `RefundSchedule`.

Purpose. Ledger keeps every Team's Account and the append-only record of Transactions, prices lines, and computes Refunds. It knows credits only: it holds no capacity information and never decides whether a line can be allocated.

Responsibilities.

- Accounts and Statements (**Ledger.Account**); four Transaction kinds, append-only (**Ledger.Transaction.Kinds**, **QA-3**); Grants, spend order, and rollover (**Ledger.Grant.Period**).
- The Quote formula, also used while drafting (**Ledger.Price.Formula**, **Draft.Quote.Line**), and the Refund step function (**Release.Refund**).
- Earnings and their correction after an early end (**Ledger.Earning.Post**, **Release.Loan.Prorate**), posted with Charges inside the caller's transaction (**CO-4**).

Provided interface.

```
final class Pricing {                                          // pure, no Spring, no database
  static Quote quote(LineSpec line, TypePricing pricing, BigDecimal priorityPremium);  // sum over hours of
     quantity x unitPrice x band, x premium if Priority
  static Credits refund(Credits charge, RefundSchedule schedule, Duration leadTime);  // step function, never 100%
}
interface LedgerService {
  AccountView account(TeamId team);                            // (read-only) granted, earned, held, balance, available
  void requireAffordable(TeamId team, Credits requiredSum); // no lock, early refusal; throws
     RuleViolation("INSUFFICIENT_BALANCE")
  void holdCredits(TeamId team, RequestLineId line, Credits quote);     // held += quote (locks account row,
     repeats the check)
```

```
  void releaseCredits(TeamId team, RequestLineId line);                // held -= that line's quote
  void commitCharges(RequestId request, List<ChargeSpec> charges);     // for each line: held -> Charge, granted
     first then earned, split recorded
  void postEarning(TeamId lendingTeam, LoanId loan, Credits amount);   // earned += amount
  void refund(TeamId team, RequestLineId line, Credits amount);        // returns in the Charge's proportions
  void correctEarning(TeamId lendingTeam, LoanId loan, Credits delta); // only Transaction that may make Balance
     negative
  void postGrant(TeamId team, PeriodId period, Credits amount, String reason, Actor actor);  // PLANNER or rollover
  void rollover(PeriodId ending, PeriodId starting);       // expire granted, record on Statement, post planned
     Grants; idempotent (no-op if already rolled)
  Statement statement(TeamId team, PeriodId period);       // (read-only) own Manager or PLANNER
}
```

`Pricing` is pure. **Requests** calls `quote` while drafting and at submission, when the result is locked on the line. `LineSpec` gives the quantity (instances or units), the occurrences in the firm's time zone, and the Priority mark. With q the quantity, p the Unit Price, $b(h)$ the multiplier of the Price Band containing hour h (1.0 if none), and π the priority premium for a Priority line (1 otherwise):

$$Quote = \pi \cdot \sum_{\text{occurrence } o} \; \sum_{\text{hour } h \in o} \; q \cdot p \cdot b(h).$$

Credits carry two decimals; a Quote is rounded half-up once per line. `refund` takes the step with the largest lead time not above `leadTime` and returns that fraction of `charge`, rounded down; a part already started returns 0. **Settings** refuses any step of 100% or more and `refund` asserts it.

`LedgerService` keeps granted, earned, and held credits per Team in `account`; Balance is granted plus earned, Available Balance is Balance minus held. Every change first locks the account row (`SELECT ... FOR UPDATE`, inserting it if missing, so **Members** never calls Ledger), after the type locks and until commit, so two Requests of one Team never see the same Available Balance. Other Teams' accounts (lending or Bumped Teams) are locked when reached; the rare cycle this allows ends in a deadlock abort, retried once.

Table 6: Ledger operations: preconditions and effects, and errors.

Operation	Preconditions and effect	Errors
`requireAffordable`	Reads the account without a lock and refuses a sum above the Available Balance (early refusal); `holdCredits` and `commitCharges` repeat the check under the account lock	`INSUFFICIENT_BALANCE`
`holdCredits`, `releaseCredits`	One `held_credit` row per Held line, added to or removed from held; releasing an unheld line is a no-op	`INSUFFICIENT_BALANCE`
`commitCharges`	Each amount is the locked Quote and fits the Balance; releases its held amount and posts a Charge with `granted_part` = min(granted, amount), `earned_part` the rest	`INSUFFICIENT_BALANCE`
`postEarning`, `correctEarning`	One Earning per Loan (Day Rate × `days_lent`); a correction is an Earning with a negative amount and `corrects_id`, and may take the Balance below zero	none
`refund`	The line's Refunds stay within its Charge; split in the Charge's proportions	`REFUND_EXCEEDS_CHARGE`
`postGrant`, `rollover`	Grant: amount positive, reason kept, Manager notified mid-Period. Rollover, idempotent: per Team `period_close` records expired granted credits, granted set to 0, planned Grant posted, Manager notified; earned untouched	`INVALID_AMOUNT`
`account`, `statement`	Read-only; a Statement (opening, Transactions, expired, closing) for its own Manager or a Capacity Planner; outside that scope it is not found	`NotFoundException` (404)

Required interfaces. **Settings**: `SettingsService.current` (pricing, premium, Period, planned Grants). **Members**: a Team's Manager, for Statement scoping. **Notifications**: `Notifier.notify`, by `rollover` (granted credits expired, new Grant posted; scenario S7) and by `postGrant` for a mid-Period Grant.

Data owned. `account(team_id, granted, earned, held)` and `transaction(id, team_id, kind, amount, granted_part, earned_part, period_id, line_id, loan_id, corrects_id, actor_id, posted_at)`, kinds `GRANT`, `CHARGE`, `EARNING`, `REFUND`, amounts positive except a correction, `actor_id` null for a job. Added to the brief: `transaction.reason` (mid-Period Grants), `held_credit(line_id, team_id, amount)` (a line's held amount without reading Requests' tables), and `period_close(team_id, period_id, expired, closing_granted, closing_earned, closed_at)`. `transaction` is append-only by grant, not convention: Flyway migrates as the schema owner, the application role has only `INSERT` and `SELECT` on it, and a correction is a new row naming the corrected one in `corrects_id`.

Invariants.

- Granted and held credits are never negative; the Balance is never negative except after an Earning correction.
- Held credits equal the Team's `held_credit` rows, which equal the locked Quotes of its Held lines.
- The account row equals the fold of the Team's Transactions and `period_close` rows (a test recomputes it).
- A Charge equals its locked Quote and its two parts sum to it, granted first; a line's Refunds stay within its Charge, in its proportions. Transaction rows never change.
- `rollover` is idempotent: rolling into a Period already rolled into is a no-op (the key of `period_close`), so the start-up catch-up in **Jobs** is safe.

Failure behavior. `INSUFFICIENT_BALANCE` is the refusal a Manager sees, with the shortfall. After a correction makes the Balance negative, the Available Balance is negative too, so every Request with a Required Exclusive or Metered line is refused until a Grant or Earning restores it. The methods join the caller's transaction, so a failed Charge rolls back the commit of every Hold (**CO-4**). `UPDATE` or `DELETE` on `transaction` fails with SQLSTATE 42501.

Information hiding. Callers see `Credits`, `AccountView`, and `Statement`; the granted and earned split, the spend order, rounding, and held amounts per line are hidden, so the Stretch demand surcharge is one more multiplier inside `Pricing`.

Worked examples. *Quote.* A room at Unit Price 10 for Thursday 14:00 to 17:00 on October 15, 22, and 29: every hour lies in the weekday band ($\times 1.5$), so an hour costs $1 \times 10 \times 1.5 = 15$, an occurrence 45, and the Quote is $3 \times 45 = 135$; as a Priority line, $135 \times 2 = 270$. From 16:00 to 19:00 an occurrence would cost $15 + 5 + 5 = 25$, the last two hours being outside the band ($\times 0.5$). *Refund.* Schedule: 75% at seven days or more, 25% under seven days, 0% once started. The 135-credit line is released on Monday, October 19 at 10:00. October 15 is consumed. October 22 (Charge 45) starts in 3 days 4 hours: $45 \times 0.25 = 11.25$. October 29 starts in 10 days 4 hours: $45 \times 0.75 = 33.75$. One Refund of 45.00 posts. *Spend order.* Granted 30 and earned 50 meet a Charge of 40: `granted_part` 30, `earned_part` 10, leaving 0 and 40. A later Refund of 20 on that line returns 15 to granted and 5 to earned.

Figures. Figure 7 shows `LedgerService`, `Pricing`, `Account`, `Transaction`, `Statement`, `Quote`, and `RefundSchedule`; Figure 18 shows the rollover.

4.7 Requests (`requests`)

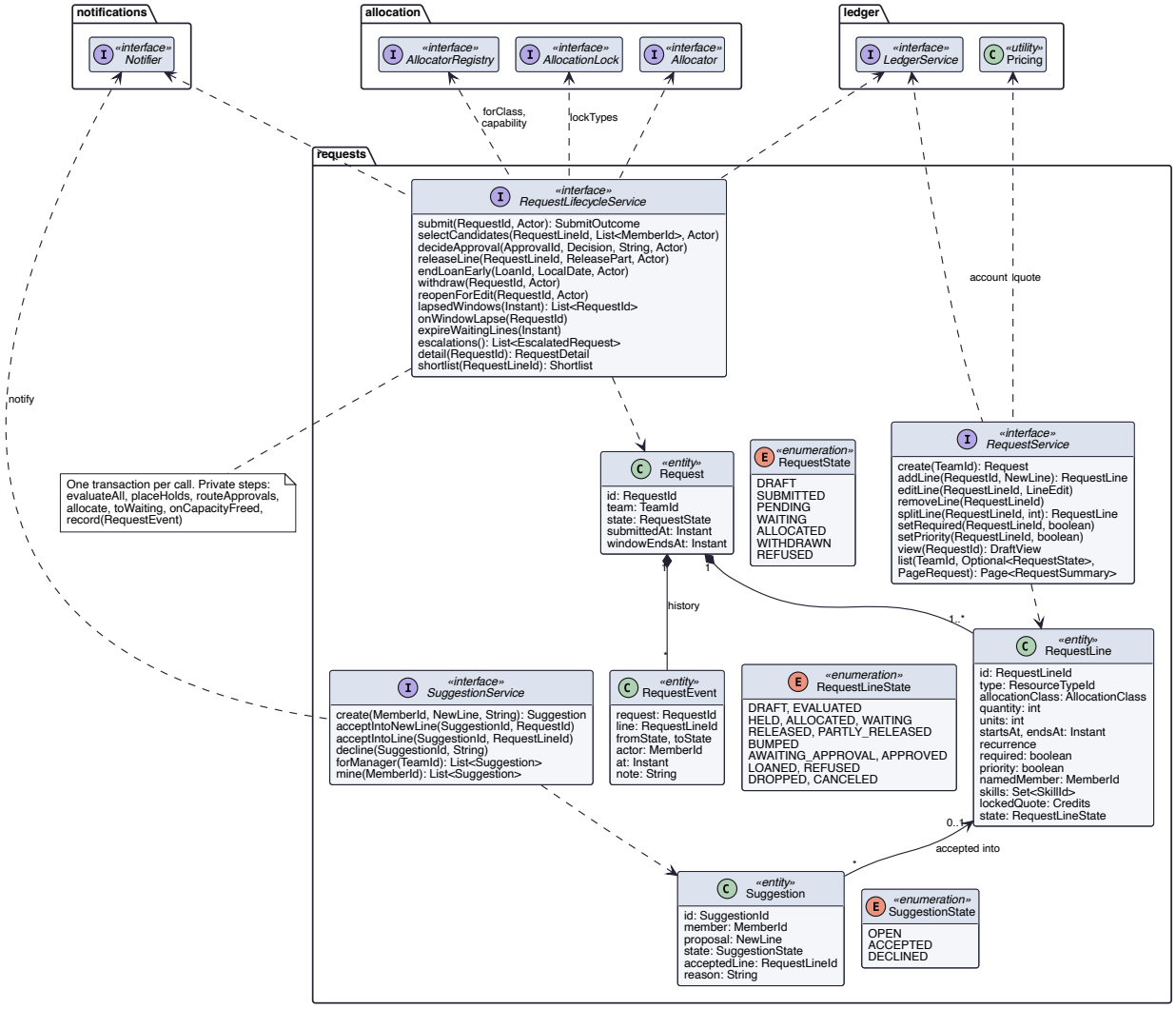


Figure 8: Classes of **Requests**: the Draft-editing and Suggestion services, the orchestrating `RequestLifecycleService`, the entities and state enums, and the module's dependencies on `Allocator`, `AllocatorRegistry`, `AllocationLock`, `LedgerService`, and `Notifier`.

Purpose. Requests owns the Request from Draft to a final state and the Suggestions that feed it. It alone changes a Request's state and alone calls both **Allocation** and **Ledger**, so the order of locks, checks, writes, and notices lives in one class.

Responsibilities.

- Suggestions (**Suggest.Create**, **Suggest.Accept.Existing**); Drafts, Quotes, and submission (**Draft.Create**, **Draft.Line.Add**, **Draft.Line.Required**, **Draft.Quote.Line**, **Draft.Submit.Lock**).
- The state machine and its history (**Lifecycle.States**, **QA-3**); evaluation, Holds, and the Objection Window (**Lifecycle.Evaluate**, **Lifecycle.Hold.Place**, **Lifecycle.Window.Start**).
- Allocation in one transaction and re-evaluation of Waiting Requests (**Lifecycle.Allocate**, **QA-2**, **Lifecycle.Waiting.Reevaluate**); replacement, escalation, and the Capacity Planner's escalation list (**Matched.Refuse.Replace**, **Matched.Escalate**, **Planner.Settings**).
- Entry points for Release and an early Loan end (**Release.Line**, **Release.Loan.Prorate**).

Provided interface.

```
interface RequestService {                                  // Draft editing; MANAGER of the owning Team
```

```
  Request create(TeamId team);
  RequestLine addLine(RequestId r, NewLine line);           // window in the future, type not retired
  void editLine(RequestLineId l, LineEdit e); void removeLine(RequestLineId l);
  RequestLine splitLine(RequestLineId l, int quantityForNewLine);   // new line same type, marked Optional by
     default
  void setRequired(RequestLineId l, boolean required); void setPriority(RequestLineId l, boolean priority);  //
     Priority: Exclusive only
  DraftView view(RequestId r);                                  // (read-only) lines with current Quotes, sums,
     Available Balance, shortfall
  Page<RequestSummary> list(TeamId team, Optional<RequestState> state, PageRequest page);  // (read-only) scoped
     to the caller
}
interface SuggestionService {
  Suggestion create(MemberId by, NewLine proposal, String note);       // MEMBER, to own Manager
  void acceptIntoNewLine(SuggestionId s, RequestId draft); void acceptIntoLine(SuggestionId s, RequestLineId
     line);  // MANAGER
  void decline(SuggestionId s, String reason);                  // MANAGER; notifies the Member
  List<Suggestion> forManager(TeamId team); List<Suggestion> mine(MemberId me);   // (read-only)
}
interface RequestLifecycleService {                             // one transaction per call; locks types then account
  SubmitOutcome submit(RequestId r, Actor actor);               // affordability, lock Quotes, evaluate, -> Allocated
     | Pending (Holds, approvals) | Waiting
  void selectCandidates(RequestLineId l, List<MemberId> members, Actor actor);
  void decideApproval(ApprovalId a, Decision d, String reason, Actor actor);   // last Required approval
     allocates; refusal -> replacement asked
  void releaseLine(RequestLineId l, ReleasePart part, Actor actor);           // Refund, free capacity, offer to
     waitlist
  void endLoanEarly(LoanId loan, LocalDate newEnd, Actor actor);              // Earning correction
  void withdraw(RequestId r, Actor actor);                      // from Draft, Pending, Waiting; cancels all lines
  void reopenForEdit(RequestId r, Actor actor);                 // Waiting -> Draft
  List<RequestId> lapsedWindows(Instant now);                   // (read-only) jobs: Requests, Pending or Waiting,
     whose Objection Window ended at or before now with an approval still Outstanding and not yet escalated
  void onWindowLapse(RequestId r);                              // jobs, idempotent: Pending: toWaiting, then
     escalate; Waiting: escalate only; skips a Request with no unescalated Outstanding approval
  void expireWaitingLines(Instant now);                         // jobs
  List<EscalatedRequest> escalations();                         // (read-only) PLANNER
  RequestDetail detail(RequestId r);                            // (read-only) states, lines, Holds, events timeline
  Shortlist shortlist(RequestLineId l);                         // (read-only) pass-through to
     ApprovalDriven.buildShortlist for the requesting Manager
}
```

`RequestService` validates added, edited, and suggested lines alike, and again at submission: window in the future and on whole hours, type not retired, a named Person outside the Team, Priority only for a `Bumpable` class. Lines are Required by default; `splitLine` is the only way to accept less. `view` prices lines with `Pricing.quote` at current settings. `SuggestionService` keeps Suggestions Open, Accepted, or Declined (only Open ones change; none is deleted); accepting into a new or existing line of the same type records `accepted_line_id` and keeps the original ask.

Table 7: Requests operations. A Request outside the caller's scope is a `NotFoundException` (404); a wrong role on a visible one is a `ForbiddenException`.

Operation	Preconditions	Errors
line edits, Suggestion `create`	Request in Draft; line valid as above	`WRONG_STATE`, `WINDOW_NOT_FUTURE`, `WINDOW_INVALID`, `TYPE_RETIRED`, `OWN_TEAM`, `PRIORITY_NOT_OFFERED`
`splitLine`; accept, decline	$1 \leq$ `quantityForNewLine` $<$ quantity; Suggestion Open, target line in a Draft, same type	`LINE_NOT_ATOMIC`; `SUGGESTION_CLOSED`, `TYPE_MISMATCH`
`submit`	Draft with a line; lines valid now; Required Quotes within Available Balance	line errors, `EMPTY_REQUEST`, `INSUFFICIENT_BALANCE` (with shortfall)
`selectCandidates`, `decideApproval`	As `select` and `decide`	codes of Table 4
`releaseLine`, `endLoanEarly`	Allocated Exclusive or Metered line, or Loan, of the Team; new Loan end before the old, not in the past	`NOT_ALLOCATED`, `LOAN_END_INVALID`
`withdraw`, `reopenForEdit`	State allows it (Table 8)	`WRONG_STATE`

`RequestLifecycleService` is the orchestrator: one `@Transactional` method per transition, retried once after a deadlock or lock timeout, always in this order: (1) lock the Resource Types of its own lines with one ascending

`AllocationLock.lockTypes`; a waitlist fill may then lock a waiting Request's other types, the one path that can take a type after an account, covered by the retry (Section 5); (2) lock the Team's account row through the first locking Ledger call (`holdCredits` or `commitCharges`), never a type after an account outside that fill; (3) read; (4) write; (5) `record`; (6) `Notifier`, whose rows share the transaction, so a rolled-back transition tells nobody. `requireAffordable` at submission reads the account without a lock, before (1), only as an early refusal.

Private steps.

`evaluateAll evaluate` every line; a Priority misfit asks `planBump`, and a plan counts as a fit.
`placeHolds` Only while the window runs: Required, then affordable Optional lines; `applyBump` where planned, `hold`, `holdCredits`.
`routeApprovals ApprovalDriven.select` for named Persons at submission and for the Manager's picks.
`allocate` Always in this order. (a) Re-run `evaluateAll` under the locks; an Approved Candidate who is no longer eligible makes the line `UNSATISFIABLE(NOT_A_CANDIDATE)`, and the system refuses that approval. If a Required line fails, `toWaiting` runs instead, before any commit, and a replacement is asked. If the Balance no longer covers the Required Charges (after a rollover or an Earning correction while Pending), `toWaiting` runs instead with reason `INSUFFICIENT_BALANCE`; the approval is kept. (b) Drop unfit, refused, or unaffordable Optional lines with the reason. (c) Apply Bump plans, then `Allocator.commit` each Exclusive and Metered line (HELD to ALLOCATED). (d) `LedgerService.commitCharges`. (e) `MatchedAllocator.commit` each Matched line, forming the Loans; it repeats the eligibility check only as a guard that throws `ALLOCATION_CONFLICT` and rolls back the transition. (f) `postEarning` per Loan from `loansFor(line)`. (g) Set the Request Allocated, `record`, and notify the Manager and each Person through `Notifier`. An Optional Matched line still awaiting approval forms its Loan when approved later.
`toWaiting cancel` and `releaseCredits` for Held lines; `join` for unsatisfied Required lines; approvals stay; `onCapacityFreed`.
`onCapacityFreed` For each line from `waitingFor(type, window)`, in order: allocate its Request if every Required line fits at once and is approved; if only approvals are missing and the window runs, back to Pending with `placeHolds`; else next.
`record` Insert a `request_event`; a null actor is the system (jobs).

Table 8: Request transitions. Submitted is transient: `submit` leaves it in the same transaction.

From	Event	Guard	To	Side effects
Draft	`submit`	Team's Manager; lines valid; Required Quotes within Available Balance	Submitted	Quotes locked, `submitted_at` set; `evaluateAll`
Submitted	evaluation	No line needs approval; every Required line fits (a Bump plan counts)	Allocated	`allocate`
Submitted	evaluation	Some line needs approval	Pending	`window_ends_at` set; `placeHolds`; misfit Required lines wait; `routeApprovals`
Submitted	evaluation	Otherwise	Waiting	`toWaiting`; misfit Optional lines dropped
Pending	approval, `onCapacityFreed`	Every Required line approved and fits at once	Allocated	`allocate`
Pending	refusal, lapse, Bump of a Held line, Candidate ineligible, Balance short at `allocate`	A Required line unsatisfied but still possible, or the Balance short of the Required Charges	Waiting	`toWaiting`; on refusal or an ineligible Candidate a replacement asked; on lapse `escalate`
Pending, Waiting	refusal	The Required line named a Person, or no Candidate is left	Refused	`Allocator.cancel` on every line
Waiting	`selectCandidates`, `onCapacityFreed`	Window running; approvals outstanding (unfit Required lines stay on the Waitlist)	Pending	`placeHolds`, window not restarted; `routeApprovals`
Waiting	approval, `onCapacityFreed`	Every Required line approved and fits at once	Allocated	`allocate`, no Hold first
Waiting	lapse	An Outstanding approval not yet escalated	Waiting	`escalate` only; nothing is held
Waiting	`reopenForEdit`	Team's Manager	Draft	`Allocator.cancel` on every line; Quotes unlocked
Allocated	Bump by another Request	Bumped line Required	Waiting	Full Refund; line at the head of its waitlist; other lines stay
Allocated	Release, early Loan end, late Optional approval	none	Allocated	Refund or Earning correction; `onCapacityFreed`
Draft, Pending, Waiting	`withdraw`	Team's Manager	Withdrawn	`Allocator.cancel` on every line; held credits released

Line states. Draft, then Evaluated. Exclusive and Metered: Held, Allocated, Released or PartlyReleased; Held to Waiting and back; Allocated to Bumped to Waiting. Matched: AwaitingApproval, Approved, Loaned, or Refused and back to AwaitingApproval on replacement. Optional lines may be Dropped; withdrawal Cancels every line. A waiting line whose start passes leaves the waitlist and stays Waiting (`expireWaitingLines`).

Window, escalation, Release. `window_ends_at` is set when the Request first enters Pending, to that instant plus `objectionWindowWorkingDays` Monday-to-Friday days (example default 2, so a Request entering Pending on Friday at 10:00 lapses on Tuesday at 10:00), and is never reset; `placeHolds` does nothing after it. Each minute `DueWorkJob` asks `lapsedWindows(now)` and calls `onWindowLapse` per Request in its own transaction. `onWindowLapse` is idempotent: a Request with no unescalated Outstanding approval is skipped, so a repeated run notifies nobody twice. For a Pending Request it runs `toWaiting`, releasing every Hold and its credits, then `escalate`; for a Waiting Request, such as one whose approval was left Outstanding after a refusal, it runs `escalate` only, so that **Matched.Escalate** holds after a refusal. `escalate` sets `escalated` on every Outstanding approval and hands it to every Capacity Planner, who decides from `escalations()`; from then on only a moment when every Required line fits at once allocates the Request. `releaseLine` prices the given-back part as its share of the locked Quote, applies `Pricing.refund` per occurrence with the lead time to that occurrence's start (a started occurrence returns 0%), posts one `refund`, calls `Allocator.release`, and runs `onCapacityFreed`. `endLoanEarly` calls `release` and posts `correctEarning` for the lost days at the original Earning's daily rate.

Required interfaces. **Allocation**: all of its interfaces. **Ledger**: `Pricing`; `LedgerService` except `postGrant`, `rollover`, `statement`. **Inventory**: `InventoryQueries.type`. **Members**: `summary`. **Settings**: `current`. **Notifications**: `Notifier.notify`.

Data owned.

- `request(id, team_id, state, submitted_at, window_ends_at)`.
- `request_line(id, request_id, type_id, allocation_class, quantity, units, starts_at, ends_at, recurrence, required, priority, named_member_id, skills, locked_quote, state)`.
- `suggestion(id, member_id, proposal..., state, accepted_line_id, reason)`, the proposal columns mirroring a line plus a note.
- `request_event(id, request_id, line_id, from_state, to_state, actor_id, at, note)`.

Invariants.

- A Request is in exactly one state; every Request or line state change writes a `request_event` in the same transaction.
- `locked_quote` and `submitted_at` are set at submission and cleared only by `reopenForEdit`; `window_ends_at` never changes once set, and Held lines exist only in a Pending Request before it.
- Lapse is judged by the injected `Clock`, not by the job: `decideApproval` compares `window_ends_at` with now, so an approval arriving after the window ended is treated as lapsed (`WINDOW_LAPSED` for the lending Manager, no Hold kept) even before `onWindowLapse` has run.
- An Allocated Request had every Required line Allocated or Loaned when it entered the state. Only Optional lines are Dropped; only Required lines wait.

Failure behavior. Any exception rolls back the whole transition, notifications included. A `RuleViolation` becomes `problem+json` with its `code` (409 state conflict, 422 invalid input); a deadlock or lock timeout, after one retry, `ConcurrencyRetryException` (503). A failed job call is logged and retried next minute; other Requests proceed in their own transactions.

Information hiding. The state machine, the private steps, and the lock order are internal to `RequestLifecycleService`. Controllers and jobs call one method per transition and cannot run a step alone, so no caller can commit Holds without posting their Charges.

Worked example. A Team with granted 2000 and earned 1000 asks, all Required, for two engineers with one skill from October 12 to November 20, four GPUs from October 12 to 26 (Unit Price 2), and a lab room on six Thursdays from October 15, 14:00 to 17:00 (Unit Price 10). The Day Rate is an illustrative 20 (Table 9).

Table 9: A three-line Request from submission to Allocated.

Step	What happens	Rows written
Draft, Mon Oct 5	Quotes 1984 (GPUs: $4 \times 2 \times (80 \times 1.5 + 256 \times 0.5)$) and 270 (room: 6×45); Required 2254, Available 3000	`request`, three `request_line`, events
`submit`, Mon 10:00	Types locked in id order, then the account; Quotes locked; GPUs and room fit, people need approval: Pending until Wed 10:00	one `metered_allocation` and six `exclusive_occurrence` `HELD`; two `held_credit` (held 2254); events; notice to pick
`selectCandidates`	Diaz and Baptiste (Table 5)	two `candidate`, two `approval`; notices
`decideApproval` Tue, Wed 09:00	Both lending Managers approve; the second triggers `allocate`; both still eligible	seven rows `ALLOCATED`; `CHARGE` 1984 (granted) and 270 (granted 16, earned 254); `held_credit` deleted; two `loan` of 30 days; two `EARNING` of 600; Allocated; events; notices

Figures. Figure 8 shows the services, entities, and state enums with their dependencies on `Allocator`, `LedgerService`, and `Notifier`; Figure 10 and Figure 11 give the state machines. The transitions run as in Figure 12 (submit), Figure 13 (last approval), Figure 14 (refusal), Figure 15 (lapse), Figure 16 (Bump), and Figure 17 (Release and waitlist fill).

4.8 Dashboard (`dashboard`)

Purpose. Dashboard answers the showback questions: how much of each Resource Type was used and how much stood idle, how many lines wait, and what each Team was granted, charged, and earned. It is the one read-only reporting module the dependency rule allows (Section 2): it reads other modules' tables through SQL views, writes nothing, and limits every answer to the caller's scope.

Responsibilities.

- **Dashboard.Utilization**: utilization per Exclusive and Metered type and Period, idle capacity for the Period to date and the next seven days, and waiting lines; this is where the unused capacity of **Release.Line** shows as idle.
- **Dashboard.Credits**: Grants, Charges, Earnings, Refunds, credits expired, Balance, Available Balance, and person-days lent and borrowed per Team and Period, scoped as **SE-4** requires and current when requested.

Provided interface.

```
// dashboard (read-only)
interface DashboardQueries {
  Utilization utilization(ResourceTypeId type, PeriodId period);   // allocated hours / available hours; idle =
     available - allocated
  Idle idleNextDays(ResourceTypeId type, int days); int waitingCount(ResourceTypeId type);
  TeamCredits credits(TeamId team, PeriodId period);        // Grants, Charges, Earnings, Refunds, expired,
     Balance, Available
  PersonDays personDays(TeamId team, PeriodId period);      // lent and borrowed
}
```

Caller's role	`utilization`, `idleNextDays`, `waitingCount`	`credits`, `personDays`
Manager	every type	own Team only
Capacity Planner	every type	every Team
Resource Owner	types of the Resources they own	none
Member with no other role	none	none (own Loans are on the Profile screen)

Several roles give the union. Scope is applied in the SQL `WHERE` clause from the caller's Team and roles, never by filtering results afterward.

Required interfaces. Read-only SQL through the views below, the documented exception to reaching other modules only through their services. **Members**: `MemberService.profile` for the caller's Team; roles come from the authenticated principal.

Data owned. No tables. Dashboard's migration defines three plain (not materialized) views. Every time bound is a query parameter from the injected `Clock`, never the database's `now()`, so tests with a fixed Clock see the same figures.

- `v_type_hours`, per type and day: allocated and available hours (instance-hours for Exclusive, unit-hours for Metered, 24 a day) from `resource` and the Allocated rows of `exclusive_occurrence` and `metered_allocation`. Instances count while in service; Metered uses the current Pool Capacity, since unit changes keep no history.
- `v_team_credits`, per Team and Period: sums per Transaction kind with granted and earned parts, expired, closing Balance, and person-days lent and borrowed, from `transaction`, `loan`, `period`, and `period_close`, plus `account` for the current Balance and Available Balance. Expired is read from `period_close`, the row `rollover` writes for each Team and closed Period, so dashboard and Statement agree.
- `v_waiting`, per waiting line: type and window start, from `waitlist_entry`.

Invariants.

- Dashboard writes nothing. Every method is `@Transactional(readOnly = true)`, which the PostgreSQL driver runs as a `READ ONLY` transaction, so an accidental write fails in the database; the ArchUnit test also forbids saving repositories here.
- Figures include everything committed before the query began: plain views are read at request time and nothing is cached. A Hold is not utilization.

Failure behavior. An unknown type or Period raises `NotFoundException`, and so does an object outside the caller's scope (404). The one `RuleViolation`, `CLASS_MISMATCH`, refuses utilization of a Matched type. PostgreSQL refuses to drop or retype a column a view uses, so another module's migration that would break a view fails at start-up and must redefine it.

Information hiding. The Java code sees only the views, never the tables behind them, so a change to another module's tables changes a view definition and no Java; callers see result records, not SQL.

Worked example. A Metered type with Pool Capacity 8 has 192 unit-hours available on 15 October 2026. With 4 units allocated from 09:00 to 17:00 (32 unit-hours) and 2 for the whole day (48), utilization is $80/192 = 41.7\%$ and idle capacity 112 unit-hours. A 2-unit Hold from 14:00 to 16:00 changes neither figure.

Figures. Figure 2 draws Dashboard's read-only access as dashed arrows.

4.9 Jobs (`jobs`)

Purpose. Jobs runs the timed work inside the server process: lapsing Objection Windows, removing waiting lines whose window has started, and rolling a Period over. It decides when work is due and calls the module that owns it; it never decides what the work does. It also declares the `Clock` bean through which every module reads time.

Responsibilities.

- **CO-5**: timed work as scheduled jobs in the server process. **PE-4**: lapse, Hold expiry, and Waitlist expiry within five minutes of due time; rollover within fifteen minutes of the boundary.
- **Lifecycle.Window.Start**: the lapse, through `onWindowLapse`. A Hold never outlives its Objection Window, so Hold expiry is the same event and needs no job of its own.
- **Ledger.Grant.Period**: the rollover, through `LedgerService.rollover`. **Exclusive.Waitlist.Join**: a waiting line leaves the Waitlist when its window start passes, through `expireWaitingLines`.

Provided interface. Jobs has no callers; its interface is its schedule.

```
class DueWorkJob { @Scheduled(fixedDelay = 60_000) void run(); }   // lapsedWindows(now) -> onWindowLapse;
    expireWaitingLines
class PeriodRolloverJob { @Scheduled(cron = "0 5 0 * * *") void run(); @EventListener(ApplicationReadyEvent) void
    catchUp(); }
```

Job	Trigger	Action (one transaction per call)	Lag (limit)
`DueWorkJob.run`	60 s after the last run ended	`onWindowLapse` for each Request from `lapsedWindows(now)`; then `expireWaitingLines(now)`	about 1 min (5)
`PeriodRolloverJob.run`	00:05 daily, company zone	if the current Period starts today, `rollover(previous, current)`	5 min (15)
`PeriodRolloverJob.catchUp`	application start	`rollover(previous, current)`	at start-up

Both jobs run under a system security context, so `@PreAuthorize` admits them and every audit row they cause names the system as actor. The `Clock` bean is `Clock.system(companyZone)`; tests substitute a movable clock and call `run()` directly.

Required interfaces. **Requests**, through `RequestLifecycleService`: `lapsedWindows`, `onWindowLapse`, and `expireWaitingLines`. The read-only `lapsedWindows(now)` returns the Requests, Pending or Waiting, whose Objection Window ended at or before `now` with an approval still Outstanding and not yet escalated (escalated is a flag on the approval). `onWindowLapse` skips a Request with no unescalated Outstanding approval; for a Pending Request it releases the Holds and held credits, moves the Request to Waiting, and escalates, and for a Waiting one it only escalates. **Ledger**: `LedgerService.rollover`. **Settings**: `SettingsService.current`, for the current Period and its predecessor.

Data owned. None. All state is in the owning modules' tables, so a restart loses nothing: the first run after start-up finds every window that lapsed meanwhile, and `catchUp` handles a missed Period boundary.

Invariants.

- Jobs are idempotent: running one twice for the same due item has the effect of running it once. Each call re-checks state in its own transaction, so a Request with no unescalated Outstanding approval is skipped, and `rollover` ignores a boundary it has already recorded.
- Each Request is its own transaction, so one failure never blocks the others. No job acts before the due time read from the `Clock`, and the job's lag never lengthens a window, because **Requests** compares the window end with the Clock on every decision.
- One instance runs the jobs; a second would need a job lock table (Section 9).

Failure behavior. Jobs defines no `RuleViolation` codes. When the call for one Request throws, a concurrency failure included, the error is logged with the Request id and the run moves on; the Request still has a lapsed window and an unescalated Outstanding approval, so the next run retries it. A failed rollover is logged as an error and retried by the next start-up's `catchUp`, which covers one missed boundary; a longer outage is outside **QA-4**.

Information hiding. Jobs does not know what a lapse or rollover does, and **Requests** and **Ledger** do not know a timer calls them: they receive ids and instants. Polling is private to Jobs, so per-item timers or a job lock table would change no other module.

Worked example. An Objection Window ends at 09:03:00. The run that starts at 09:02:59 finds nothing; it ends within a second, so the next run starts at 09:04:00, finds `window_ends_at` $\leq$ now, and calls `onWindowLapse`. The Request is Pending, so in that one transaction the Holds and held credits are released, the Request moves to Waiting, and its outstanding approvals are escalated to the Capacity Planner, each event recorded at 09:04:00 with the system as actor: one minute after the due time, inside the five-minute bound.

Figures. Figure 15 (lapse) and Figure 18 (rollover).

4.10 Web client (`web`)

Purpose. The Web client is the single-page browser application through which every role works. Its only interface to the system is the REST API: it renders what the server computes and sends what the user decides, and it holds no business rule.

Responsibilities.

- **UI-1**, **UI-2**, **UI-3**: the screens the SRS names, with their columns and actions.
- **QA-5**: a three-line Request drafted and submitted on one screen in under five minutes. **QA-4**: current Chrome, Firefox, Safari, and Edge, desktop layout only.
- **SI-2**, **CO-3**: every action through the same REST API any client could call, with the same role checks. Field help (**UD-3**) ships as static text.

Provided interface. The client's interface is its routes. Each screen calls the API groups of the modules named (endpoints in Section 7); the roles are those the server enforces, which the client uses only to choose menu items.

Screen	Route	API groups	Roles
Requests	`/requests`	**Requests**	Manager; Planner reads
Draft Request	`/requests/:id` (Draft)	**Requests** (Draft view, line edits, split, submit, withdraw, Suggestions), **Inventory** (types)	Manager
Request detail	`/requests/:id`	**Requests** (detail, Release, end Loan early), **Allocation** (Waitlist position)	Manager; Planner reads
Suggestions	`/suggestions`	**Requests** (Suggestions)	Member; Manager decides
Shortlist	`/shortlist/:lineId`	**Requests** (`shortlist`, `selectCandidates`)	requesting Manager
Approvals	`/approvals`	**Allocation** (outstanding), **Requests** (`decideApproval`)	lending Manager
Inventory	`/inventory`, `/inventory/resources/:id`	**Inventory**, **Allocation** (Holds timeline, free for a window)	everyone; Resource Owner edits own
Dashboard	`/dashboard`	**Dashboard**	by role
Statement	`/statement`	**Ledger** (Statement; mid-Period Grant)	Manager (own); Planner
Settings	`/settings`	**Settings**; escalated Requests from **Requests**	Capacity Planner
Escalations	`/escalations`	**Requests** (`escalations`, `decideApproval`)	Capacity Planner
Administration	`/admin/members`, `/admin/teams`, `/admin/skills`, `/admin/types`	**Members**, **Inventory** (Resource Types)	Administrator
Profile	`/profile`, `/profile/:memberId`	**Members**, **Allocation** (Loans)	self; own Manager
Guide	`/guide`	none (static, one page per role)	everyone

Navigation and roles. `/login` posts to Spring Security's form login; the client then fetches the session (Member, Team, roles) once. The menu offers the screens above as the roles allow, and a header on every route shows the signed-in Member, the Team, and the unread notifications. A route the roles do not allow shows a "not allowed" page, a convenience only, since the server refuses the call anyway (**SE-1**).

State and data fetching. Server state lives only in the TanStack Query cache, keyed by resource (a Request, its Draft view, the Account); a mutation invalidates the keys it affects, so a submit refreshes the Request, the list, and the Account. Form state stays in its screen; there is no other store. Types are generated from the OpenAPI document by `openapi-typescript`, and a back-end test fails when the live document differs from the committed copy. A small fetch wrapper adds the CSRF header, turns `problem+json` into a typed error with its `code`, and sends a 401 to `/login`. The Draft Request screen never prices anything: each edit is saved and the Draft view refetched, and its Quotes, totals, and shortfall are drawn as returned, so the Quote shown is the Quote locked. Unread notifications refresh every 30 seconds.

Packaging. The client lives in `web/`. `frontend-maven-plugin` installs a pinned Node, runs `npm ci` and the Vite build, and writes the output into the jar's static resources, so one `mvn package` yields the one image. Spring serves `index.html` for every path outside `/api`, so deep links work; in development Vite proxies `/api` to the back end.

Required interfaces. The REST API only (Section 7), described by the OpenAPI document at `/api/docs`.

Data owned. None on the server; in the browser, the in-memory query cache (cleared on sign-out) and the `HttpOnly` session cookie. Nothing goes to browser storage.

Invariants.

- The Web client never computes a Quote, a Balance, eligibility, or a state transition; it shows what the API returns, which is already scoped to the signed-in Member.
- Every write is an API call carrying the CSRF header; hiding a menu item is never the only guard.

Failure behavior. A `RuleViolation` (409 or 422) is shown at the field or line concerned, worded from a message table keyed by `code`. A 403 shows the "not allowed" page, a 401 returns to sign-in, a 503 asks the user to try again. Reads are retried up to three times, writes never automatically.

Information hiding. Screens know only the generated API types, never the server's tables or rules. Session, CSRF, and error handling are hidden in the fetch wrapper, and screens share data only through query keys, so one screen can be rewritten without touching another.

Figures. Figure 9 shows the routes per role and the API groups each calls.

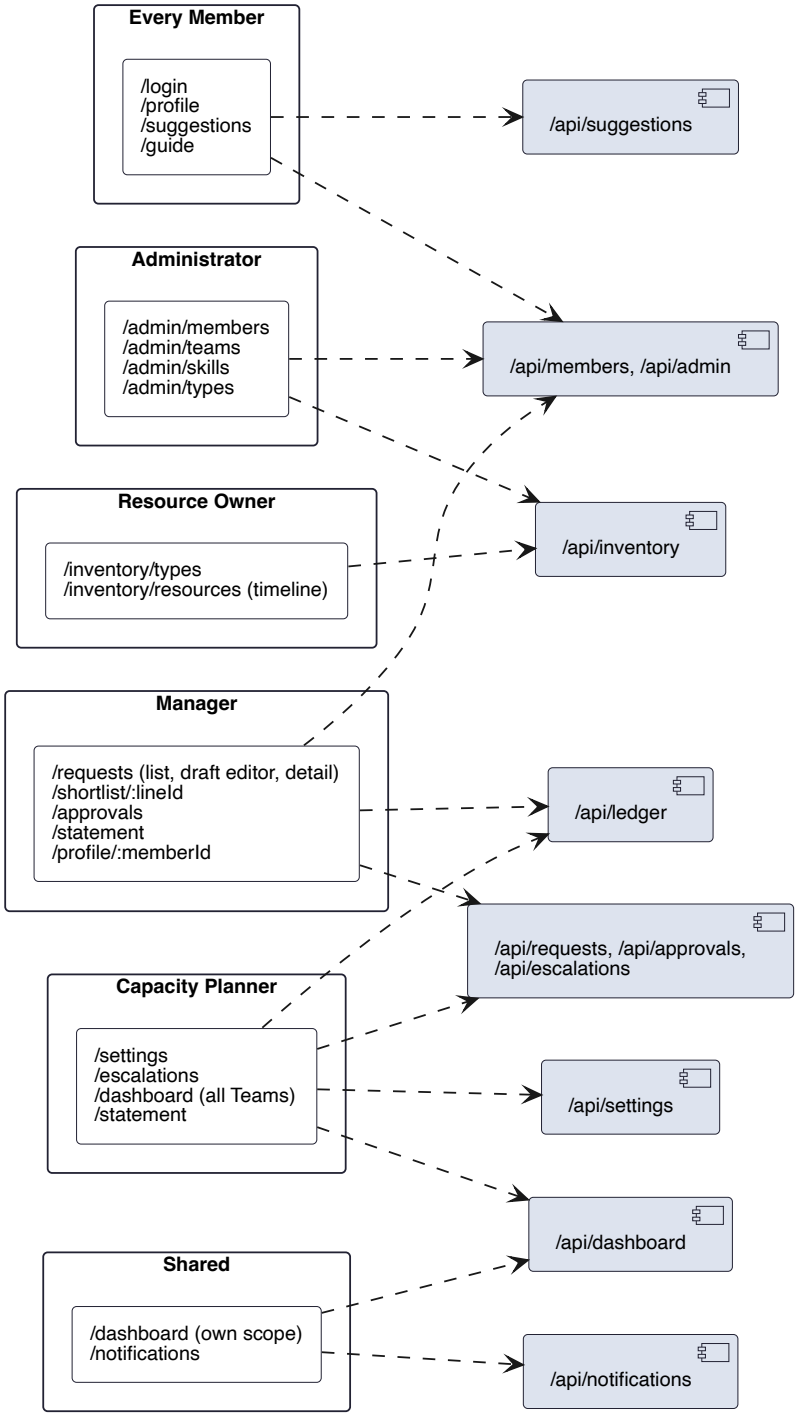


Figure 9: Routes of the web client grouped by the role that uses them, and the API groups each route calls (dashed arrows). The client holds no business rules: every Quote, Shortlist, and state change comes from the API.

5 Runtime View

This section shows the modules at run time: the locking discipline that keeps allocation correct under concurrent use, the state machines, and seven scenarios traced through the provided interfaces of Section 4. The private steps of `RequestLifecycleService` (`evaluateAll`, `placeHolds`, `routeApprovals`, `allocate`, `toWaiting`, `onCapacityFreed`, `record`) are specified with the **Requests** module (Section 4.7).

5.1 Transactions, locks, and the correctness guarantee

Every user or job action is one call to one `@Transactional` application-service method, which is one database transaction; modules call each other's services inside it, so a submission that places Holds, sets credits aside, writes history, and notifies lending Managers happens completely or not at all (**QA-2**, **CO-4**). No method sets an isolation level, so all run at PostgreSQL's default, READ COMMITTED, and `spring.jpa.open-in-view` is off. The correctness guarantee (**QA-1**, ADR-0006) is one sentence:

> *Every allocation decision runs in one PostgreSQL READ COMMITTED transaction that first locks the row of each Resource Type it touches, in ascending id order, so checks and writes for the same type never interleave; an exclusion constraint additionally makes it impossible for two Held or Allocated occurrences on the same Exclusive instance to overlap.*

Lock order. An allocating method locks before it reads any capacity, always in this order: (1) through `AllocationLock.lockTypes`, the `resource_type` row of every type its Request's lines name, Person included for Matched lines, one `SELECT ... FOR UPDATE` per id in ascending order; (2) in `LedgerService.holdCredits` or `commitCharges`, the Team's `account` row, where the Available Balance is checked; (3) then it writes. Type ids are read first as plain numbers (a line's type never changes after submission) and entities are loaded only after the locks, so the persistence context never holds a row read before the lock. A type-level lock makes choosing the least-used free instance (**Exclusive.Conflict**) and inserting it one step; the Person type serializes every Matched decision, so the one-Loan check of **RC-1** cannot race; and two actions on one Request, such as a last approval and a window lapse, run one after the other. Plain reads never wait. `requireAffordable` reads the account without a lock, before the type locks, only as a quick refusal; the binding check is repeated under the account lock. Only a waitlist fill takes a type after an account (see "The one retry" below).

Why READ COMMITTED is required. The lock serializes the Metered check only if the hourly sum is read by a statement that starts after the lock is granted. At READ COMMITTED each statement sees what was committed before it began, so the sum after `lockTypes` sees the rows of the transaction it waited for. At REPEATABLE READ the snapshot is taken by the transaction's first statement, the locking `SELECT` itself, before it waits, so the sum is computed from a snapshot older than the commit it waited for. The local check behind this design ran two sessions on PostgreSQL 16, each adding 5 units to a pool of 8: at READ COMMITTED the second session saw the first and refused; at REPEATABLE READ with the same lock both sessions committed, 10 of 8 units. The isolation level is therefore never raised on an allocation path, and the ArchUnit test class rejects an `isolation` attribute on `@Transactional` in **Allocation** and **Requests** (Section 8).

What the exclusion constraint adds. The lock binds only code that takes it. The exclusion constraint on `exclusive_occurrence` (Section 6) is checked by PostgreSQL on every write from every code path: one that forgot the lock, a scheduled job, a repair typed into `psql`, or a bug. Since instances are chosen under the lock, a violation (SQLSTATE 23P01, `DataIntegrityViolationException`) means a bug: the transaction rolls back, nothing is allocated, and the error is logged and answered 409 `ALLOCATION_CONFLICT`. A weekly recurrence inserts all its occurrences in one transaction, so one collision allocates none. A second constraint on `loan` keeps a Person out of two overlapping Loans. A Metered sum over many rows cannot be a constraint, so Pool Capacity and Quota rely on the lock alone.

Bump flush order. At flush Hibernate runs inserts before updates, and with identity keys it inserts an entity as soon as it is persisted. A Bump turns the Bumped occurrences to BUMPED and inserts the Priority occurrence on the same instance and window; in Hibernate's order the insert meets the old live rows and fails, as the local check confirmed. `Bumpable.applyBump` therefore marks the old rows BUMPED and calls `flush()` before it persists the new occurrence. The constraint stays NOT DEFERRABLE, so a real overlap fails inside the method, with its context, not at commit.

The one retry. Paths that lock in one order cannot wait for each other in a cycle. Three rare cases remain: a waitlist fill that locks a waiting Request's other types after holding the freed type; a second Team's `account` row (an Earning, or a Refund to a Bumped Team) written in the opposite order by another transaction; and a wait longer than `lock_timeout` (5 s on the application's connections). Spring reports these (40P01, 55P03) as

`PessimisticLockingFailureException`, after the transaction has rolled back, so the retry wraps the transactional proxy: controllers and jobs call allocating services through a helper in `common` that calls the method once more in a new transaction. A second failure becomes `ConcurrencyRetryException`, HTTP 503 with a stable `code`; nothing has changed.

Notifications are rows. `Notifier.notify` inserts a `notification` row in the caller's transaction (**CI-2**), so a rolled-back action notifies nobody and a retried one notifies once. Nothing is sent after commit in Release 1; email (Stretch) would add an after-commit step (Section 9).

Table 10: Locks and writes per path. "Types" are `resource_type` rows locked in ascending id by `AllocationLock.lockTypes`; every path also writes `request_event` and `notification` rows.

Path	Locks taken, in order	What it writes
Submit (`submit`)	Types of every line; then the Team's `account`	`request`, `request_line` (locked Quotes); `candidate`; `approval`; occurrence and metered rows (HELD, or ALLOCATED at once); `waitlist_entry`; `account`; `transaction` (CHARGE)
Approval decision (`decideApproval`)	Types of the Request's lines; the requesting Team's `account`; each lending Team's `account`	`approval`; when it allocates: rows to ALLOCATED, `loan`, `transaction` (CHARGE, EARNING), `account` rows; when a Required line is refused: rows HELD to EXPIRED, `account` (held), `waitlist_entry`
Release (`releaseLine`, `endLoanEarly`)	The line's type; the Team's `account`; for a Loan only the lending Team's `account`	Released rows; `transaction` (REFUND, or an EARNING correction and `loan`); `account`; then the waitlist fill
Bump (`applyBump`)	The Priority line's type (already held); the Bumped Team's `account`	Old rows to BUMPED, flush, new occurrence; `transaction` (REFUND) or held credits released; `waitlist_entry`; the Bumped `request`
Window lapse (`onWindowLapse`)	Types of the Request's lines; for a Pending Request the Team's `account`	Pending only: rows HELD to EXPIRED, `account` (held), `waitlist_entry`, `request`, then the waitlist fill; Pending or Waiting: `approval.escalated` set
Waitlist fill (`onCapacityFreed`)	Inside the freeing transaction (freed type held); the waiting Request's other types; its Team's `account`	`waitlist_entry` deleted; rows HELD or ALLOCATED; when it allocates, `account`, `transaction`, `loan`; `request`
Rollover (`rollover`)	Every `account` row, ascending Team id; no types	`period_close` (expired amount, closing balances); `account` (granted to zero, then the Grant); `transaction` (GRANT)

5.2 State machines

Request. `request.state` holds one of the seven states of Figure 10, unchanged from the SRS; only `RequestService` and `RequestLifecycleService` change it, each change with a `request_event` row (**Lifecycle.States**, **QA-3**). Submitted is entered and left inside `submit`'s transaction, so no other transaction ever reads it; it exists so that the history records the submission apart from its outcome. Withdrawn and Refused are final, and an Allocated Request stays Allocated after its windows pass. Table 11 gives every transition.

Table 11: Request transitions. Every row also calls `record` and sends the notifications of **Lifecycle.Allocate**.

From	Event (operation)	Guard	To	Side effects
(start)	`RequestService.create`	Caller is the Team's Manager	Draft	Nothing locked or held.
Draft	Line edits; a Suggestion accepted	Window in the future; type not retired	Draft	Quotes shown (`Pricing.quote`).
Draft	`submit`	Required sum $\leq$ Available Balance	Submitted	Quotes locked; `submitted_at` set; types, then account locked.
Submitted	`evaluateAll`	No approval needed; every Required line satisfiable	Allocated	`allocate`: unfit Optional lines Dropped, rows written as ALLOCATED, `commitCharges`.
Submitted	`evaluateAll`	Some line needs approval	Pending	`placeHolds`; Shortlists built; unfit Required lines waitlisted; `routeApprovals`; Objection Window starts.
Submitted	`evaluateAll`	No approval needed; a Required line unsatisfiable	Waiting	`toWaiting`: unfit Required lines join the Waitlist; nothing held.
Pending	`decideApproval` (approve), or a waitlist fill	Every Required line satisfied; Candidate re-check passes	Allocated	`allocate`: Holds committed, `commitCharges`, Loans formed, `postEarning`.

continues on next page

Table 11 (continued)

From	Event (operation)	Guard	To	Side effects
Pending	Refusal with a Candidate left; `onWindowLapse`; a Required held line Bumped; re-check fails; Balance short at `allocate`	A Required line no longer satisfied, or the Balance short of the Required Charges	Waiting	`toWaiting`: Holds and held credits released, Required lines waitlisted, capacity offered; on refusal a replacement asked; on lapse `escalate`.
Pending	`decideApproval` (refuse)	No Candidate left, or a named Person	Refused	`Allocator.cancel` on every line; capacity offered.
Waiting	`selectCandidates`, `onCapacityFreed`	Objection Window not lapsed; approvals outstanding	Pending	`routeApprovals`; `placeHolds` again; unfit Required lines stay on the Waitlist; window not restarted.
Waiting	`onCapacityFreed`, `decideApproval`	Every Required line fits now and is approved	Allocated	`allocate` at the locked Quotes, rows written as ALLOCATED directly.
Waiting	`reopenForEdit`	None	Draft	Lines leave the Waitlist; open approvals canceled.
Waiting	`decideApproval` (refuse)	No Candidate left, or a named Person	Refused	As Pending to Refused.
Waiting	`onWindowLapse`	An Outstanding approval not yet escalated	Waiting	`escalate` only; nothing held or released.
Allocated	`applyBump`	The Bumped line is Required	Waiting	Full Refund; line at the head of the Waitlist; other lines stay allocated.
Allocated	`releaseLine`, `endLoanEarly`	Part not yet consumed	Allocated	Refund (`Pricing.refund`) or Earning correction; capacity offered.
Draft	`withdraw`	None	Withdrawn	Nothing to release.
Pending	`withdraw`	None	Withdrawn	`Allocator.cancel` on every line; capacity offered.
Waiting	`withdraw`	None	Withdrawn	Lines leave the Waitlist; open approvals canceled.

Request Line. Each line has its own state (Figure 11). A line is Draft until submission and Evaluated at once. An Exclusive or Metered line becomes Held, Allocated, or Waiting; a Held line becomes Allocated when the Request allocates or Waiting when its Hold is released, and a Waiting line becomes Held or Allocated when capacity frees. An Allocated line can be Released, PartlyReleased, or Bumped, and a Bumped line is Waiting at the head of its Waitlist. A Matched line is AwaitingApproval, then Approved, then Loaned when the Request allocates; a Refused line returns to AwaitingApproval when a replacement is selected. An Optional line can be Dropped, and every line is Canceled when its Request is withdrawn (the lines of a Refused Request likewise). Allocation rows have a smaller state: HELD, ALLOCATED, RELEASED, BUMPED, or EXPIRED, and only HELD and ALLOCATED rows count against capacity.

Approval. An `approval` is Outstanding from `ApprovalDriven.select` until the Candidate's Manager decides it, Approved or Refused, with actor, time, and reason. `decide` first re-checks that the Candidate is still eligible and refuses an ineligible approval with `RuleViolation("NOT_A_CANDIDATE")`. When the Objection Window lapses, `escalate` sets `escalated` on every Outstanding approval; the flag is not a state and stays set after the decision. Only a Capacity Planner may then decide such an approval, and later approvals on that Request are created with `escalated` set (**Matched.Escalate**). `Allocator.cancel` marks the open approvals of a canceled line Canceled.

5.3 Scenarios

Each scenario has one sequence diagram with numbered messages named after the operations of Section 4 and `alt` boxes for the branches. Each user or job action is one transaction that locks as in Table 10.

5.3.1 S1 Submit a Request (UC-4)

1. The web client posts `/api/requests/{id}/submit`; the controller calls `RequestLifecycleService.submit`.
2. `submit` calls `LedgerService.requireAffordable`, an early check that takes no lock, with the Required lines' Quotes, locks every Quote, records `submitted_at`, and moves the Request to Submitted.
3. `AllocationLock.lockTypes` locks the lines' types; `evaluateAll` calls `Allocator.evaluate` on each line's allocator (`AllocatorRegistry.forClass`), which answers SATISFIABLE, NEEDS_APPROVAL, or UNSATISFIABLE.
4. If no line needs approval and every Required line is satisfiable, `allocate` Drops unfit Optional lines, writes the rows as ALLOCATED at once, and calls `commitCharges`; the Request is Allocated.
5. If a line needs approval, `placeHolds` calls `Allocator.hold` and `LedgerService.holdCredits` for each satisfiable Exclusive and Metered line; `buildShortlist` computes each skill-based Shortlist on read; nothing is stored

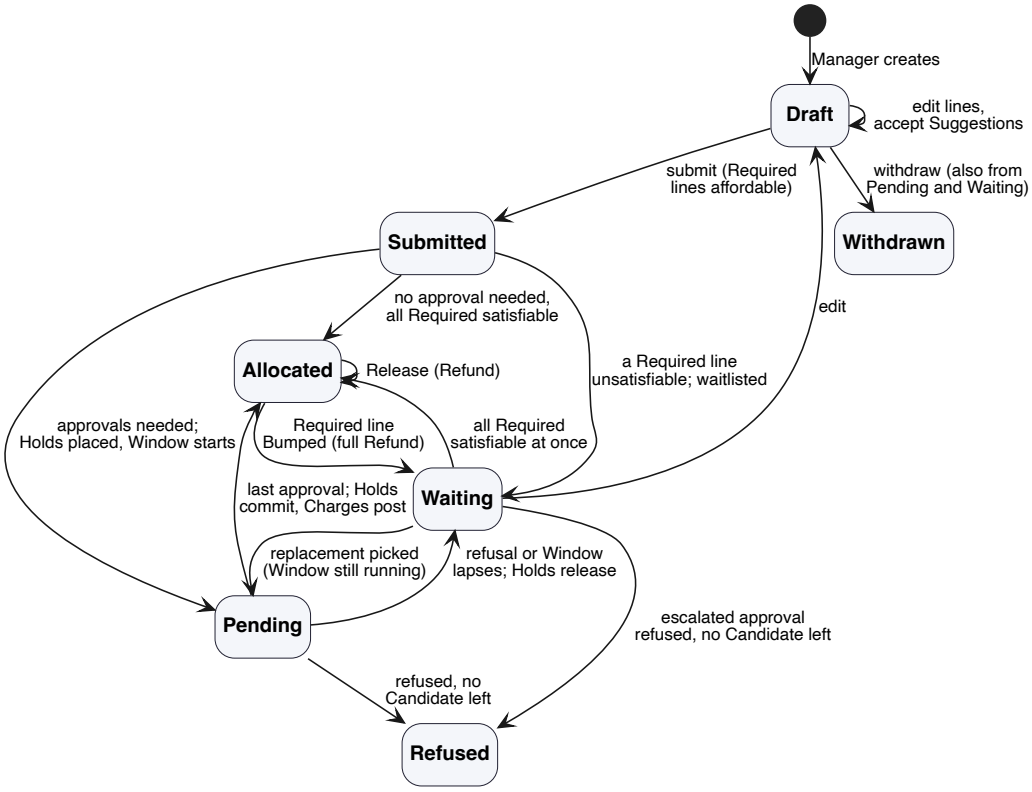


Figure 10: Request state machine, unchanged from the SRS. Withdraw is drawn once, from Draft; it is also allowed from Pending and Waiting. Table 11 gives the guard and side effects of each arrow.

until `select` writes `candidate` rows; `routeApprovals` creates named-Person approvals now and skill-based ones when the Manager picks Candidates (`selectCandidates`, `ApprovalDriven.select`), each notifying the Candidate's Manager; the Objection Window starts (`window_ends_at` is the instant the Request enters Pending plus `objectionWindowWorkingDays` Monday-to-Friday days) and the Request is Pending.

6. Otherwise `toWaiting` puts the unfit Required lines on the Waitlist and the Request is Waiting. `record` logs every change and the transaction commits.

Failure branches.

- The Required sum exceeds the Available Balance: `RuleViolation("INSUFFICIENT_BALANCE")`; nothing is locked or written and the Request stays Draft. The same code comes from `holdCredits` or `commitCharges` if a concurrent action of the Team spent the credits first.
- A window no longer starts in the future, or a type was retired: `RuleViolation`; the Request stays Draft.
- A Required Exclusive or Metered line is unsatisfiable: it joins the Waitlist, and the Request is Waiting, or Pending if another line needs approval. An unsatisfiable Optional line is never waitlisted and is Dropped at allocation.
- A skill-based Matched line has fewer eligible Members than its quantity: it is unsatisfied; a later profile change rebuilds its Shortlist.
- A Priority line finds no free instance: S5 runs inside this transaction.

5.3.2 S2 Last approval allocates (UC-6)

1. The lending Manager approves; `decideApproval` with `APPROVE` locks the Request's types.
2. `ApprovalDriven.decide` checks the actor (the Candidate's Manager, or after escalation a Capacity Planner), re-checks the Candidate's eligibility, and records Approved.
3. If a Required line is still unsatisfied, the Request stays Pending. Otherwise `allocate` re-runs `evaluateAll` under the locks; an approved Candidate who is no longer eligible (unavailable on a day, or in an overlapping Loan) makes the line UNSATISFIABLE(NOT_A_CANDIDATE).
4. Unfit, refused, or unaffordable Optional lines are Dropped with a reason; `Allocator.commit` turns HELD rows

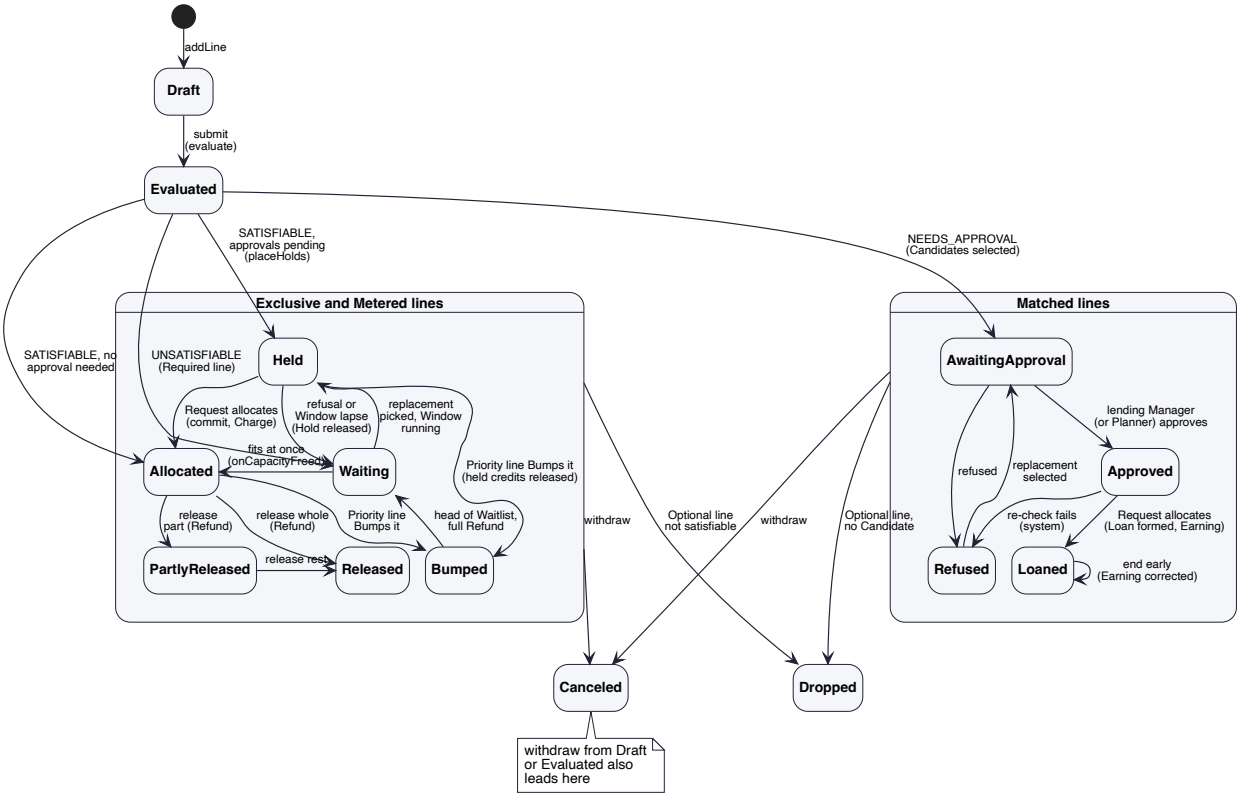


Figure 11: Request Line state machine. Exclusive and Metered lines move through Held, Allocated, Waiting, and Bumped; Matched lines through AwaitingApproval, Approved, and Loaned. Dropped (Optional lines) and Canceled are reachable from any state and drawn once.

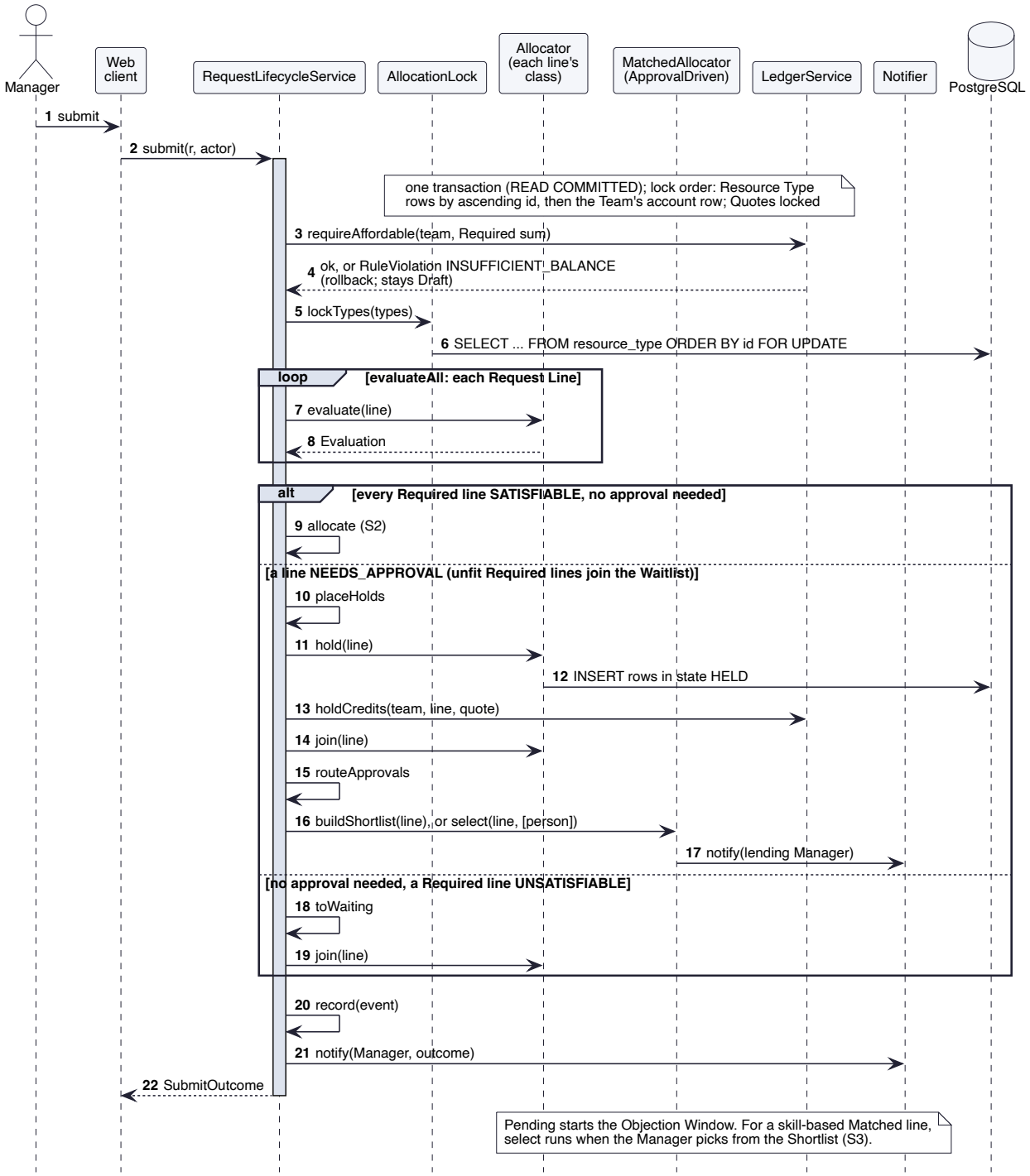


Figure 12: S1, submit a Request (UC-4). The `alt` box shows the three outcomes of evaluation (allocated at once, Pending with Holds and approvals, Waiting); the refusal for an insufficient Available Balance comes before any lock.

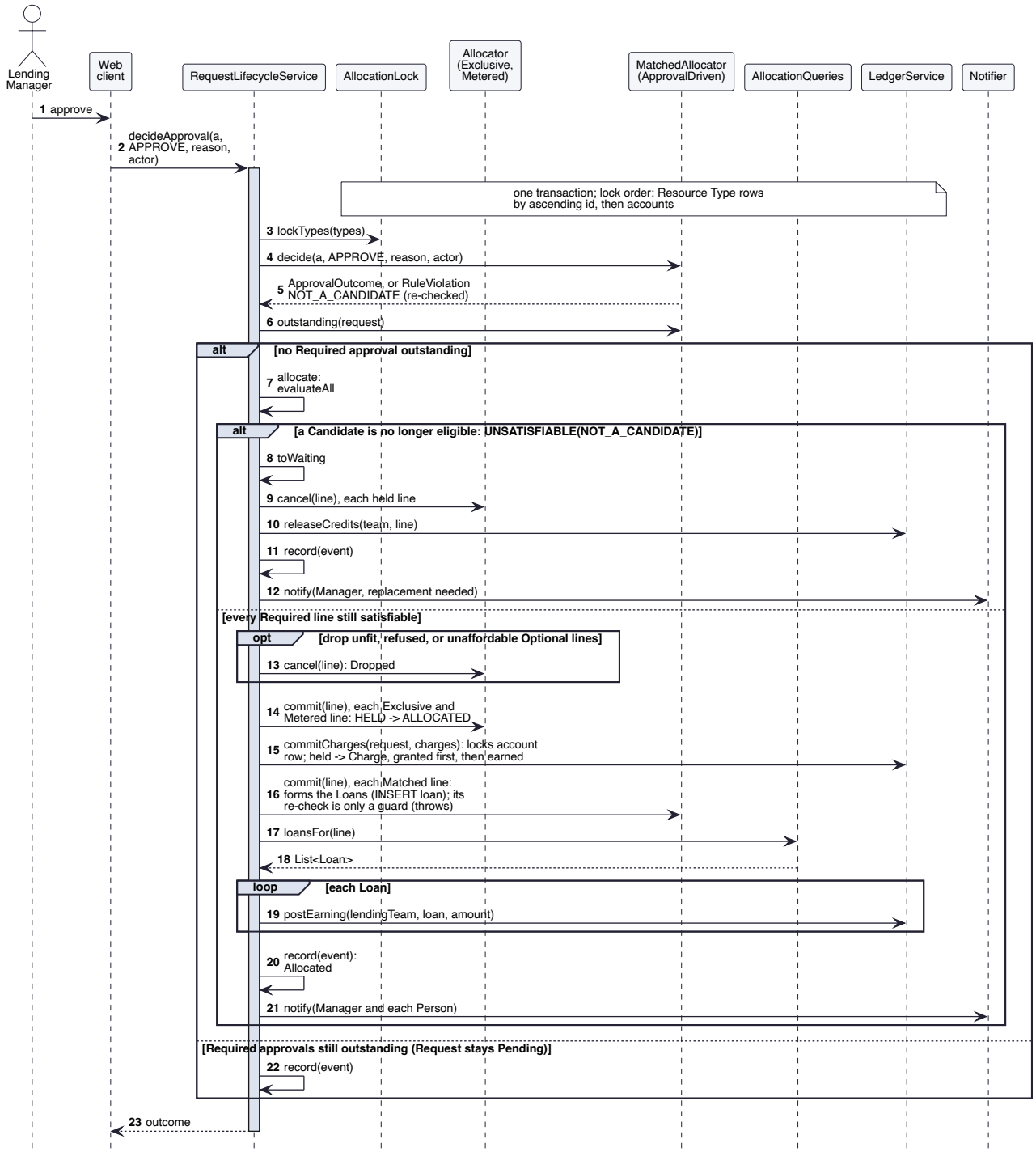


Figure 13: S2, the last Required approval allocates the Request (UC-6). Holds commit and Charges, Loans, and Earnings post in one transaction; the `alt` box shows a Candidate who is no longer eligible.

into ALLOCATED; `LedgerService.commitCharges` turns held credits into Charges equal to the locked Quotes, granted credits first.
5. `MatchedAllocator.commit` forms the Loans, and `LedgerService.postEarning` credits each lending Team the Day Rate times the person-days, once per Loan from `AllocationQueries.loansFor(line)`; the Request is Allocated, `record` logs it, and `Notifier` tells the Manager, the Persons, and the lending Managers.

Failure branches.

- The Candidate became ineligible before approval: `decide` refuses the approval with a `RuleViolation` whose code is `NOT_A_CANDIDATE`; the lending Manager can refuse instead (S3).
- A Candidate fails the re-check: `evaluateAll`, re-run at the start of `allocate`, answers UNSATISFIABLE(NOT_A_CANDIDATE), so a Required line goes to `toWaiting` before any commit and a replacement is asked (S3); an Optional line is Dropped and allocation continues. `MatchedAllocator.commit` repeats the check only as a guard that throws `ALLOCATION_CONFLICT` and rolls back.
- The Balance no longer covers the Required Charges (after a rollover or an Earning correction while Pending): `toWaiting` runs instead with reason `INSUFFICIENT_BALANCE`; the approval is kept.
- The actor may not decide this approval: `ForbiddenException`.
- A Required line is still waitlisted: the Request stays Pending until the line is filled (S6) or the window lapses (S4).

5.3.3 S3 Refusal and replacement (UC-6, UC-5)

1. The lending Manager refuses; `decideApproval` with `REFUSE` locks the types, and `ApprovalDriven.decide` records Refused.
2. If the refused line is Required, it is no longer satisfied, so `toWaiting` calls `Allocator.cancel` on each held line and `LedgerService.releaseCredits`, puts the Required lines on the Waitlist, and offers the freed capacity (`onCapacityFreed`). A refused Optional line releases nothing. Other approvals stay Outstanding.
3. `Notifier` asks the requesting Manager for a replacement; the Request is Waiting, or still Pending with its Holds if the refused line is Optional.
4. The Manager picks from the Shortlist; `selectCandidates` calls `ApprovalDriven.select`, which notifies that Candidate's Manager.
5. The window still runs, so `placeHolds` holds every satisfiable line again and the Request is Pending; `window_ends_at` is unchanged.

Failure branches.

- No Candidate is left on a Required line, or it named a Person: the Request is Refused and every line canceled (**Matched.Refuse.Replace**). On an Optional line, the line is Dropped.
- The refused line is Optional and Candidates remain: a replacement is asked, but the Request stays Pending with its Holds.
- The replacement comes after the window lapsed: no Hold is placed (ADR-0001); the approval goes to the Capacity Planner (S4).
- A line no longer fits when Holds are placed again: it joins the Waitlist and the Request is Pending with that line waiting.

5.3.4 S4 Objection Window lapses (UC-7)

1. Every 60 s `DueWorkJob.run` calls `lapsedWindows(now)`, which returns the Requests, Pending or Waiting, whose `window_ends_at` has passed with an approval still Outstanding and not yet escalated, and calls `onWindowLapse` for each, one transaction per Request.
2. `onWindowLapse` locks the types, reads the Request again, and does nothing unless it is still Pending or Waiting with an Outstanding approval not yet escalated (a last approval may have committed first, or an earlier run escalated it), so a repeated call is harmless.
3. For a Pending Request, `toWaiting` releases every Hold (rows HELD to EXPIRED) and its held credits, waitlists the Required lines, and offers the capacity; the Request is Waiting. A Waiting Request holds nothing, so this step is skipped.
4. `ApprovalDriven.escalate` sets `escalated` on every Outstanding approval, and `Notifier` tells the Manager and every Capacity Planner, found with `MemberService.withRole`.
5. The Capacity Planner decides each from `escalations()` through `decideApproval` (S2 or S3). No Hold is placed again: the Request allocates only when every Required line fits at one moment (**Lifecycle.Waiting.Reevaluate**).
6. The same run calls `expireWaitingLines(now)`, which takes lines whose window start has passed off the Waitlist.

Failure branches.

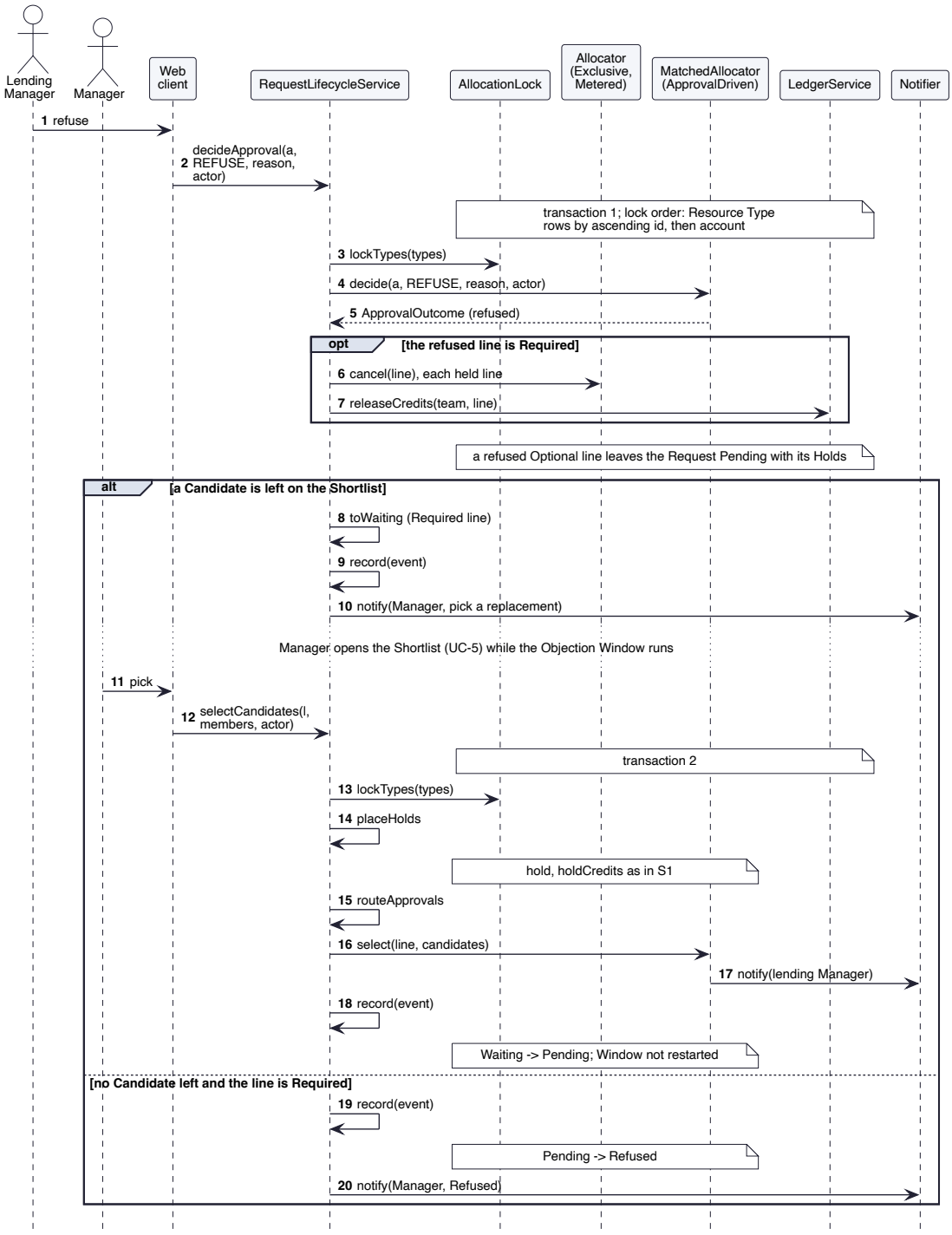


Figure 14: S3, refusal and replacement (UC-6, UC-5). The refusal of a Required line releases the Holds, which are placed again when a replacement is picked, without restarting the Objection Window; a refused Optional line keeps them; the `alt` box shows the Request Refused when no Candidate is left.

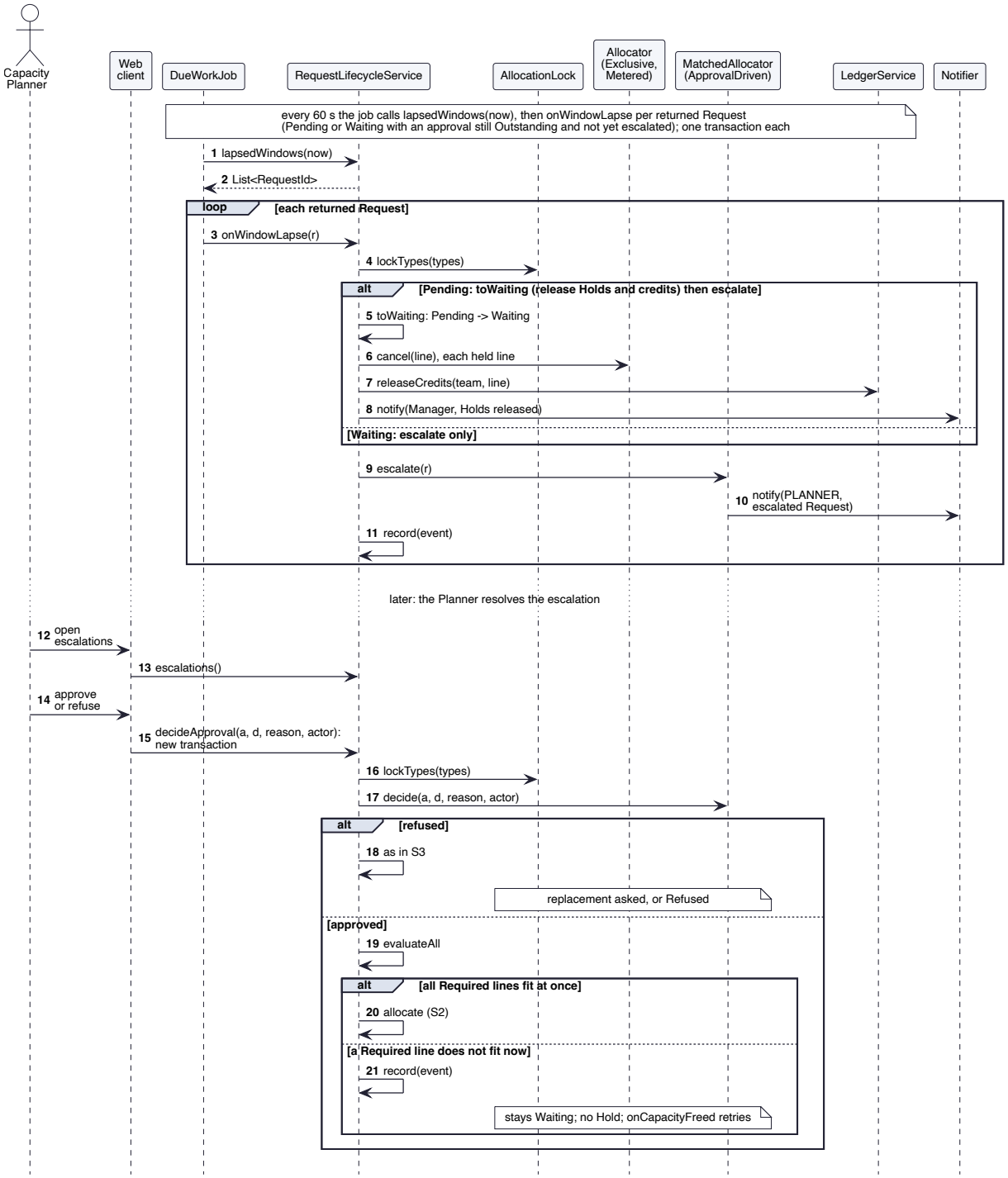


Figure 15: S4, the Objection Window lapses (UC-7). For a Pending Request the job releases every Hold and the Request moves to Waiting; for a Pending or Waiting one its Outstanding approvals escalate to the Capacity Planner, whose decision continues as S2 or S3.

- The job ran late (a restart): `lapsedWindows` returns every overdue Request, so the next run catches up, within the five minutes of **PE-4**.
- A last approval arrives in the same second: the type locks order the two, and whichever runs second sees the other's result.
- One Request's lapse fails: only its transaction rolls back, and the next run retries it because its approval is still Outstanding and not escalated.
- The job runs again after a lapse succeeded: every Outstanding approval is already escalated, so `lapsedWindows` no longer returns the Request and nobody is notified twice.

5.3.5 S5 Priority line Bumps a standard line (UC-8)

1. During `submit` (S1), `ExclusiveAllocator.evaluate` answers UNSATISFIABLE for a Priority line.
2. The lifecycle gets `Bumpable` from `AllocatorRegistry.capability` and calls `planBump`: the instance needing the fewest Bumps of standard lines, allocated or on Hold, preferring the latest-submitted, with none starting within the bump cutoff.
3. `applyBump` marks those occurrences BUMPED, flushes, and inserts the Priority occurrences as HELD; the exclusion constraint checks the insert.
4. For a Bumped line that was Allocated, `LedgerService.refund` posts the full Charge back to the Bumped Team; a line that was only on Hold had no Charge, so `LedgerService.releaseCredits` releases its held credits instead.
5. `Waitlistable.join` puts each Bumped Required line at the head of its Waitlist (a Bumped Optional line is Dropped instead); its Request moves to Waiting (other lines keep their allocations; a Pending Request releases its other Holds).
6. `Notifier` tells the Bumped Manager why; `record` logs the Bump on both lines; the submission continues as in S1.

Failure branches.

- A blocking line starts inside the bump cutoff, or only Priority lines block: no plan; the Required Priority line joins the Waitlist ahead of all standard lines.
- The Bumped line is Optional: it is Dropped and not waitlisted, with the full Refund (or, if it was only on Hold, its held credits released), and its Request keeps its state.
- The insert violates the constraint (the flush was skipped): the whole submission rolls back; nothing is Bumped or charged.

5.3.6 S6 Release and waitlist fill (UC-9, UC-10)

1. The Manager Releases a line or part of it; `releaseLine(l, part, ...)` locks its type, and `Allocator.release` frees that part while the rest keeps its instance and prices (**Release.Line**).
2. `Pricing.refund` applies the Refund Schedule step for the lead time to the part's start; `LedgerService.refund` posts it to the granted and earned parts in the Charge's proportions (**Release.Refund**).
3. `onCapacityFreed` asks `Waitlistable.waitingFor` for the waiting lines in order: Bumped, then Priority, then by submission time.
4. For each waiting Request in turn, the lifecycle locks its other types and checks every Required line. The first that fits is allocated (`allocate`), or, if it still needs approvals and its window runs, held (`placeHolds`) and Pending.
5. `Notifier` tells both Managers; `record` logs both histories.

Failure branches.

- The part was already consumed: `RuleViolation`; nothing changes.
- No waiting Request fits, or one cannot afford its locked Quote, or its window lapsed while it still needs approvals: it keeps its place and the next is tried; capacity left over shows as idle.
- Release of a Loan: `endLoanEarly` applies no Refund Schedule; `correctEarning` posts an Earning correction (negative, `corrects_id` set) so the lending Team earns the Day Rate for the days actually lent (**Release.Loan.Prorate**). It may take that Balance below zero, which blocks new Required Exclusive or Metered lines until restored. The Person is free from the day after the new last day.
- Locking a waiting Request's types deadlocks: PostgreSQL aborts one transaction, and the helper retries it once.

5.3.7 S7 Period rollover (UC-11)

1. `PeriodRolloverJob.run` fires daily at 00:05 and acts only when a Period boundary has passed; `catchUp` runs the same check at start-up.
2. It calls `LedgerService.rollover` with the ending and starting Periods (from `SettingsService.current()`); `rollover` locks every `account` row in ascending Team id.

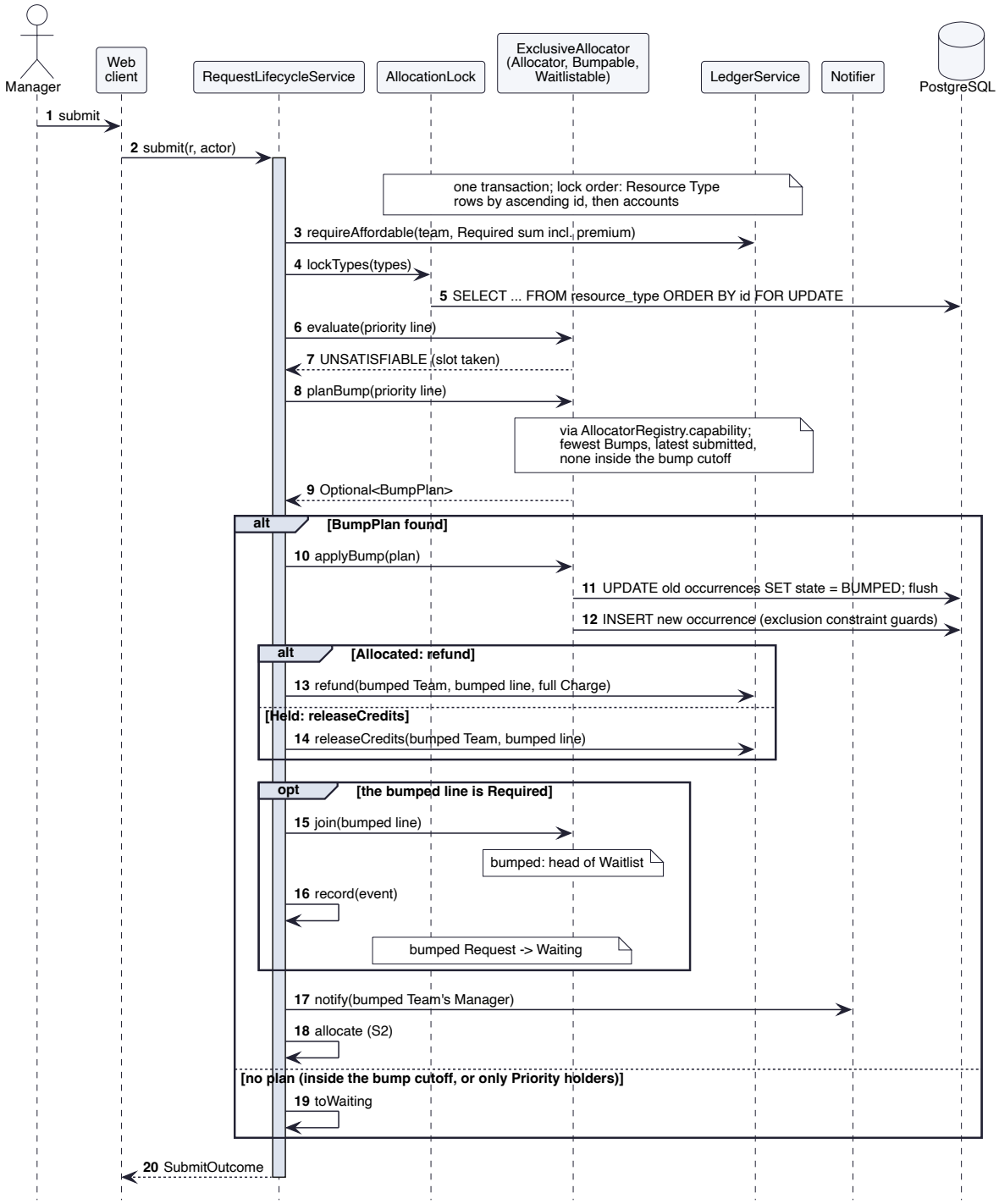


Figure 16: S5, a Priority line Bumps a standard line (UC-8). Shows the flush before the insert, the full Refund for an Allocated Bumped line or the release of held credits for a Held one, the Waitlist head for a Bumped Required line, and, in the `alt` box, the bump cutoff forbidding the Bump.

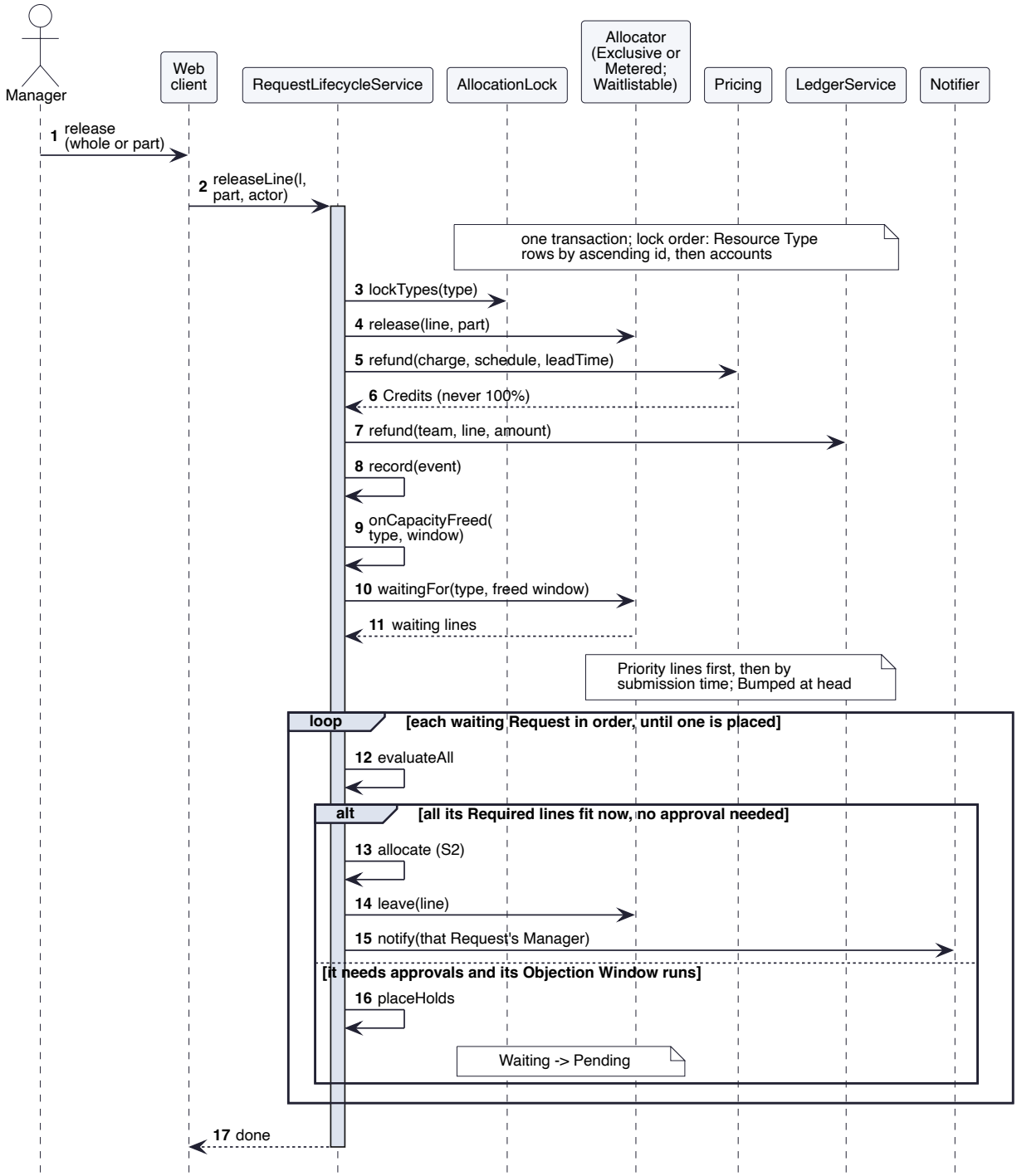


Figure 17: S6, Release and waitlist fill (UC-9, UC-10). The Refund follows the Refund Schedule, and the freed capacity is offered to the Waitlist in the same transaction; the `alt` box places Holds for a waiting Request that still needs approvals.

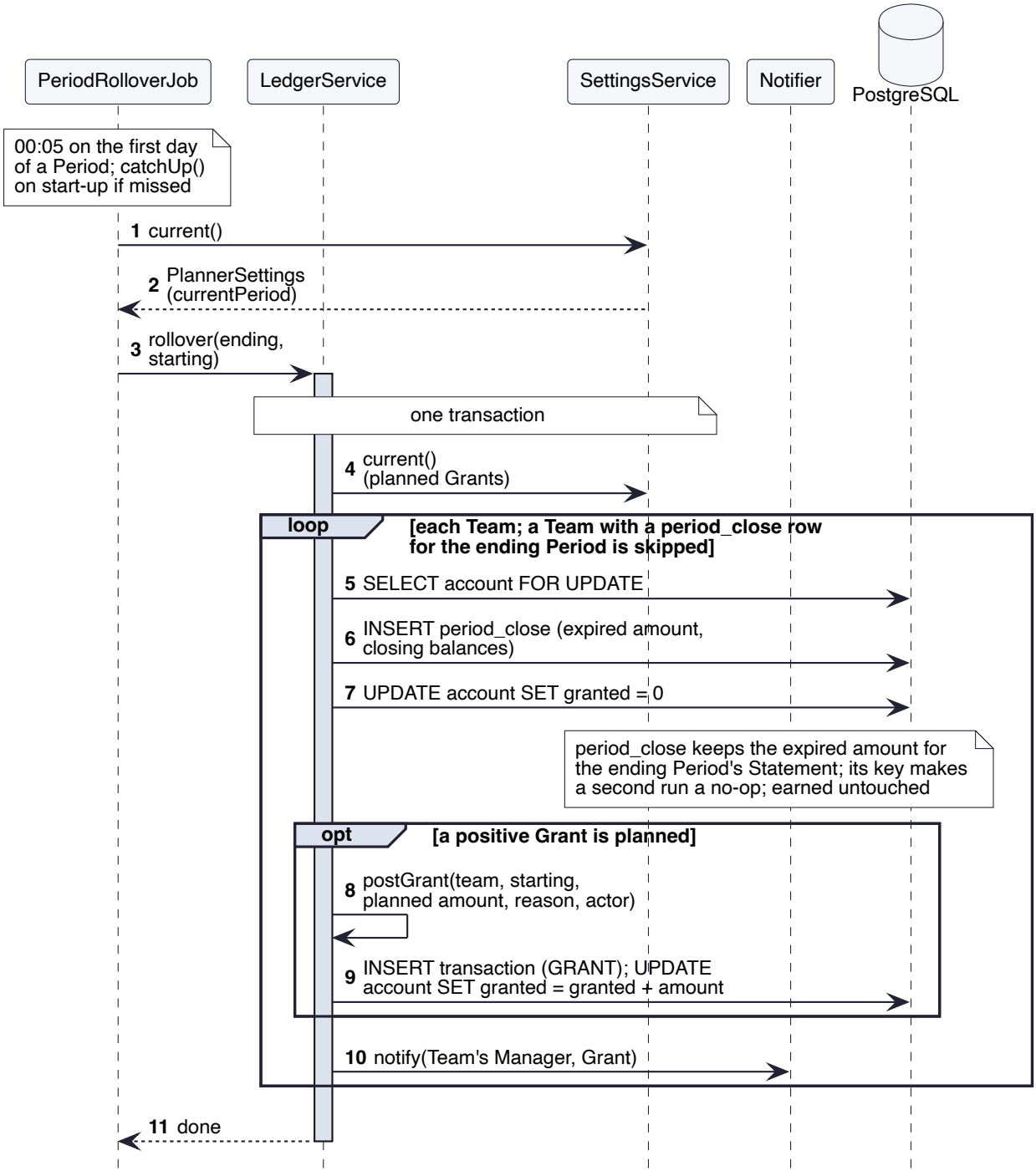


Figure 18: S7, Period rollover (UC-11). Granted credits expire, the new Grants post, and Managers are notified; the start-up catch-up runs the same steps after a missed boundary.

3. For each Team the remaining granted credits are the expired amount; `granted` is set to zero, and earned and held credits are untouched (**Ledger.Grant.Period**).
4. It posts the starting Period's planned Grant from `grant_plan` (`postGrant`, when the planned amount is positive) and `Notifier` tells each Manager.

No fifth Transaction kind records the expiry: `rollover` writes one `period_close` row per Team with the expired amount and closing balances; its key makes a second run a no-op.

Failure branches.

- The server was down at the boundary: `catchUp` rolls over into the current Period, covering one missed boundary; a longer outage is outside **QA-4**.
- The job fires twice: a Team with a `period_close` row for the ending Period is skipped, so nothing expires twice.
- A deadlock or lock timeout: the helper retries once; a second failure is logged and the next start-up's catch-up completes it.

6 Data Design

OpenBench keeps all its state in one PostgreSQL 16 database (**CO-2**, **SI-1**). This section lists the tables by owner, the constraints that carry business rules in the database itself, and how the schema is created and tested.

6.1 Schema by owner

Figures 19 and 20 show the tables grouped by owning module and Table 12 lists them. Conventions: primary keys are `bigint GENERATED ALWAYS AS IDENTITY` (join tables and per-Team or per-type settings use their natural keys); instants are `timestamptz`, mapped to `Instant`, and windows are half-open, so back-to-back windows do not overlap; Loan, availability, and Period days are `date`, inclusive at both ends; credits are `numeric(12,2)`; every row has `created_at` and `created_by` (null when a job writes it); states and kinds are `text` with a CHECK on the allowed values. A foreign key may cross a module boundary as a read-only reference (`request_line.type_id` references `resource_type`), but only the owning module writes a table: its repositories are package-private, and the ArchUnit test checks it (Section 2).

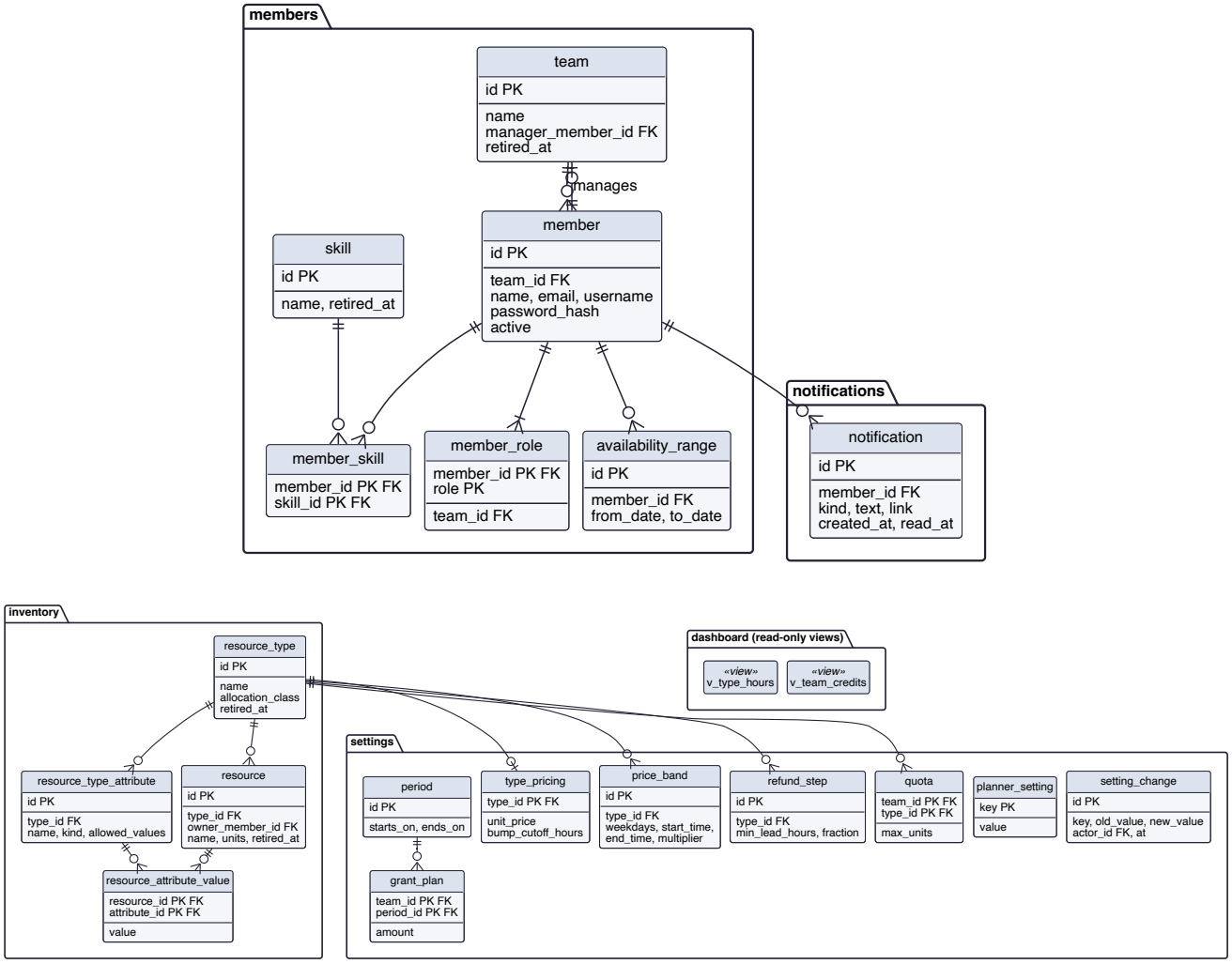


Figure 19: Database schema, foundation modules: above, the tables owned by **Members** and **Notifications**; below, those owned by **Inventory** and **Settings**, and the three read-only **Dashboard** views. Lines are foreign keys with crow's-foot cardinalities; only the owning module writes a table.

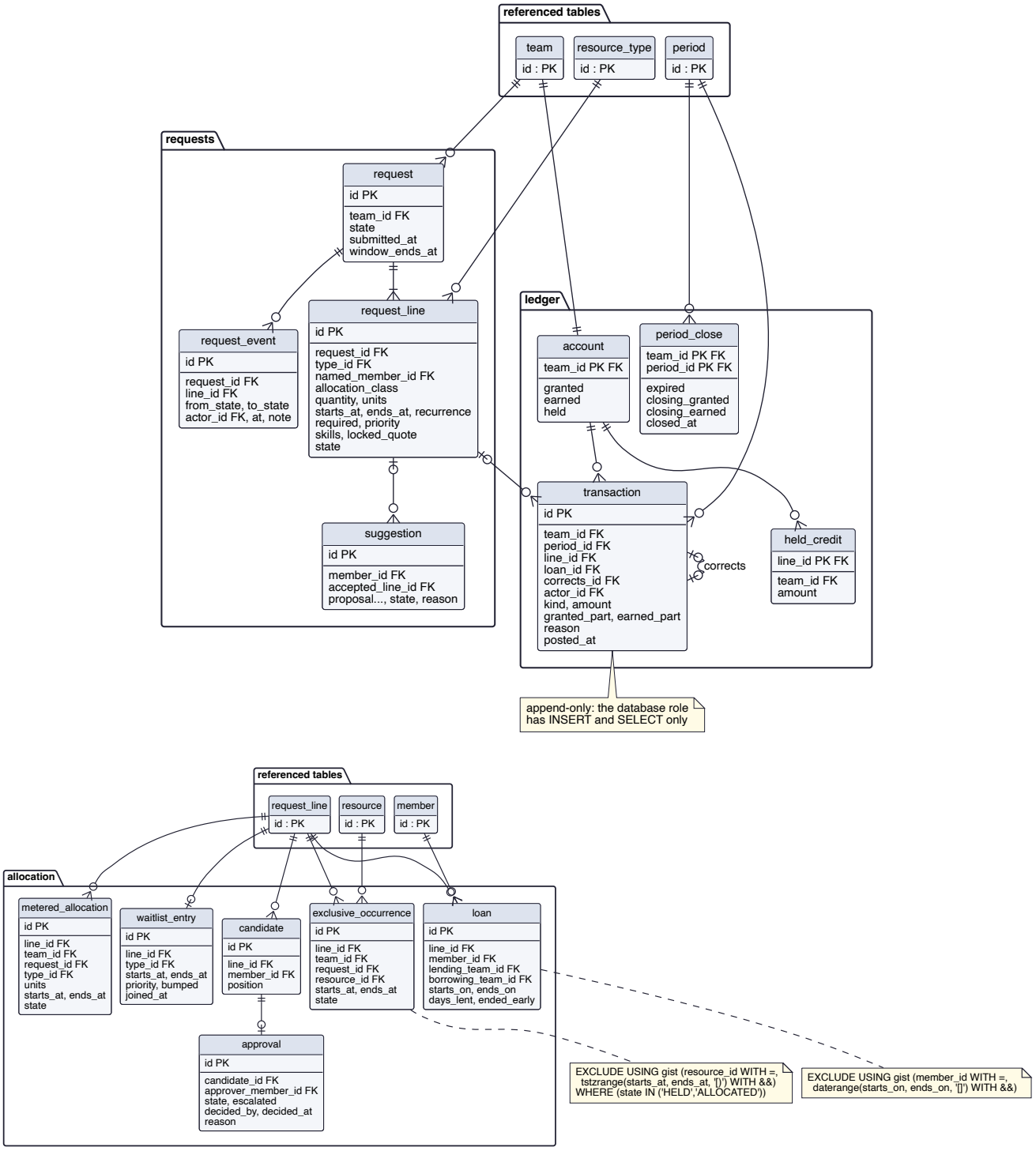


Figure 20: Database schema, core modules: above, the tables owned by **Requests** and **Ledger**; below, those owned by **Allocation**. Foundation tables they reference appear as stubs; a key that crosses a module boundary is a read-only reference. The notes quote the two exclusion constraints and the append-only rule.

Table 12: Tables by owning module (every table also has `created_at`, `created_by`).

Table	Purpose	Key columns	Notable constraints
Members			
`team`	Team	id, name, manager_member_id, retired_at	name unique
`member`	Member, login identity	id, team_id, name, email, username, password_hash, active	username, email unique; no other personal data (**SE-3**)
`member_role`	Granted roles	member_id, team_id, role	one MANAGER per Team
`skill`	Skill list	id, name, retired_at	name unique
`member_skill`	A Member's skills	member_id, skill_id	composite key
`availability_range`	Days a Member may be lent	id, member_id, from_date, to_date	from_date $\leq$ to_date
Inventory			
`resource_type`	Resource Type; the row `AllocationLock` locks	id, name, allocation_class, retired_at	name unique; Person seeded by the first migration
`resource_type_attribute`	Attribute definitions	id, type_id, name, kind, allowed_values	(type_id, name) unique
`resource`	Exclusive instance or Metered unit source	id, type_id, name, owner_member_id, units, retired_at	units $>$ 0; Pool Capacity = sum of unretired units
`resource_attribute_value`	Attribute values	resource_id, attribute_id, value	composite key
Settings			
`period`	Period	id, starts_on, ends_on	starts_on $\leq$ ends_on
`grant_plan`	Grant per Team and Period	team_id, period_id, amount	amount $\geq$ 0
`type_pricing`	Unit Price, bump cutoff	type_id, unit_price, bump_cutoff_hours	Exclusive and Metered types only
`price_band`	Price Band	id, type_id, weekdays, start_time, end_time, multiplier	start $<$ end; multiplier $>$ 0
`refund_step`	Refund Schedule step	id, type_id, min_lead_hours, fraction	0 $\leq$ fraction $<$ 1
`quota`	Quota per Team and Metered type	team_id, type_id, max_units	max_units $>$ 0
`planner_setting`	Single-valued settings	key, value	day_rate, priority_premium, objection_window_working_days, period_months
`setting_change`	Setting change log	id, key, old_value, new_value, actor_id, at	insert-only grant
Requests			
`request`	Request	id, team_id, state, submitted_at, window_ends_at	window_ends_at set once
`request_line`	Request Line, locked Quote	id, request_id, type_id, allocation_class, quantity, units, starts_at, ends_at, recurrence, required, priority, named_member_id, skills, locked_quote, state	starts_at $<$ ends_at; quantity $>$ 0; priority only if Exclusive; locked_quote never changes
`suggestion`	Suggestion	id, member_id, proposal columns, state, accepted_line_id, reason	never deleted
`request_event`	State history	id, request_id, line_id, from_state, to_state, actor_id, at, note	insert-only grant
Allocation			
`exclusive_occurrence`	One occurrence on one instance	id, line_id, team_id, request_id, resource_id, starts_at, ends_at, state	exclusion constraint on live rows
`metered_allocation`	Units over a window	id, line_id, team_id, request_id, type_id, units, starts_at, ends_at, state	units $>$ 0; partial GiST index
`waitlist_entry`	A waiting Required line	id, line_id, type_id, starts_at, ends_at, priority, bumped, joined_at	line_id unique; deleted on leaving
`candidate`	Shortlist entry	id, line_id, member_id, position	(line_id, member_id) unique
`approval`	Approval of a Candidate	id, candidate_id, approver_member_id, state, decided_by, decided_at, reason, escalated	candidate_id unique
`loan`	Loan	id, line_id, member_id, lending_team_id, borrowing_team_id, starts_on, ends_on, days_lent, ended_early	exclusion constraint per Person; Teams differ
Ledger			
`account`	Account	team_id, granted, earned, held	granted, held $\geq$ 0

continues on next page

Table 12 (continued)

Table	Purpose	Key columns	Notable constraints
`transaction`	Grant, Charge, Earning, Refund	id, team_id, kind, amount, granted_part, earned_part, period_id, line_id, loan_id, corrects_id, reason, actor_id, posted_at	insert-only grant; amount = granted_part + earned_part
`held_credit`	Held credits per line on Hold	line_id, team_id, amount	one row per Held line; deleted when the Hold commits or releases
`period_close`	Expired and closing credits	team_id, period_id, expired, closing_granted, closing_earned, closed_at	key (team_id, period_id): one row per Team and closed Period; the rollover idempotency key
Notifications			
`notification`	In-app notification	id, member_id, kind, text, link, created_at, read_at	index on unread rows
Dashboard (views; owns no table)			
`v_type_hours`	Allocated and available hours per type and day	over `exclusive_occurrence`, `metered_allocation`, `resource`	read-only
`v_team_credits`	Credits per Team and Period by kind, expired amount	over `transaction`, `loan`, `period`, `period_close`, plus `account`	read-only
`v_waiting`	Waiting lines per type	over `waitlist_entry`	read-only

6.2 Constraints that carry business rules

A rule between two rows, or between one row and a fixed bound, is stated in the schema, where no code path can skip it. A rule over a sum of rows is a query run under the type lock of Section 5.

No overlapping allocation, no double Loan. The **Allocation** migration states **Exclusive.Conflict** and the one-Loan rule of **RC-1**:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;   -- GiST support for "bigint WITH ="

CREATE TABLE exclusive_occurrence (
  id          bigint      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  line_id     bigint      NOT NULL REFERENCES request_line,
  team_id     bigint      NOT NULL REFERENCES team,
  request_id  bigint      NOT NULL REFERENCES request,
  resource_id bigint      NOT NULL REFERENCES resource,
  starts_at   timestamptz NOT NULL,
  ends_at     timestamptz NOT NULL,
  state       text        NOT NULL
              CHECK (state IN ('HELD', 'ALLOCATED', 'RELEASED', 'BUMPED', 'EXPIRED')),
  created_at  timestamptz NOT NULL DEFAULT now(),
  created_by  bigint      REFERENCES member,
  CHECK (starts_at < ends_at),
  CONSTRAINT exclusive_occurrence_no_overlap EXCLUDE USING gist (
    resource_id WITH =,
    tstzrange(starts_at, ends_at, '[)') WITH &&
  ) WHERE (state IN ('HELD', 'ALLOCATED'))
);

ALTER TABLE loan ADD CONSTRAINT loan_one_at_a_time EXCLUDE USING gist (
  member_id WITH =,
  daterange(starts_on, ends_on, '[]') WITH &&
);
```

A row with the same Resource as a live row and an overlapping window is rejected. `btree_gist`, a trusted contrib extension present in the official image, supplies GiST support for the scalar `resource_id WITH =`. The second constraint keeps a Person in at most one Loan at a time on every code path, with no exception for `ended_early`, which only marks that an Earning correction was posted. Only HELD and ALLOCATED rows take part in the first constraint; RELEASED, BUMPED, and EXPIRED rows stay as history and block nothing. The half-open range is built from two `timestamptz` columns, so 09:00 to 12:00 and 12:00 to 13:00 do not conflict, and Java sees only two `Instant` fields. The `loan` constraint uses inclusive `daterange` bounds because Loans are whole days: a Loan ending on a Friday and one starting that Friday overlap. `endLoanEarly` sets `ends_on` to the new last day and `ended_early`; the shortened Loan stays in the constraint for the days it lent.

The append-only Ledger. Flyway connects as the schema owner; the application connects as `openbench_app`, which owns nothing and gets these grants and checks in the last migration:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO openbench_app;
REVOKE UPDATE, DELETE ON transaction, request_event, setting_change FROM openbench_app;

ALTER TABLE transaction
  ADD CHECK (kind IN ('GRANT', 'CHARGE', 'EARNING', 'REFUND')),
  ADD CHECK (amount = granted_part + earned_part),
  ADD CHECK (amount >= 0 OR (kind = 'EARNING' AND corrects_id IS NOT NULL));
```

The application can insert and read Transactions but never change or delete one, so **Ledger.Transaction.Kinds** and **QA-3** hold even against a bug. A correction is a new row whose `corrects_id` names the one it corrects; the only one in Release 1, the Earning correction of an early Loan end, is the only negative amount and the only Transaction that may take a Balance below zero. `granted_part` and `earned_part` record where a Charge drew from, so a Refund returns credits in the same proportions. The `account` row is the running total, updated under its row lock. `request_event` and `setting_change` get the same insert-only grant.

Check constraints. Positive amounts: `units > 0` (`resource`, `metered_allocation`, Metered lines), `quantity > 0`, `max_units > 0`, `multiplier > 0`, and `granted, held >= 0`. Ordered windows: `starts_at < ends_at` on every table with a window, and the same for the date and time pairs. A Refund Schedule never reaches 100% (**Release.Refund**): `CHECK (fraction >= 0 AND fraction < 1)` on `refund_step`. Priority only on Exclusive lines: `CHECK (NOT priority OR allocation_class = 'EXCLUSIVE')`. A Loan crosses Teams: `lending_team_id <> borrowing_team_id`. Exactly one Manager per Team (**Admin.Member.Create**): "at most one" is the index below, and "at least one" is enforced by `revokeRole`, which refuses to remove the last Manager, and `grantRole`, which deletes the previous Manager's row and flushes before inserting the new one (the flush-order rule of Section 5). `member_role.team_id` follows `member.team_id` through a composite foreign key with `ON UPDATE CASCADE`, so moving a Member moves their roles.

```
CREATE UNIQUE INDEX member_role_one_manager ON member_role (team_id) WHERE role = 'MANAGER';
```

The Metered hourly sum. **Metered.Pool.Check** needs, in every hour of the window, the units of live rows on the type plus the requested units within Pool Capacity, and the Team's share within its Quota. `MeteredAllocator` runs this native query after `lockTypes`, in the same READ COMMITTED transaction, and reads Pool Capacity after the lock too:

```
-- :starts_at, :ends_at on the hour; one row per hour of the window, in order.
SELECT h.hour_start,
       coalesce(sum(m.units), 0)                                      AS pool_used,
       coalesce(sum(m.units) FILTER (WHERE m.team_id = :team_id), 0) AS team_used
FROM generate_series(:starts_at, :ends_at - interval '1 hour', interval '1 hour') AS h(hour_start)
LEFT JOIN metered_allocation m
       ON m.type_id = :type_id
      AND m.state IN ('HELD', 'ALLOCATED')
      AND tstzrange(m.starts_at, m.ends_at, '[)')
          && tstzrange(h.hour_start, h.hour_start + interval '1 hour', '[)')
GROUP BY h.hour_start
ORDER BY h.hour_start;

CREATE INDEX metered_allocation_live ON metered_allocation
  USING gist (type_id, tstzrange(starts_at, ends_at, '[)'))
  WHERE state IN ('HELD', 'ALLOCATED');
```

`generate_series` yields the hours, and the join sums the live units overlapping each hour for the pool and, through `team_id` on the row itself, for the Team, so the query reads only **Allocation**'s own table; the partial GiST index serves the overlap test. The pure class `HourlyLoad` (Section 4.5) then walks the hours in order and returns the earliest one in which `pool_used` plus the requested units exceed Pool Capacity (`POOL_FULL`) or, when the Team has a Quota on the type, `team_used` plus them exceed it (`QUOTA_EXCEEDED`); no such hour means the line fits. With zero requested units and the reduced capacity, the same query and `HourlyLoad` answer `ReductionGuard.earliestHourExceeding` (**Inventory.Resource.Add**). It is native SQL because JPQL has neither `generate_series` nor range operators.

State history. `record` writes one `request_event` per state change of a Request or line, in the same transaction: from and to states, actor (null for a job, named in `note`), time, and a note such as a refusal reason, the Bump partner, or the Refund. A Bump writes a row on each line, each naming the other. `RequestLifecycleService.detail` reads the timeline from these rows (**Lifecycle.States**, **QA-3**).

6.3 Migrations and the test database

Flyway at start-up. The schema is a series of versioned SQL files under `src/main/resources/db/migration` (`V<n>__<name>.sql`), one per module in layer order, then the views and grants; a released file is never edited. Spring Boot runs Flyway at start-up, before Hibernate, because the `EntityManagerFactory` depends on database initialization. `spring.jpa.hibernate.ddl-auto=validate` then checks every entity against the migrated tables and stops start-up on a mismatch; Hibernate never creates a table, and the migrations alone own the extension, the exclusion constraints, the partial indexes, and the grants.

Why H2 was dropped (RC-3). The SRS first required tests on H2 in memory with the same migrations. H2 cannot run them: in PostgreSQL mode it rejects `CREATE EXTENSION`, has no range types or `EXCLUDE`, and cannot parse the hourly sum's interval arithmetic; nothing in its documentation says its locking behaves like PostgreSQL's. Keeping H2 would mean forking the migrations or dropping the constraints, and the most important test, two allocations racing for one type, could not run. SRS 0.3 therefore changes **CO-2** and **SI-1** to PostgreSQL 16 in a container started by the test suite.

Testcontainers. Integration tests extend one base class:

```
@SpringBootTest @Testcontainers
abstract class PostgresTest {
  @Container @ServiceConnection
  static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16-alpine");
}
```

`@ServiceConnection` points the data source at the container, so Flyway runs the production migrations, extension and constraints included. The concurrency test runs two allocations on two threads against one pool and asserts that exactly one succeeds; another connects as `openbench_app` (created by the container's init script) and checks that updating a Transaction is refused (Section 8). Developers need Docker, which building the application image already requires; unit tests of `Pricing` and the state rules need no database.

Retention. Nothing is purged in Release 1 (**QA-7**): no code path deletes a Request, a line, a Suggestion, a `request_event`, or a Transaction, and Teams, skills, Resource Types, and Resources are retired, not deleted. The application deletes only working state: Waitlist entries, whose history is in `request_event`, `held_credit` rows, which exist only while a line is on Hold, and revoked role rows. At 10,000 Requests a year (**PE-1**), two years of rows are small for one database.

7 REST API

The browser client talks to the server only through one REST/JSON API, and every use case of the SRS can be carried out through it under the same role checks (**SI-2**, **CO-3**). springdoc generates the OpenAPI document from the controllers, so the running system is the authoritative reference; Table 14 summarizes it for design review.

7.1 Conventions

Paths. Every endpoint lives under `/api`; any other path returns the single-page application. Each module serves its own nouns, as the group headings of Table 14 show. Where a path needs the data of a higher module, that module's controller serves it, so no controller breaks the dependency rule of Section 2: the holder views under `/inventory` and a Member's Loans under `/members` are served by **Allocation**. In member paths `me` stands for the caller's own id. There is no version segment: client and server ship in one jar and cannot disagree; a breaking change for another client would add `/api/v2`.

Methods and transitions. GET reads and never changes state; POST creates an object (201 with `Location`) or performs one transition; PUT replaces one value; DELETE removes a Draft line or a role. A transition is a POST to a sub-resource named by the verb of the state machine, such as POST `/api/requests/{id}/submit`, `/withdraw`, and `/reopen`, POST `/api/requests/{id}/lines/{lineId}/release`, and POST `/api/approvals/{id}/decision`; it answers 200 with the object's new state. One URL per transition gives each one its own operation, role check, and audit row, which a generic update of a `state` field would blur. Repeating a transition is harmless: a second `submit` finds the Request no longer in Draft and gets 409.

One path through the server. A controller method binds and validates the body, builds the caller's `Actor` from the session, calls one application-service operation (the free-Resource search alone calls two read-only queries), and maps the result to JSON. It holds no business rule and opens no transaction. `@PreAuthorize` sits on the application-service methods, not on the controllers, so the screens and any other API client pass the same checks (**CO-3**, **SE-1**).

Representation. JSON in UTF-8 with camelCase names; enumerations as upper-case strings (`PENDING`); ids as JSON integers (the typed wrappers such as `RequestId` serialize as their number); credits as JSON numbers; instants as ISO-8601 date-times with the offset of the company time zone (`2026-10-12T09:00:00-04:00`), accepted with any offset and normalized; Loan days and Period bounds as ISO dates. A time window is `{"start", "end"}`, half-open as `TimeWindow`; a day range is `{"from", "to"}`, inclusive as `DateRange`. Lists marked *paged* take `page` (from 0) and `size` (default 50, at most 200) and return `{"content", "page", "size", "totalElements"}`; short lists return a plain array.

Errors. Every error is an RFC 9457 problem detail (`application/problem+json`, Spring's `ProblemDetail`) with `type`, `title`, `status`, `detail` (one sentence the client shows as is), `instance` (the request path), and the extension field `code`, a stable identifier the client may branch on; a 400 adds `errors`, one entry per invalid field. Table 13 gives the status per error class; Section 8 says where each is raised.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{ "type": "/problems/insufficient-balance", "title": "Available Balance too low", "status": 422,
  "detail": "The Required lines need 1840 credits; the Available Balance is 1200.",
  "code": "INSUFFICIENT_BALANCE", "instance": "/api/requests/42/submit" }
```

Table 13: HTTP status per error class.

Status	Error class and typical cause	Example `code`
400 Bad Request	Validation at the controller: malformed JSON, a missing field, a value out of range	`VALIDATION_FAILED`
401 Unauthorized	Unauthenticated: no session, an expired session, wrong credentials at login	`UNAUTHENTICATED`, `BAD_CREDENTIALS`
403 Forbidden	A missing role (`@PreAuthorize`), a missing or wrong CSRF token, or `ForbiddenException`	`FORBIDDEN`, `CSRF_REJECTED`
404 Not Found	`NotFoundException`, including any object outside the caller's scope	`NOT_FOUND`
409 Conflict	`RuleViolation` on a state conflict: the object's state does not allow the action; or an exclusion-constraint violation (SQLSTATE 23P01), logged as a bug	`WRONG_STATE`, `ALREADY_DECIDED`, `ALLOCATION_CONFLICT`

Status	Error class and typical cause	Example code
422 Unprocessable Content	`RuleViolation` on a business rule	`INSUFFICIENT_BALANCE`, `NOT_A_CANDIDATE`
503 Service Unavailable	`ConcurrencyRetryException`: a lock wait or deadlock failed twice; `Retry-After: 1`	`CONCURRENCY_RETRY`
500 Internal Server Error	Anything else; logged with its stack trace, never returned in the body	`INTERNAL`

API documentation. springdoc serves the OpenAPI 3 document at `/api/docs` and a browsable page at `/api/docs/ui` to signed-in Members. The build writes the document into the front-end source tree and `openapi-typescript` turns it into the client's types (Section 4.10), so a changed endpoint breaks the front-end build instead of a screen.

7.2 Authentication and Sessions

Authentication is Spring Security form login with a server-side HTTP session (**CO-6**, **SE-2**).

- **Login and logout.** POST `/api/login` takes the form fields `username` and `password` and answers with JSON instead of a redirect: 200 with the body of GET `/api/me`, or a 401 that does not say whether the username exists. The `UserDetailsService` that **Members** provides (Section 4.1) loads the Member and their roles; a deactivated Member cannot sign in (**Admin.Member.Create**). The session id changes at login against session fixation. POST `/api/logout` invalidates the session.
- **Identity.** GET `/api/me` returns the Member's id, name, Team, and roles. The client builds its menu from the roles; the server never relies on that menu.
- **Session cookie.** `HttpOnly; Secure; SameSite=Lax`. Sessions live in the memory of the one process and expire after 12 hours without a request. A restart signs everyone out, which is acceptable because restarts happen outside business hours (**QA-4**). `deactivate`, `grantRole`, and `revokeRole` expire the Member's sessions through Spring Security's session registry, so changed roles apply from the next sign-in.
- **CSRF.** Cookie to header: the token travels in the cookie `XSRF-TOKEN`, which script may read, and the client copies it into the header `X-XSRF-TOKEN` on every POST, PUT, and DELETE, the login included. A missing or wrong token gets 403.
- **Passwords and transport.** bcrypt through Spring's delegating password encoder, which stores the algorithm with the hash so it can be upgraded without a reset; no password is logged or returned. HTTPS only (**CI-1**): TLS ends at nginx or the embedded Tomcat, plain HTTP only redirects, responses carry Strict-Transport-Security, and behind nginx the forwarded headers are trusted so redirects and the `Secure` cookie use the external scheme.
- **Sign-in log.** Every authentication attempt, successful or not, is logged with username, remote address, and outcome (**SE-2**).

Why a session cookie and not a JSON Web Token. The client comes from the same origin as the API, so the browser sends the cookie by itself: no credential is stored in script and no cross-origin setup is needed. There is one process, so sessions need no shared store, and a self-contained token's benefit, validation across many servers, buys nothing. Revocation is simpler: an invalidated session is refused on the next request, while a token stays valid until it expires unless a deny list is kept, which is a session store again. An `HttpOnly` cookie cannot be read by script, so a cross-site scripting bug cannot carry it off. The costs are the CSRF protection above and, for a second instance, sticky sessions or a shared session store (Section 9).

7.3 Endpoints

Table 14 lists every endpoint by the module that serves it. Each group heading names the interface of its operations and the use cases it serves, so every use case maps to endpoints (**SI-2**). Paths are relative to `/api`. Roles are granted authorities, and every signed-in Member holds `MEMBER`; a scope in the notes (own Team, own Resource, self) is enforced inside the query (Section 8). Operations only other modules call, such as `holdCredits`, `isAvailable`, `onWindowLapse`, and `rollover`, have no endpoint.

Table 14: Endpoints by module. Paths are relative to `/api`.

Method	Path	Roles	Operation	Notes
Session (Spring Security; every use case)				
POST	`/login`	anonymous	form login	200 or 401
POST	`/logout`	`MEMBER`	logout	204

continued on the next page

Method	Path	Roles	Operation	Notes
GET	/me	MEMBER	MemberService.profile	id, name, Team, roles
Members and administration (**Members**: MemberService unless named)				
GET	/teams	MEMBER	teams	active Teams
GET	/teams/{id}/members	MANAGER, PLANNER, ADMIN	roster	Manager: own Team; paged
POST	/admin/members	ADMIN	createMember	Team required
POST	/admin/members/{id}/deactivate	ADMIN	deactivate	ends the Member's sessions
POST	/admin/members/{id}/move	ADMIN	moveToTeam	body teamId
POST	/admin/roles	ADMIN	grantRole	a new Manager replaces the old
DELETE	/admin/roles/{memberId}/{role}	ADMIN	revokeRole	422 if a Team loses its Manager
POST	/admin/teams	ADMIN	createTeam	
PUT	/admin/teams/{id}	ADMIN	renameTeam	body name
POST	/admin/teams/{id}/retire	ADMIN	retireTeam	422 while Members remain
GET	/skills	MEMBER	skills	the skill list
POST	/admin/skills	ADMIN	addSkill	
POST	/admin/skills/{id}/retire	ADMIN	retireSkill	
GET	/members/{id}/profile	MEMBER	profile	self, own Manager, PLANNER, ADMIN
PUT	/members/me/skills	MEMBER	updateSkills	self
PUT	/members/{id}/availability	MEMBER	updateAvailability	self, or the Member's Manager
GET	/members/{id}/loans	MEMBER	AllocationQueries.loansOf	by **Allocation**; scope as profile
Inventory (**Inventory**: ResourceTypeService, ResourceService, InventoryQueries; UC-1, UC-2)				
GET	/inventory/types	MEMBER	types	flag includeRetired
GET	/inventory/types/{id}	MEMBER	type, poolCapacity	with Pool Capacity
POST	/inventory/types	ADMIN	define	no code change
PUT	/inventory/types/{id}	ADMIN	change	class fixed once a Resource exists
POST	/inventory/types/{id}/retire	ADMIN	ResourceTypeService.retire	refused for Person
GET	/inventory/resources	MEMBER	search	by type and attributes; paged
GET	/inventory/resources/free	MEMBER	search, then AllocationQueries.holders	by **Allocation**; type, from, to
POST	/inventory/resources	RESOURCE_OWNER	add	caller becomes the owner
PUT	/inventory/resources/{id}	RESOURCE_OWNER	edit	own Resource
PUT	/inventory/resources/{id}/units	RESOURCE_OWNER	setUnits	422 names the earliest failing hour
POST	/inventory/resources/{id}/retire	RESOURCE_OWNER	ResourceService.retire	same guard with units 0
GET	/inventory/resources/{id}/holders	MEMBER	AllocationQueries.holders	by **Allocation**; Team and Request per scope
Settings (**Settings**: SettingsService; UC-11)				
GET	/settings	PLANNER	current	every setting, typed
PUT	/settings/{key}	PLANNER	update	logged; keys below
GET	/settings/periods	MEMBER	periods	for the Period selector
Requests (**Requests**: RequestService for Drafts, RequestLifecycleService for transitions; UC-4, UC-5, UC-8 to UC-10)				
GET	/requests	MANAGER, PLANNER	list	own Team, all for PLANNER; paged
POST	/requests	MANAGER	create	for the caller's Team
GET	/requests/{id}	MANAGER, PLANNER	detail	lines, Holds, Waitlist positions, history
GET	/requests/{id}/draft	MANAGER	view	Quotes, sums, Available Balance
POST	/requests/{id}/lines	MANAGER	addLine	window in the future
PUT	/requests/{id}/lines/{lineId}	MANAGER	editLine	Draft only
DELETE	/requests/{id}/lines/{lineId}	MANAGER	removeLine	Draft only
POST	/requests/{id}/lines/{lineId}/split	MANAGER	splitLine	new line is Optional
PUT	/requests/{id}/lines/{lineId}/required	MANAGER	setRequired	Required or Optional
PUT	/requests/{id}/lines/{lineId}/priority	MANAGER	setPriority	Exclusive lines only
POST	/requests/{id}/submit	MANAGER	submit	locks Quotes; may Bump (UC-8)
POST	/requests/{id}/withdraw	MANAGER	withdraw	from Draft, Pending, Waiting
POST	/requests/{id}/reopen	MANAGER	reopenForEdit	Waiting to Draft
GET	/requests/{id}/lines/{lineId}/shortlist	MANAGER, PLANNER	shortlist via ApprovalDriven.buildShortlist	Shortlist order (**PE-3**)

continued on the next page

Method	Path	Roles	Operation	Notes
POST	/requests/{id}/lines/{lineId}/candidates	MANAGER	selectCandidates	exactly the line's quantity
POST	/requests/{id}/lines/{lineId}/release	MANAGER	releaseLine	whole, or window, occurrences, units
POST	/loans/{id}/end	MANAGER	endLoanEarly	borrowing Manager; newEnd
Suggestions (**Requests**: SuggestionService; UC-3)				
POST	/suggestions	MEMBER	create	to the caller's own Manager
GET	/suggestions/mine	MEMBER	mine	paged
GET	/suggestions/team	MANAGER	forManager	own Team, Open first; paged
POST	/suggestions/{id}/accept	MANAGER	acceptIntoNewLine, acceptIntoLine	requestId or lineId
POST	/suggestions/{id}/decline	MANAGER	decline	optional reason
Approvals and escalations (**Requests**: RequestLifecycleService unless named; UC-6, UC-7)				
GET	/approvals	MANAGER, PLANNER	AllocationQueries.approvalsAwaiting	addressed to the caller, with time left
POST	/approvals/{id}/decision	MANAGER, PLANNER	decideApproval	PLANNER once escalated
GET	/escalations	PLANNER	escalations	with outstanding approvals
Ledger (**Ledger**: LedgerService; UC-11, UC-12)				
GET	/ledger/account	MANAGER, PLANNER	account	own Team; PLANNER any
GET	/ledger/statements/{teamId}/{periodId}	MANAGER, PLANNER	statement	own Team; PLANNER any
POST	/ledger/grants	PLANNER	postGrant	mid-Period, with a reason
Dashboard (**Dashboard**: DashboardQueries; UC-12)				
GET	/dashboard/utilization	MANAGER, RESOURCE_OWNER, PLANNER	utilization	Resource Owner: own types
GET	/dashboard/idle	as above	idleNextDays	days, default 7
GET	/dashboard/waiting	as above	waitingCount	lines waiting per type
GET	/dashboard/credits	MANAGER, PLANNER	credits	own Team; PLANNER any
GET	/dashboard/person-days	MANAGER, PLANNER	personDays	lent and borrowed; same scope
Notifications (**Notifications**: NotificationQueries; every use case)				
GET	/notifications	MEMBER	unread	own, newest first; paged
POST	/notifications/{id}/read	MEMBER	markRead	own

The key of PUT /api/settings/{key} names one setting and the body carries its new value: day_rate, priority_premium, objection_window_working_days, period_months, grant.{teamId}.{periodId}, quota.{teamId}.{typeId}, unit_price.{typeId}, bump_cutoff.{typeId}, price_bands.{typeId}, or refund_steps.{typeId}. Each call is one SettingChange and one change-log row, and affects only Quotes not yet locked and events after the change (**Planner.Settings**). UC-10, filling a freed slot from the Waitlist, has no endpoint of its own: it runs inside releaseLine, withdraw, decideApproval, and the jobs, wherever capacity frees.

8 Crosscutting Concerns

Each concern below is solved once, in the `security`, `common`, or `config` package or in one module.

8.1 Authorization and Data Scoping

Every request passes two checks: a role check and a scope (**SE-1**, **SE-4**).

Roles. Each `member_role` row becomes a Spring Security granted authority, `MANAGER`, `RESOURCE_OWNER`, `PLANNER`, or `ADMIN`, and every signed-in Member also holds `MEMBER`. Each application-service method a controller calls carries `@PreAuthorize` with the roles of Table 14, so the check runs on the server in the same place whether the call comes from a screen or a script (**CO-6**, **CO-3**); an ArchUnit rule fails the build if a controller reaches a service method without one. Operations that only other modules or the jobs call carry no role check, because no controller reaches them.

Scope. Which rows a role may see is decided inside the query, by the caller's Team, own id, or owned Resources, never by loading rows and filtering them afterward. The controller passes the caller as an `Actor` (Member id, Team id, roles), and repositories take its fields as parameters. Read operations without an `Actor` parameter take it from a request-scoped `CurrentActor` bean filled by the controller (the system actor in jobs). A module reading another module's data for its own rule (Allocation's Candidate check, Inventory's new-owner check) calls an unscoped internal read such as `MemberService.summary(MemberId)`. A row outside the scope is not found, so the caller gets 404 and learns nothing about it. A Member with several roles sees the union of their scopes: the query joins the predicates of their roles with OR. Table 15 gives the scopes; a Manager never sees another Team's Statement, Account, or Requests.

```
// RequestRepository: the scope is part of the query
@Query("select r from Request r where r.id = :id and (r.teamId = :team or :planner = true)")
Optional<Request> findVisible(RequestId id, TeamId team, boolean planner);
// RequestLifecycleService.detail
Request r = requests.findVisible(id, actor.teamId(), actor.has(PLANNER)).orElseThrow(NotFoundException::new);
```

Table 15: What each role sees. Roles add up: a Member sees the union of the rows for the roles they hold.

Role	Sees	Scope in the query
Member (everyone)	Own profile, Loans, Suggestions, and notifications; the skill list, Teams, Resource Types, Resources, and whether each Resource is free, on Hold, or allocated in a window	`member_id = caller`; Inventory open, holding Team and Request hidden
Manager	Own Team's roster, profiles, and Loans; the Team's Requests with Shortlists, Holds, Waitlist positions, and history; Suggestions and approvals addressed to them; the Team's Account, Statements, credits, and person-days; company-wide utilization, idle capacity, and waiting counts	`team_id = caller's Team`; approvals by `approver_member_id`
Resource Owner	Timelines of the Resources they own, with holding Team and Request; utilization of their Resources	`owner_member_id = caller`
Capacity Planner	Every Team's roster, Requests, Shortlists, Account, Statements, credits, and person-days; every timeline; escalated Requests; every setting	none
Administrator	Members, Teams, roles, skills, profiles, and Resource Types; no Requests, Ledger, or settings	Members and Inventory definitions only

8.2 Audit Trail

Three records keep the history, each written by its owning module in the same transaction as the change it records, so a change and its record commit together or not at all (**QA-3**, **Lifecycle.States**).

- **Request history.** `RequestLifecycleService.record` writes a `request_event` row (Request, line, from-state, to-state, actor, time, note) for every state change of a Request or a Request Line. A Bump writes one row for each of the two lines, each naming the other (**Exclusive.Bump.Effect**); an approval decision also keeps `decided_by`, `decided_at`, and `reason` on the `approval` row.
- **Setting changes.** `SettingsService.update` writes a `setting_change` row (key, old value, new value, actor, time) for every Capacity Planner change (**Admin.Member.Create**, **Planner.Settings**).
- **Ledger.** Rows of `transaction` are append-only (**Ledger.Transaction.Kinds**): the application's database role may insert and select them but not update or delete them, so a correction is a new row naming the one it corrects.

Every table also carries `created_at` and `created_by`. There is no Hibernate Envers and no generic event store: one explicit table per concern, written where the change happens and readable with plain SQL. Nothing is purged

in Release 1 (**QA-7**).

Who the actor is. The actor is the Member who acted. Changes a job makes (Objection Window lapse, Waitlist expiry, Period rollover) record the system: `actor_id` is null and the note names the job. Consequences the system draws on other objects from a Member's action, such as a Waitlist fill after a Release or an Optional line dropped at allocation, also record the system, with a note naming the cause. When the Capacity Planner decides an escalated approval on a silent Manager's behalf, both are kept: the `approval` row keeps `approver_member_id`, the lending Manager it was addressed to, and sets `decided_by` to the Capacity Planner and `escalated` to true; the `request_event` row names the Capacity Planner as actor and the Manager in its note (**Matched.Escalate**).

8.3 Notifications

Table 16 lists every notification, derived from the SRS sentences cited in its last column. Texts are sketches; angle brackets mark values filled in.

Table 16: Notifications: event, recipient, text, and where they are raised.

Event	Recipient	Text sketch	Raised in (SRS)
Request Allocated	requesting Manager	"Request 42 is Allocated. Optional line 3 was dropped: no longer affordable."	`allocate` (**Lifecycle.Allocate**)
Request to Pending, Waiting, or Refused	requesting Manager	"Request 42 is Waiting: line 2 does not fit. Waitlist position 3."	each transition in **Requests** (**Lifecycle.Allocate**, **Exclusive.Waitlist.Join**)
Approval request	Candidate's Manager	"Team ⟨T⟩ asks to borrow ⟨Person⟩ Oct 12 to 23, for an Earning of 1000 credits. Decide by Oct 8, 17:00."	`select` (**Matched.Approve.Route**)
Candidate selected	the Candidate	"You are proposed for a Loan to Team ⟨T⟩, Oct 12 to 23. Your Manager decides."	`select` (**Matched.Loan.Form**)
Approval refused	requesting Manager	"⟨Manager⟩ refused ⟨Person⟩ for line 1: ⟨reason⟩. Select a replacement."	`decideApproval` (**Matched.Refuse.Replace**)
Loan formed	the Person; lending Manager	"You are on Loan to Team ⟨T⟩ from Oct 12 to 23." / "Earning of 1000 credits posted for ⟨Person⟩."	`allocate` (**Matched.Loan.Form**)
Objection Window lapsed	each Capacity Planner; requesting Manager	"Request 42 is escalated: 1 approval is outstanding past the Objection Window."	`onWindowLapse` (**Matched.Escalate**)
Line Bumped	Bumped line's Manager	"Line 2 (⟨Resource⟩, Oct 14, 09:00 to 12:00) was Bumped by a Priority line. Full Refund of 270 credits; first on the Waitlist."	after `applyBump` (**Exclusive.Bump.Effect**)
Waitlist fill, or waiting line expired	waiting Request's Manager	"Freed capacity filled line 2. Request 42 is Allocated."	`onCapacityFreed`, `expireWaitingLines` (**Exclusive.Waitlist.Join**)
Suggestion accepted or declined	the Member who made it	"Your Suggestion (⟨type⟩, Oct 12 to 16) was declined: ⟨reason⟩."	`SuggestionService` (**Suggest.Accept.Existing**)
Period rollover, mid-Period Grant	each Manager	"Period ⟨P⟩ began: Grant of 5000 credits. 320 granted credits expired."	`rollover`, `postGrant` (**Ledger.Grant.Period**)

Written in the caller's transaction. The module that causes the event calls `Notifier.notify(to, kind, text, link)` inside its own transaction, and `Notifier` inserts the row and hands it to each `Delivery`; `InAppDelivery`, the only one in Release 1, does nothing more, so a rolled-back action leaves no notification and nothing is sent after commit (**CI-2**). The client polls GET `/api/notifications` every 30 seconds and shows the unread count; `link` is the client route to open, such as `/requests/42`. Recipients are resolved through **Members**: a Team's Manager, or every Member holding `PLANNER`.

Email (Stretch). `EmailDelivery` implements `Delivery`. Inside the transaction it only registers an after-commit callback, which sends the message through the SMTP server the Administrator configured. A failed send is logged and the in-app row remains, so email is a copy and never the record. No caller of `Notifier` changes (Section 9).

8.4 Error Taxonomy and Validation

Four exception classes in `common` carry every refusal, and one `@RestControllerAdvice` in `common` turns them into the problem details of Table 13.

- `NotFoundException`: the object does not exist or lies outside the caller's scope (404).
- `ForbiddenException`: the caller sees the object but may not act on it, such as a lending Manager ending a Loan only the borrowing Manager may end (403). A missing role surfaces as Spring's `AccessDeniedException`, also 403.

- `RuleViolation(code, message)`: a business rule refused the action. Codes that describe the object's current state (`WRONG_STATE`, `ALREADY_DECIDED`, `WINDOW_LAPSED`, `SUGGESTION_CLOSED`, `NOT_ALLOCATED`, `PERIOD_STARTED`) map to 409, all others to 422; the message, written for a Manager, becomes `detail`.
- `ConcurrencyRetryException`: an allocation transaction failed twice on a deadlock or lock timeout, which Spring reports as `PessimisticLockingFailureException` (503). The retry advice sits outside the transaction advice and repeats the whole transaction, and since audit and notification rows are written inside it, a retry repeats no side effect. With ordered locks this should not happen, so a 503 points at a bug or an overloaded database.

An exclusion-constraint violation (SQLSTATE `23P01`) on `exclusive_occurrence` or `loan` cannot occur while the allocators choose under the type lock; if one does, it is logged as a bug, with its stack trace, and answered with 409 `ALLOCATION_CONFLICT`. Anything else is a 500 whose details stay in the log.

Where validation happens. At the edge, bean validation (`@Valid` on request bodies) checks shape only: presence, types, ranges such as a positive quantity, formats, an end after a start; it reads no data. The application services and domain objects check every business rule: a window in the future, a type not retired, affordability, the Candidate count and eligibility, capacity, the allowed transitions. Rules never live in controllers or in the client: the client may disable Submit when the draft view reports a shortfall, but it shows what the server computed, and Quotes are always fetched (Section 4.10). A refusal rolls back the whole transaction, audit and notification rows included (**QA-2**).

8.5 Configuration and Time

Deployment configuration comes from environment variables through Spring Boot's relaxed binding (Table 17), so the one Docker image runs unchanged everywhere and no secret lives in the image or the repository (**UD-2**). Fixed settings (Flyway on, schema validation, a two-thread scheduler) ship in the jar. Policy is not configuration: everything the Capacity Planner sets is data in **Settings**, changed at run time without a restart (**Planner.Settings**).

Table 17: Environment variables.

Variable	Meaning	Default or example
`SPRING_DATASOURCE_URL`	JDBC URL of the PostgreSQL 16 database	`jdbc:postgresql://db:5432/openbench`
`SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`	the application's database role (INSERT and SELECT only on `transaction`)	required
`SERVER_SERVLET_SESSION_TIMEOUT`	idle session expiry	`12h`
`OPENBENCH_TIME_ZONE`	the company time zone, an IANA id	required, e.g. `America/New_York`
`OPENBENCH_LOCK_TIMEOUT`	PostgreSQL `lock_timeout` on allocation transactions	`5s`
`OPENBENCH_INITIAL_ADMIN_USERNAME`, `OPENBENCH_INITIAL_ADMIN_PASSWORD`	the first Administrator, created only while no Member exists	unset after the first start

Time. Time is read only from one injected `java.time.Clock` bean set to the company zone, never from the no-argument `now()` methods, which an ArchUnit rule forbids; tests swap in a clock they move by hand to lapse an Objection Window or cross a Period boundary. All windows, Price Bands, and Periods are in the one company time zone (**LI-1**). Instants are stored as `timestamptz`, and the zone is applied wherever the calendar matters: the weekday and hour for Price Bands, the Monday-to-Friday days of a Loan, the midnight that starts a Period, the bump cutoff, the rollover job's `cron`, and the API's offsets. The JVM default zone and the database session's `TimeZone` are the same zone, so dates convert alike in Java and SQL. Pricing and the hourly check step through real hours as instants, so a daylight-saving day's 23 or 25 hours are each counted once.

8.6 Testing Strategy

Pure rules are tested as plain objects; everything the database guarantees is tested on PostgreSQL 16 in a container started by the test suite with the production migrations, never an in-memory substitute (**RC-3**, **CO-2**). `mvn verify` runs every level of Table 18 except usability; the container tests need Docker.

Table 18: Test levels.

Level	What it covers and how	Requirements
Unit: pricing and time	JUnit 5 with a fixed `Clock`, no Spring or database: `Pricing.quote` (band edges, premium, a daylight-saving day) and `Pricing.refund` (each step, never 100%); window overlap and weekly recurrence expansion; hourly sums and the earliest failing hour	**Ledger.Price.Formula**, **Release.Refund**, **Exclusive.Conflict**, **Metered.Pool.Check**
Unit: rules	The Shortlist filter (skills, availability, overlapping Loan, own Team, inactive) and order; the Request and line state machines, every allowed transition and every other one refused	**Matched.Shortlist.Build**, **Lifecycle.States**
Integration	Spring Boot tests on PostgreSQL 16 in Testcontainers with the Flyway migrations: the hourly-sum SQL with Pool Capacity and Quota, both exclusion constraints, the append-only permissions on `transaction`, lapse and rollover with a moved clock, scoping (another Team's Request is 404), the problem-detail mapping	**QA-1**, **QA-3**, **SE-4**, **PE-4**
Concurrency	Two threads released together by a latch, each in its own transaction. Metered: both ask for the last units of one hour; exactly one is Allocated, the other Waiting, and no hour exceeds Pool Capacity. Exclusive: both ask for one instance in overlapping windows; exactly one gets it	**QA-1**, **CO-4**
Architecture	One ArchUnit test class: the dependency rule of Section 2 (layer order, no cycles, entities and repositories private to their module, `ReductionGuard` the only upward reference); `@PreAuthorize` on every service method a controller calls; no no-argument `now()`; no `isolation` attribute on `@Transactional` in **Allocation** and **Requests**	**CO-7**, **QA-6**, **SE-1**
End-to-end	One test over HTTP with a real session and CSRF token: a Manager submits the three-line Request (Matched, Exclusive, Metered) and selects a Candidate; the lending Manager approves; the Request is Allocated and the Charges and the Earning appear on both Teams' Statements	**Lifecycle.Allocate**, **QA-2**
Front end	Vitest and React Testing Library at the route level, the API mocked through the generated types: each route renders for the right roles, sends the right call, and shows the problem `detail` on a refusal	**UI-1**, **UI-2**, **UI-3**
Usability	Before the final demonstration, at least three untrained people from outside the project team each draft and submit a three-line Request in under five minutes; times and problems are recorded	**QA-5**

Every Release 1 functional requirement also has at least one integration test on PostgreSQL in Testcontainers whose display name carries its identifier (**QA-7**); unit tests add the edge cases. Each test's display name carries the identifiers it covers, so the traceability of Appendix A can be checked against the test report (**QA-7**).

8.7 Logging

The server writes one JSON object per line to standard output (Spring Boot structured logging), which Docker keeps. A servlet filter gives every HTTP request a correlation id, or takes the one nginx sent in `X-Request-Id`, puts it in the logging context, and returns it in the `X-Request-Id` response header, so a Manager reporting an error can cite it; every job run gets its own.

`RequestLifecycleService.record` writes one INFO line per state transition next to the `request_event` row, with Request id, line id, from-state, to-state, actor id or `system`, and the correlation id. There is also one line per Transaction posted, setting change, authentication attempt (username, remote address, outcome), and job run (job, items processed, duration); ERROR with the stack trace for every 500 and exclusion-constraint violation; WARN for every 503.

```
{"@timestamp":"2026-10-12T09:14:03.512-04:00","level":"INFO","correlationId":"7f3c2a9e",
 "event":"transition","request":42,"line":null,"from":"PENDING","to":"ALLOCATED","actor":17}
```

Logs carry no personal data beyond ids (**SE-3**): Member, Team, Request, and Resource ids, never names, email addresses, skills, availability, notes, reasons, or request bodies. The username appears only in authentication lines, where it is the identifier being checked; a password never appears.

9 Maintenance and Evolution

Release 1 is fixed. Stretch items are built only if time allows, and Future items have no requirements yet. For each of them, and for the operational changes a successful deployment would bring, the design names the one place it lands, so none of them is a precondition for Release 1 and none changes the dependency direction of Section 2.

9.1 Extension Points

Table 19 covers every Stretch and Future item of the SRS scope table plus two operational changes. Most rows add a class or a column behind an interface that already exists.

Table 19: Extension points for the Stretch, Future, and operational items.

Item	Tier	Where it lands	What changes	What does not change
Service tiers and preemption on Metered pools (**Metered.Tier.Define**, **Metered.Tier.Bump**)	Stretch	**Allocation**, **Ledger**, **Settings**, **Requests** (a tier column and setter)	`MeteredAllocator` also implements `Bumpable`, planning Bumps of lower-tier units outside the bump cutoff; a tier premium multiplies the Unit Price in `Pricing.quote` as the priority premium does; tiers per type become settings; `request_line` gains a tier	the lifecycle in `RequestLifecycleService`, which already asks `AllocatorRegistry` for `Bumpable` on any class; the Bump Refund and Waitlist path; `ExclusiveAllocator`
Demand surcharge (**Ledger.Surcharge**)	Stretch	**Requests**, **Ledger**, **Settings**	**Requests** reads the requested window's fullness (allocated plus held over capacity) from `AllocationQueries` and passes it into `Pricing.quote` as an extra input; one multiplier inside `quote`; the surcharge per type is a setting	`Pricing` stays pure and **Ledger** never calls **Allocation**; the Quote still locks at submission and the Charge equals it
Actual usage recording and comparison (**Metered.Usage.Record**, **Metered.Usage.Compare**, **Dashboard.Usage**)	Stretch	**Allocation**, **Dashboard**	a `used_units` column on `metered_allocation`, written at Period end by a new Resource Owner operation in **Allocation**; one **Dashboard** view comparing requested and used units per Team and type	**Ledger**, since usage has no Ledger effect; the hourly check, which keeps using requested units
Email notifications (**CI-2**)	Stretch	**Notifications**, **Settings**	`EmailDelivery` implements `Delivery` and sends after commit; SMTP host, port, and sender become settings the Administrator edits, the SMTP password an environment variable	every caller of `Notifier`; the in-app row stays the record
Demand analytics on declined Suggestions	Future	**Dashboard**	one view over Declined `suggestion` rows (type, quantity, window, skills, reason) per Period	no data changes: Suggestions keep their original ask and are never deleted (**Suggest.Create**)
Person consent (lifts **LI-4**)	Future	**Allocation**	`MatchedAllocator.select` first creates a consent approval addressed to the Person, then the Manager's; `approval` gains a kind; a Person's refusal takes the refusal path	`decideApproval`, the Objection Window, escalation, **Requests**, **Ledger**
Skill-based Day Rate	Future	**Settings**, **Ledger**	a rate per skill in Settings with the one Day Rate as the default; a rate lookup next to `Pricing` computes a Loan's Earning from its skills	`postEarning`, `correctEarning`, Loans, the Shortlist; Matched lines still cost the borrower nothing (ADR-0004)
Team rank priority	Future	**Allocation**, **Settings**	a rank per Team set by the Capacity Planner; one comparator in the Bump plan of `ExclusiveAllocator` consults it	Priority by premium (ADR-0003); the exclusion constraint; Refunds; **Requests**
Inter-Team transfers	Future	**Ledger**	a fifth Transaction kind, Transfer, and one operation posting a pair of Transactions between two Teams with a reason; both Statements show it; the glossary's "exactly four kinds" becomes five	append-only Transactions, Account arithmetic, the other four kinds, every caller of `LedgerService`

continued on the next page

Item	Tier	Where it lands	What changes	What does not change
Forecasting accuracy reward	Future	**Ledger**, **Jobs**	at rollover, a Grant computed from requested versus used units (needs usage recording)	the Grant posting path and its Transaction kind
Usage feeds	Future	**Allocation**	an adapter fills `used_units` from a scheduler or license server instead of the Resource Owner	as for usage recording
Auctions	Future	**Allocation**	a new Allocation Class whose allocator defers, as `ApprovalDriven` does, until bidding closes, with the winning bid as the Quote; the largest Future item, needing its own design pass	the four Transaction kinds (a winning bid posts a Charge); the existing allocators
SSO (lifts **LI-3**)	Future	`security`	a Spring Security authentication provider (OpenID Connect or SAML) mapping the identity provider's subject to `member.username`; password login can be switched off	**Members**, roles, sessions, every `@PreAuthorize`
Calendar integration (lifts **AS-4**)	Future	**Members**	an adapter behind `isAvailable` and `availableWithSkills` that also removes calendar busy days	**Allocation** and the Shortlist call the same operations
HR integration	Future	**Members**	an import adapter that creates, moves, and deactivates Members and Teams from the HR system	every module above **Members**; the Administrator screens stay for corrections
Cluster-scheduler integration	Future	**Inventory**, **Jobs**	an adapter behind **Inventory** sets Resource units from the scheduler through `setUnits` and its guard; a new consumer of allocation records, a polling step in **Jobs**, pushes committed Metered allocations to the scheduler	allocation transactions make no remote call; `MeteredAllocator`; there is still no event bus
A second application instance	Operational	**Jobs**, `security`	a job lock table each job claims with a conditional update before it runs; sticky sessions at nginx or a shared session store (Spring Session over JDBC)	allocation correctness: type locks and exclusion constraints live in the database and already hold across instances
A new Allocation Class (**QA-6**, **CO-7**)	Operational	**Allocation**	one `Allocator` bean and the capability interfaces it needs; its name in the `AllocationClass` enumeration and the check on `resource_type`	**Requests**, **Ledger**, and the existing allocators: `AllocatorRegistry` finds the bean by `allocationClass()`

One dependency added during design. **Ledger** calls **Notifications** to tell each Manager about Period rollover and mid-Period Grants (Table 16). **Notifications** sits in the bottom layer, so the dependency rule still holds and the ArchUnit test allows it.

9.2 Assumptions That Would Break the Design

Table 20 lists the assumptions the design leans on hardest, what relies on each, and what would have to change if it stopped being true.

Table 20: Assumptions and what depends on them.

Assumption	What relies on it	If it changed
One company in one time zone (**LI-1**)	Price Bands, Period boundaries, person-days, the rollover time, and the API's offsets all use one configured zone	instants are already stored zone-free, but Price Band lookup, Periods, and rollover would need a zone per Team or Resource, and Periods would no longer turn over at one moment
One server instance, no high availability (**LI-1**, **QA-4**)	each job runs once because one process runs it; sessions live in memory	a job lock table and shared sessions (Table 19); allocation correctness is unaffected
Hour granularity for Metered pricing and checks; day granularity for Loans	the hourly sum and the Quote step through whole hours; Loans, Earnings, and the `loan` exclusion constraint use dates	finer windows multiply the hourly check's buckets and change Quote arithmetic; half-day Loans change person-days, the Day Rate, and the date-range constraint
The Capacity Planner sets Grants before the first Period (**AS-1**)	every Exclusive and Metered line needs credits; rollover posts the planned Grants	without Grants only Matched lines can be submitted (`INSUFFICIENT_BALANCE`); the design invents no default Grant

Assumption	What relies on it	If it changed
Loans are recorded honestly (**AS-3**)	the Earning posts when the Loan forms and is corrected only by an early end	confirming the work would need a record of days actually worked, as usage recording does for Metered pools, and an Earning posted on confirmation
The scale of **PE-1**: 500 Members, 200 Resources, 10,000 Requests a year	the Shortlist is built on read, hourly sums are computed by query, and the Waitlist is walked inside the releasing transaction	at a hundred times the scale: stored hourly totals per type, a precomputed Shortlist, and a Waitlist fill moved to `DueWorkJob`

9.3 Risks

Table 21 lists the technical and schedule risks of this design; likelihood and impact are low, medium, or high, as in the SRS.

Table 21: Risks and their mitigation in the design.

Risk	Likelihood	Impact	Mitigation in the design
Lock-order discipline breaks: a new path consumes capacity without the type locks, or takes them out of order	medium	high: an oversold Metered pool, or deadlocks	`AllocationLock.lockTypes` takes a sorted set, so the order holds by construction; every path that consumes or reduces capacity (submit, Waitlist fill, Bump, Pool Capacity reduction) runs through `RequestLifecycleService` or `ReductionGuard`, which lock first; the exclusion constraints backstop Exclusive instances and Loans; `lock_timeout` turns a stuck wait into a 503. Metered pools have no constraint backstop, so the Metered race test and code review guard them
The exclusion constraints depend on the `btree_gist` extension	low	high: the server does not start	the first Flyway migration creates the extension, so a missing privilege fails at start-up and not at the first Request; the extension ships with PostgreSQL's contributed modules and the official image; the installation guide states what the database role needs (**UD-2**); every test run proves it on PostgreSQL 16
Waitlist walk cost grows with the Waitlist	low at **PE-1**	medium: Releases slow down and hold the type lock longer	`waitingFor(type, freed)` reads only entries whose window overlaps the freed one, through an index on type and start; expired entries leave through `expireWaitingLines`; the walk stops at the first Request that allocates; logged durations show growth; if needed, the fill moves to `DueWorkJob` and still meets **PE-4**
The single polling job stalls or fails	low	medium: lapses and expiries come late (**PE-4**)	one transaction per Request, so one failure does not stop the rest; due work is found by query, so a missed run or a restart is caught up on the next run; a scheduler pool of two threads, so rollover never delays `DueWorkJob`; one log line per run with counts and duration
The front end drifts from the API	medium	medium: screens fail at run time	client types are generated from the OpenAPI document on every build, so a changed endpoint fails the TypeScript compile; the end-to-end test; Quotes and rules live only on the server
Four students, one semester (**RI-5**)	high	high: Release 1 incomplete	Release 1 is fixed and every Stretch and Future item is an extension point, never a precondition; one process, one database, no broker; modules are built bottom-up in the layer order against the interfaces of Section 4, each testable alone; Stretch work starts only after the end-to-end test of the three-line Request passes

10 References

1. J. Nana, B. Lujan, F. Turopov, Y. Shnazbayev, *Software Requirements Specification for OpenBench*, version 0.3, 2026-10-05; short edition `docs/srs/OpenBench-SRS-short.pdf` (the identifiers used here) and full edition `docs/srs/OpenBench-SRS.pdf`, project repository.
2. J. Nana, B. Lujan, F. Turopov, *OpenBench: A Policy-Pluggable Internal Resource Marketplace*, CSDS 393 informal project proposal, 2026.
3. Project vocabulary file `CONTEXT.md` and architecture decision records ADR-0001 to ADR-0007 in `docs/adr/`, project repository, 2026.
4. Spring Boot reference documentation, version 3. https://docs.spring.io/spring-boot/reference/
5. Spring Security reference documentation. https://docs.spring.io/spring-security/reference/
6. Spring Data JPA reference documentation, including locking. https://docs.spring.io/spring-data/jpa/reference/
7. Hibernate ORM 6.6 user guide. https://docs.hibernate.org/orm/6.6/userguide/html_single/
8. PostgreSQL 16 documentation, https://www.postgresql.org/docs/16/, in particular:
   - range types, https://www.postgresql.org/docs/16/rangetypes.html;
   - exclusion constraints, https://www.postgresql.org/docs/16/ddl-constraints.html;
   - the `btree_gist` extension, https://www.postgresql.org/docs/16/btree-gist.html;
   - explicit locking, https://www.postgresql.org/docs/16/explicit-locking.html;
   - transaction isolation, https://www.postgresql.org/docs/16/transaction-iso.html.
9. Flyway documentation. https://documentation.red-gate.com/fd
10. Testcontainers for Java, PostgreSQL module. https://java.testcontainers.org/modules/databases/postgres/
11. React documentation. https://react.dev/
12. Vite guide. https://vite.dev/guide/
13. TanStack Query documentation. https://tanstack.com/query/latest
14. M. Nottingham, E. Wilde, S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023. https://www.rfc-editor.org/rfc/rfc9457
15. Object Management Group, *Unified Modeling Language*, version 2.5.1, 2017. https://www.omg.org/spec/UML/2.5.1/
16. PlantUML language reference. https://plantuml.com/
17. ArchUnit user guide. https://www.archunit.org/userguide/html/000_Index.html
18. springdoc-openapi documentation. https://springdoc.org/

A Traceability

Each row traces one identifier of the SRS version 0.3 short edition to the module or modules that satisfy it, the first named owning the requirement and any others contributing to it, and to the sections of this document that design it; functional requirements appear in SRS order, Stretch requirements point at their extension point in Section 9, and the remaining identifiers follow by class. Every Release 1 requirement maps to at least one module.

Table 22: Traceability from SRS identifiers to modules and sections.

Identifier	Requirement	Module(s)	Section
Functional requirements, Release 1			
Inventory.Type.Define	Resource Types defined as data, without code	**Inventory**	4.2, 6
Inventory.Resource.Add	Add, edit, retire Resources; reductions guarded	**Inventory**, **Allocation**	4.2, 4.5
Inventory.View.Holder	Show who holds a Resource; filter Resources	**Allocation**, **Inventory**	4.5, 4.2, 7
Profile.Skills	One Team per Member; skills from a list	**Members**	4.1
Profile.Availability	Availability ranges; one Loan at a time	**Members**, **Allocation**	4.1, 4.5, 6
Suggest.Create	Member proposes one line to own Manager	**Requests**	4.7
Suggest.Accept.Existing	Manager accepts or declines an Open Suggestion	**Requests**, **Notifications**	4.7, 4.4
Draft.Create	Only a Manager creates and withdraws Requests	**Requests**	4.7, 8
Draft.Line.Add	Line names type, quantity, and future window	**Requests**	4.7
Draft.Line.Required	Lines marked Required or Optional; split allowed	**Requests**	4.7
Draft.Quote.Line	Quotes, sums, and shortfall shown while drafting	**Requests**, **Ledger**	4.7, 4.6
Draft.Submit.Lock	Submission locks every Quote and evaluates lines	**Requests**	4.7, 5
Matched.Shortlist.Build	Shortlist of eligible Persons, least lent first	**Allocation**, **Members**	4.5, 5
Matched.Approve.Route	One approval per Candidate to their Manager	**Allocation**, **Requests**, **Notifications**	4.5, 5
Matched.Refuse.Replace	Refusal asks for a replacement; window continues	**Requests**, **Allocation**	4.7, 5
Matched.Escalate	Lapsed approvals go to the Capacity Planner	**Requests**, **Allocation**	4.7, 5
Matched.Loan.Form	Approved Candidates form Loans; Earnings post	**Allocation**, **Ledger**, **Notifications**	4.5, 5, 6
Exclusive.Conflict	No overlapping Holds or allocations per instance	**Allocation**	4.5, 5, 6
Exclusive.Waitlist.Join	Unmet Required lines join an ordered Waitlist	**Allocation**, **Requests**	4.5, 5
Exclusive.Priority.Bump	Priority line Bumps standard lines outside cutoff	**Allocation**	4.5, 5
Exclusive.Bump.Effect	Full Refund, Waitlist head, and notice on Bump	**Allocation**, **Requests**, **Ledger**, **Notifications**	4.5, 5
Metered.Line.Form	Metered line: units over window, all or nothing	**Allocation**, **Requests**, **Ledger**	4.5, 4.7
Metered.Pool.Check	Hourly sums within Pool Capacity and Quota	**Allocation**	4.5, 5, 6
Lifecycle.States	Seven Request states with recorded transitions	**Requests**	4.7, 5
Lifecycle.Evaluate	Every line evaluated in one transaction	**Requests**, **Allocation**	4.7, 5
Lifecycle.Hold.Place	Pending Request places Holds; held credits count	**Requests**, **Allocation**, **Ledger**	4.7, 4.5, 5
Lifecycle.Window.Start	Objection Window runs once; lapse escalates	**Requests**, **Jobs**	4.7, 4.9, 5
Lifecycle.Allocate	All Holds commit and credits post together	**Requests**, **Allocation**, **Ledger**, **Notifications**	4.7, 5
Lifecycle.Waiting.Reevaluate	Freed capacity re-evaluates Waiting Requests	**Requests**, **Allocation**	4.7, 5
Release.Line	Release all or part; offer freed capacity	**Requests**, **Allocation**	4.7, 4.5, 5
Release.Refund	Refund by the Refund Schedule, never 100%	**Ledger**, **Requests**	4.6, 5
Release.Loan.Prorate	Ending a Loan early corrects the Earning	**Requests**, **Allocation**, **Ledger**	4.7, 4.6
Ledger.Account	Team Accounts and per-Period Statements	**Ledger**	4.6, 6
Ledger.Transaction.Kinds	Four kinds of Transaction, all append-only	**Ledger**	4.6, 6
Ledger.Grant.Period	Grants post and expire at Period boundaries	**Ledger**, **Jobs**	4.6, 4.9, 5
Ledger.Price.Formula	Quote from Unit Price, bands, and premium	**Ledger**, **Settings**	4.6, 4.3
Ledger.Earning.Post	An Earning of the Day Rate per person-day	**Ledger**	4.6
Dashboard.Utilization	Utilization and idle capacity per type	**Dashboard**	4.8, 6
Dashboard.Credits	Team credits and person-days, scoped by role	**Dashboard**	4.8, 8
Admin.Member.Create	Administrator manages Members, Teams, and roles	**Members**, **Settings**	4.1, 4.3, 8
Planner.Settings	Capacity Planner sets every policy as data	**Settings**, **Requests**	4.3, 4.7
Functional requirements, Stretch			
Metered.Tier.Define	Service tiers with premiums on Metered types	**Settings**, **Ledger** (tier premium in `Pricing`)	9, service tiers
Metered.Tier.Bump	Higher-tier Metered lines Bump lower tiers	**Allocation** (`Bumpable` on `MeteredAllocator`)	9, service tiers
Metered.Usage.Record	Resource Owner records units actually consumed	**Allocation** (usage beside `metered_allocation`)	9
Metered.Usage.Compare	Requested versus used units per Team	**Dashboard** (one more view)	9
Ledger.Surcharge	Demand surcharge multiplier on the Unit Price	**Ledger** (multiplier in `Pricing`), **Settings**	9, demand surcharge

Traceability (continued)

Identifier	Requirement	Module(s)	Section
Dashboard.Usage	Dashboard shows requested versus used units	**Dashboard** (one more view)	9
Constraints, interfaces, and nonfunctional requirements			
CO-1	Java and Spring Boot; React and Vite	All modules	2, 4.10
CO-2	PostgreSQL 16, also in tests via Testcontainers	All back-end modules	6, 8
CO-3	Front end uses only the REST API	**Web client**	7, 4.10
CO-4	Holds, Charges, and Earnings in one transaction	**Ledger**, **Requests**	4.6, 5
CO-5	Timed work as in-process scheduled jobs	**Jobs**	4.9, 5
CO-6	Hashed passwords; Spring Security enforces roles	**Members**, `security` package	7, 8
CO-7	One allocator per Allocation Class	**Allocation**	4.5, 9
UI-1	Draft Request and Request detail screens	**Web client**, **Requests**	4.10, 4.7
UI-2	Shortlist and Approvals screens with actions	**Web client**, **Allocation**	4.10, 4.5
UI-3	Inventory, Dashboard, Settings, and Administration screens	**Web client**	4.10
SI-1	PostgreSQL via JDBC, migrated at start-up	All back-end modules	6
SI-2	REST/JSON API covers every use case	All back-end modules, **Web client**	7
CI-1	HTTPS only; plain HTTP redirects	All modules (deployment)	2, 7
CI-2	Notifications delivered in the application only	**Notifications**	4.4, 8
PE-1	Scale: 500 Members, 10,000 Requests a year	All modules	2, 6
PE-2	Any page within 2 s at 95th percentile	All modules	2, 7
PE-3	Shortlist of 500 Members within 3 s	**Allocation**	4.5
PE-4	Lapses within 5 min, rollover within 15 min	**Jobs**	4.9, 5
SE-1	Server enforces roles on every action	All modules, `security` package	8, 7
SE-2	Hashed passwords, HTTPS, 12-hour idle sessions	**Members**, `security` package	7, 8
SE-3	No personal data beyond the profile fields	**Members**	4.1, 6
SE-4	Each role sees only its own scope	All modules	8
QA-1	No double allocation or overrun under concurrency	**Allocation**	4.5, 5, 6
QA-2	All-or-nothing commit of Holds and credits	**Requests**, **Ledger**	4.7, 5
QA-3	Append-only Ledger; state changes recorded	**Ledger**, **Requests**, **Settings**	8, 6
QA-4	Business hours; four desktop browsers supported	**Web client**	4.10, 2
QA-5	Untrained Manager submits three lines in 5 min	**Web client**	4.10
QA-6	New type needs no code; new class, one allocator	**Allocation**, **Inventory**	4.5, 9
QA-7	Keep records two years; test every requirement	All modules	6, 8
User documentation, assumptions, and dependencies			
UD-1	One-page in-app guide per role	**Web client**	4.10
UD-2	Guide for installation and first configuration	**Members**, **Settings**	8, 9
UD-3	Help text for settings and Draft fields	**Web client**	4.10
AS-1	Grants set for every Team before first Period	**Settings**, **Ledger**	4.3, 9
AS-2	Resource Owners keep the Inventory current	**Inventory**	4.2
AS-3	Managers record Loans honestly; no work check	**Allocation**	4.5, 9
AS-4	Availability from profiles only; no calendars	**Members**	4.1, 9
DE-1	Platform software available on site	All modules	2
DE-2	Toolchain pinned in the repository's Nix flake	All modules (build)	2

B Glossary

The domain vocabulary below is generated at build time from the project's shared vocabulary file and is identical to the glossary of the SRS; the design terms after it are specific to this document.

People and roles

Member Any user of the system. Belongs to exactly one Team, has a skill profile and availability, and is lendable as a Person.

Team The organizational unit that owns an Account and every Request made on its behalf. Has exactly one Manager.

Manager The Member who leads a Team. The only actor who submits Requests, and the approver for lending that Team's Members.

Resource Owner A Member responsible for the Inventory records of specific Exclusive or Metered resources. Never approves allocations.

Capacity Planner The role that sets each Team's Grant per Period, Unit Prices, the Day Rate, and Refund Schedules, and receives escalations.

Administrator The role that manages users, Teams, and Resource Type definitions. Configures the system; sets no policy.

Resources

Resource Anything that can be allocated: a Person, an Exclusive resource such as a room, or a Metered pool such as GPU hours. Every Resource has exactly one Resource Type.

Resource Type A named definition, created as data by an Administrator, that fixes a Resource's Allocation Class and attributes.

Allocation Class One of three policies a Resource Type declares, deciding how Requests for it are allocated: Matched, Exclusive, or Metered.

Matched The Allocation Class for People. Allocated by matching skills and availability, gated by the Person's Manager.

Person A Member viewed as a Matched Resource: someone with skills and availability who can be lent to another Team.

Exclusive The Allocation Class for Resources held whole over a time window, such as rooms or instruments. Allocated with conflict detection, a waitlist, and priority bumping.

Metered The Allocation Class for divisible Resources drawn against a Team quota, such as compute hours or license seats.

Pool Capacity The total units a Metered Resource Type can supply at once, such as 32 GPUs, maintained by its Resource Owner. Allocations in any hour may not exceed it.

Quota An optional per-Team cap on the units of one Metered Resource Type the Team may hold concurrently, set by the Capacity Planner. A safety valve against monopolizing a pool, not a rationing mechanism.

Inventory The record of every Resource that exists, its Resource Type, and who currently holds it or has it on Hold.

Requests

Request A single request, owned by a Team and submitted by its Manager, containing one or more Request Lines that may span all three Allocation Classes.

Request Line One need inside a Request: a Resource Type, a quantity, and a time window. Every line has exactly one Allocation Class, taken from its Resource Type.

Suggestion A single Request Line a Member proposes to their own Manager. Has no allocation effect until the Manager accepts it into a Request; several Suggestions may be accepted into the same line.

Required line / Optional line A Manager's marking on each Request Line. A Request is fulfilled only when every Required line is allocated; Optional lines allocate if they can and never block the outcome.

Hold A tentative, uncharged allocation of an Exclusive or Metered line placed while a Request's Matched lines await approval. Commits when the Request is fulfilled, releases when it fails or the Objection Window lapses.

Objection Window The bounded period a lending Manager has to approve or refuse a Matched line before the Request escalates to the Capacity Planner. Also the lifetime of any Hold on that Request.

Candidate A Person who satisfies a Matched line's skills and availability, has no Loan overlapping the line's window, and appears on its Shortlist. Becomes a Loan once their Manager approves.

Shortlist The list of Candidates the system produces for a skill-based Matched line, ordered by fewest person-days lent in the current Period, from which the requesting Manager selects.

Waitlist The ordered list of Required lines that could not be satisfied for a Resource Type and window. Priority lines first, then by submission time; a Bumped line goes to the head. Optional lines are never waitlisted.

Loan A Person allocated to a Team other than their own for a time window. The fulfilled form of a Matched line. A Person is in at most one Loan at a time.

Priority line An Exclusive Request Line that is charged a premium, a multiple of the Unit Price set by the Capacity Planner. Can Bump a standard line and cannot itself be Bumped.

Bump The displacement of a standard Exclusive line by a Priority line. The displaced line goes to the head of the waitlist with a full Refund. Not allowed inside the bump cutoff before the window starts.

Release A Manager giving back an allocated Exclusive or Metered line, in whole or in part, before it is consumed, or ending a Loan early. Triggers a Refund according to the Refund Schedule.

Credits

Credit The unit of account in the Ledger. No real money; used for showback, not billing.

Ledger The record of every Team's Account and every Transaction. Holds no capacity information.

Account A Team's standing in the Ledger: its granted credits, its earned credits, and its held credits.

Balance A Team's granted credits plus earned credits.

Available Balance Balance minus held credits. What a Team can commit to a new Request.

Held credits Credits set aside for a Request that is Pending, equal to what its Holds will cost when they commit.

Transaction One entry in the Ledger against an Account. Exactly four kinds: a Grant (from the Capacity Planner), a Charge

(for an allocated Exclusive or Metered line), an Earning (for a Loan), or a Refund (for a Release).

Grant Credits given to a Team by the Capacity Planner at the start of a Period, or as a mid-period adjustment. Expire at the end of the Period.

Period The fixed calendar interval over which Grants are issued and expire. Set by the Capacity Planner.

Statement A Team's per-Period view of its Account: Grants, Charges, Earnings, Refunds, and what expired. The showback artifact.

Unit Price The base credits per unit of an Exclusive or Metered Resource Type (per hour of a room, per GPU hour), set by the Capacity Planner, before any Price Band multiplier. Matched lines have no Unit Price.

Price Band A time-of-day and day-of-week window, defined per Resource Type by the Capacity Planner, carrying a multiplier on the Unit Price. Rewards use at low-demand times.

Quote The credits a Request Line will cost, shown to the Manager while drafting and locked when the Request is submitted. The Charge that posts equals the Quote.

Day Rate The flat credits a Team earns per person-day of a Loan, set once by the Capacity Planner. The same for every Person regardless of skill.

Refund Schedule The rule, set per Resource Type by the Capacity Planner, giving the fraction of a Charge returned on Release as a step function of how far ahead of the window start the Release happens. Never reaches 100%.

Design terms

Module A unit of the design with one area of responsibility: one Java package under `edu.cwru.openbench`, or the web client. A back-end module owns its tables and exposes only its application services and the DTOs they take and return.

Application service A Spring bean that is a module's public entry point, such as `RequestLifecycleService`. Each method runs in one transaction unless marked read-only and carries the role check for its action.

Port An interface declared by a lower module and implemented by a higher one, so the lower module can ask a question without depending upward. The only port is `ReductionGuard`, declared by Inventory and implemented by Allocation.

Capability interface An optional interface an allocator implements beside `Allocator` when its Allocation Class has that behavior: `Waitlistable`, `Bumpable`, or `ApprovalDriven`. Requests asks the `AllocatorRegistry` whether a class has one.

Allocator The component that evaluates, holds, commits, releases, and cancels lines of one Allocation Class behind the common `Allocator` interface: `ExclusiveAllocator`, `MeteredAllocator`, or `MatchedAllocator`.

Hold (as a row state) The state `HELD` of an Exclusive occurrence or Metered allocation row. It blocks capacity exactly as `ALLOCATED` does but carries no Charge; committing turns it into `ALLOCATED`, and a release or lapse ends it.

Occurrence One time window of an Exclusive line on one instance. A line with a weekly recurrence has several, each stored as its own row so that the exclusion constraint checks every window.

Exclusion constraint A PostgreSQL constraint that rejects a row whose values conflict with an existing row under the given operators. It keeps Held or Allocated occurrences of one Resource from overlapping and Loans of one Person from overlapping.

Type lock The row lock (`SELECT ... FOR UPDATE`) that every allocation decision takes on each Resource Type row it touches, in ascending id order, so that checks and writes for the same type never interleave.

Transaction (database) A unit of database work that commits entirely or not at all; one per user or job action, at READ COMMITTED. Not to be confused with a Ledger Transaction, which is one row of the `transaction` table.

Append-only table A table the application may insert into and read but never update or delete from, enforced by the database role's grants. The Ledger's `transaction` table is one; a correction is a new row.

Migration A versioned SQL script that Flyway applies at start-up to bring the schema to the current version. Production and tests run the same migrations, and Hibernate only validates the result.

Read model A read-only view of data shaped for display rather than for updates. The Dashboard module's SQL views are the only read model in the design.

Scheduled job Work the server process runs on a timer rather than on a user action: `DueWorkJob` every 60 seconds and `PeriodRolloverJob` at each Period start and on start-up.

ArchUnit test A test, written with the ArchUnit library, that inspects the compiled classes and fails the build when a dependency breaks the dependency rule, such as a lower module importing a higher one.

Testcontainers A test library that starts a disposable PostgreSQL 16 container for the integration tests, so tests run against the same database and migrations as production.

Problem detail The JSON error body of RFC 9457 (`application/problem+json`). Every error response carries one, with a stable `code` such as `INSUFFICIENT_BALANCE`.

Session cookie The HTTP cookie that identifies a signed-in Member's server-side session after form login. It is sent only to the same origin as the single-page application and expires after 12 hours idle.

Single-page application A web client that loads once and then changes screens in the browser, fetching data from the REST API. Here, the React client served by the same server process.

C Inspection Report

This document was drafted on 2026-10-05 from the design brief and the architecture decision records. On the evening of 2026-10-05 the four team members, J. Nana, B. Lujan, F. Turopov, and Y. Shnazbayev, ran a drafting review over the complete first build, with a reviewing agent, that checked every module, interface, and table name against the brief, every requirement against Appendix A, every worked example's arithmetic, and every figure against the text that cites it. It raised 42 issues; the 21 that touched the design (eight of them rated high) are the rows below, each credited to the member who raised it, all resolved in this version, and the rest were wording, spelling, and layout corrections applied directly. Each member then reads the whole document and records further findings; the team meets, decides a resolution for each finding, and records its status.

One row per finding. Categories: ambiguity, incompleteness, incorrectness, inconsistency, infeasibility, traceability, clarity; status Open, Closed (version), or Rejected (reason).

Table 23: Inspection findings.

ID	Found by	Location	Category	Finding	Resolution	Status
I-1	J. Nana	Sec. 4.7, 4.9, 3.2.4, 5.3.4; Fig. 15	inconsistency	Lapse handling defined two ways: `lapsedWindows` covered Pending and Waiting Requests in one place and Pending only elsewhere, so an approval left Outstanding after a refusal would never escalate, and nothing stopped a re-escalation every minute.	`lapsedWindows` returns Pending or Waiting Requests whose window ended with an unescalated Outstanding approval; `onWindowLapse` is idempotent. All five places agree.	Closed (1.0)
I-2	B. Lujan	Table 6; Sec. 4.7; Fig. 12, 16	inconsistency	`requireAffordable` was said to lock the account row, yet S1 and S5 call it before `lockTypes`, reversing the stated order (types, then account) and inviting deadlock.	`requireAffordable` is an unlocked early refusal; `holdCredits` and `commitCharges` take the account lock after the type locks and repeat the check.	Closed (1.0)
I-3	F. Turopov	Sec. 4.7 vs 5.3.2; Fig. 13	inconsistency	The `allocate` step and scenario S2 ordered commits, Charges, Loans, and the dropping of Optional lines differently; S2 charged lines it then dropped.	S2 text and figure follow the Sec. 4.7 order: re-evaluate, drop Optional lines, commit, charge, form Loans, post Earnings.	Closed (1.0)
I-4	Y. Shnazbayev	Sec. 6.2; Table 12; Fig. 20	inconsistency	The hourly-sum query joined Requests' tables for the Team, against the dependency rule, while Sec. 4.5 put `team_id` and `request_id` on Allocation's own rows; the schema omitted both columns.	Query reads only Allocation tables; both columns added to the table list, the DDL, and the schema figure.	Closed (1.0)
I-5	B. Lujan	Fig. 6, 20; Sec. 6.2	inconsistency	The Loan exclusion constraint was drawn with a `WHERE` clause excluding early-ended Loans, which would let a new Loan overlap days a shortened Loan actually lent.	Clause removed everywhere; `endLoanEarly` shortens `ends_on`, so the Loan stays in the constraint for the days it lent.	Closed (1.0)
I-6	J. Nana	Table 12; Fig. 20 vs Sec. 4.6	incompleteness	Ledger relied on `held_credit` and `period_close` tables and a `transaction.reason` column that the schema section and figure did not list.	Both tables and the column added to the table list and the schema figure.	Closed (1.0)
I-7	Y. Shnazbayev	Sec. 5.3.7; Fig. 18; Sec. 4.8	inconsistency	Rollover idempotency was keyed on `period_close` in Ledger but on posting a zero Grant in S7, which `postGrant` refuses; the expired amount was derived two ways.	`period_close` is the key and holds the expired amount; S7, Fig. 18, and the Dashboard view read it.	Closed (1.0)

Inspection findings (continued)

ID	Found by	Location	Category	Finding	Resolution	Status
I-8	F. Turopov	Fig. 12	incorrectness	S1's guards sent a Request needing approval but with one unsatisfiable Required line to Waiting; the SRS (**Lifecycle.Evaluate**) sends it to Pending.	Guards corrected: unfit Required lines join the Waitlist while the Request enters Pending.	Closed (1.0)
I-9	J. Nana	Sec. 5.3.2; Fig. 13; Sec. 4.5	infeasibility	S2 had `MatchedAllocator.commit` report an ineligible Candidate and then `toWaiting` write in the same transaction, but allocator beans are `MANDATORY`, so the exception would roll everything back.	The eligibility re-check runs in `evaluateAll` before any commit; `commit` only guards and throws `ALLOCATION_CONFLICT`.	Closed (1.0)
I-10	B. Lujan	Table 13; Sec. 8.4 vs Tables 4, 7	inconsistency	The API mapped a state-conflict code `ILLEGAL_TRANSITION` that no module raises; the modules raise `WRONG_STATE`, and other state codes fell to 422.	`WRONG_STATE` used; all codes describing the object's current state map to 409, the rest to 422.	Closed (1.0)
I-11	Y. Shnazbayev	Sec. 8.4 vs 4.5	inconsistency	An exclusion-constraint violation was answered 409 in one section and 500 in another.	Logged as a bug and answered 409 `ALLOCATION_CONFLICT` in both.	Closed (1.0)
I-12	F. Turopov	Sec. 4.1, 4.8, Table 6 vs Sec. 7, 8	inconsistency	Out-of-scope reads raised `ForbiddenException` in the module specs but 404 in the API and tests.	Out-of-scope reads raise `NotFoundException` (404); `ForbiddenException` is kept for acting on a visible object.	Closed (1.0)
I-13	J. Nana	Sec. 4.7, Table 4 vs 5.1, Table 10, S6	inconsistency	Sec. 4.7 forbade taking a type lock after an account lock, but the waitlist fill locks a waiting Request's other types after the Refund's account lock.	The waitlist fill is named as the one path that may do so, covered by the single retry.	Closed (1.0)
I-14	B. Lujan	Table 8 vs Table 11	inconsistency	The Waiting-to-Pending guard differed between the two transition tables.	Both read: window running, approvals outstanding; unfit Required lines stay on the Waitlist.	Closed (1.0)
I-15	F. Turopov	Fig. 6; Sec. 5.2; Table 10	inconsistency	Escalated appeared both as an approval state and as a flag, which changes what still Outstanding means.	Escalated is a flag; states are OUTSTANDING, APPROVED, REFUSED, CANCELED everywhere.	Closed (1.0)
I-16	Y. Shnazbayev	Fig. 6; Table 10	inconsistency	Occurrence and Metered row states in the class diagram omitted RELEASED and EXPIRED, and a refused Hold went to RELEASED instead of EXPIRED.	States completed in the figure; refused Holds go HELD to EXPIRED.	Closed (1.0)
I-17	B. Lujan	Fig. 5, 6, 8	incompleteness	Class diagrams omitted operations the text relies on (`lapsedWindows`, `shortlist`, `list`, `approvalsAwaiting`, `loansFor`, `withRole`, `managerOf`, `teams`, `periods`).	All added to the diagrams.	Closed (1.0)
I-18	J. Nana	Sec. 8.1 vs 4.1, 4.5, 4.7	ambiguity	Scoped reads took no `Actor`, and a scope taken from the security context would refuse Allocation's and Inventory's own cross-module checks.	A request-scoped `CurrentActor` supplies scope; modules use the unscoped `MemberService.summary` for their own rules.	Closed (1.0)
I-19	Y. Shnazbayev	Sec. 4.7	incompleteness	If a rollover expired granted credits while a Request was Pending, `commitCharges` would throw and roll back the approval with nothing recorded.	`allocate` sends the Request to Waiting with `INSUFFICIENT_BALANCE` and keeps the approval.	Closed (1.0)

Inspection findings (continued)

ID	Found by	Location	Category	Finding	Resolution	Status
I-20	F. Turopov	Fig. 11	incompleteness	The line state machine lacked Held to Bumped (the SRS Bumps lines on Hold) and the system-refused approval after a failed re-check.	Both transitions added.	Closed (1.0)
I-21	J. Nana	Sec. 5.1 vs Table 18	inconsistency	Section 5.1 says the ArchUnit test class rejects an `isolation` attribute on `@Transactional` in Allocation and Requests, but the Architecture row of the test table did not list that check.	The check added to the Architecture test row.	Closed (1.0)